

BIT 2203 – ADVANCED PROGRAMMING

TAKE-AWAY CAT

Name: MARYCYNTHIA GITAU

Registration Number: SCT221-0730/2023

GitHub Repository: <https://github.com/cynthiawgitau68-lab/BIT2203-AdvancedProgramming-CAT>

QUESTION ONE – 6 MARKS

(a) TCP versus UDP

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are transport-layer protocols used for communication between client and server.

TCP

TCP is a connection-oriented and reliable* **BIT 2203 – ADVANCED PROGRAMMING**

TAKE-AWAY CAT

Name: MARYCYNTHIA GITAU

Registration Number: SCT221-0730/2023

GitHub Repository: <https://github.com/cynthiawgitau68-lab/BIT2203-AdvancedProgramming-CAT>

QUESTION ONE – 6 MARKS

(a) TCP versus UDP

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are transport-layer protocols used for communication between client and server.

TCP

TCP is a **connection-oriented and reliable** protocol. Before data is transmitted, a connection is established between the client and server. TCP ensures that data packets arrive, arrive in the correct order, and are retransmitted if they are lost.

Characteristics of TCP:

- * Connection-oriented.
- * Reliable data delivery.
- * Packets are delivered in order.
- * Error detection and retransmission are supported.
- * Slower than UDP because of its reliability mechanisms.

UDP

UDP is a **connectionless and faster** protocol. It sends data without first establishing a connection. It does not guarantee that packets will arrive or that they will arrive in the correct order.

Characteristics of UDP:

- * Connectionless.
- * Faster than TCP.
- * Lower overhead.
- * Does not guarantee delivery.
- * Suitable for applications where speed is more important than perfect reliability.

Scenario where UDP is preferred

UDP would be preferred in **live video conferencing or online gaming**.

For example, in an online game, if one packet containing a player's position is lost, it is normally better to continue receiving newer position updates rather than wait for the old packet to be retransmitted. The small amount of lost information is less important than maintaining real-time communication.

Therefore, UDP is preferred because it provides ****low latency and faster communication****, which is important for real-time applications.

(b) URLParser.java

The given URL is:

``https://api.techstore.com:8443/products/category/electronics?sort=price&limit=20#reviews``

The following program uses Java's ``URL`` class to extract the required components.

URLParser.java

```
```java
import java.net.URL;
import java.util.HashMap;
import java.util.Map;

public class URLParser {

 public static void main(String[] args) {

 try {

 // Create a URL object
 URL url = new URL(
```

```
"https://api.techstore.com:8443/products/category/electronics"
 + "?sort=price&limit=20#reviews"
);
```

```
// Display URL components
System.out.println("Protocol: " + url.getProtocol());
System.out.println("Host: " + url.getHost());
System.out.println("Port: " + url.getPort());
System.out.println("Path: " + url.getPath());
```

```
// Extract and display query parameters
String query = url.getQuery();
```

```
System.out.println("Query Parameters:");
```

```
 if (query != null) {
 String[] parameters = query.split("&");
```

```
Map<String, String> queryParameters = new HashMap<>();
```

```
 for (String parameter : parameters) {
 String[] pair = parameter.split("=", 2);
```

```
 if (pair.length == 2) {
 queryParameters.put(pair[0], pair[1]);
 }
 }
```

```
 for (Map.Entry<String, String> entry : queryParameters.entrySet()) {
 System.out.println(
 entry.getKey() + " = " + entry.getValue()
```

```

);
 }
}

// Display fragment
System.out.println("Fragment: " + url.getRef());

 } catch (Exception e) {
System.out.println("Error parsing URL: " + e.getMessage());
 }
}
}
'''

```

### Expected output

```

'''text
Protocol: https
Host: api.techstore.com
Port: 8443
Path: /products/category/electronics
Query Parameters:
 sort = price
 limit = 20
Fragment: reviews
'''

```

---

# QUESTION TWO – 8 MARKS

## ## (a) PreparedStatement versus Statement

`PreparedStatement` is more secure and efficient than `Statement`, especially when handling login credentials.

### ### Security

A normal `Statement` can be vulnerable to **SQL injection** when user input is directly inserted into an SQL statement.

For example:

```
```java
String sql = "SELECT * FROM students WHERE reg_no = "
            + regNo + " AND password = " + password + """;
```
```

A malicious user could enter SQL commands as part of the input.

`PreparedStatement` separates SQL code from user-supplied values. Values are treated as data rather than executable SQL commands.

### ### Efficiency

A prepared statement can be precompiled by the database and reused with different values. This can improve performance when the same SQL statement is executed multiple times.

### ### PreparedStatement authentication example

```
```java
import java.sql.*;
```

```

        public class StudentLogin {

public static boolean authenticate(String regNo, String password) {

    String url = "jdbc:mysql://localhost:3306/university_db";

        String username = "admin";

        String dbPassword = "secure123";

    String sql = "SELECT * FROM students "
+ "WHERE registration_number = ? AND password = ?";

        try (

            Connection connection =

DriverManager.getConnection(url, username, dbPassword);

            PreparedStatement statement =

connection.prepareStatement(sql)

        ) {

            // Set values safely

            statement.setString(1, regNo);

            statement.setString(2, password);

            try (ResultSet resultSet = statement.executeQuery()) {

                return resultSet.next();

            }

        } catch (SQLException e) {

            System.out.println("Database error: " + e.getMessage());

            return false;

```

```

    }

}

public static void main(String[] args) {

    String registrationNumber = "BIT/001/2026";

    String password = "mypassword";

    if (authenticate(registrationNumber, password)) {

        System.out.println("Login successful.");

    } else {

        System.out.println("Invalid registration number or password.");

    }

}

}

}

...

```

****Note:**** In a real production application, passwords should not be stored as plain text. They should be securely hashed using a password-hashing algorithm such as BCrypt or Argon2.

QUESTION TWO (b) – UniversityDatabase.java

The program below:

1. Connects to `university\_db`.
2. Creates the `courses` table.
3. Inserts three courses.
4. Retrieves courses belonging to the Computer Science department.


```
### UniversityDatabase.java
```

```
``java
```

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class UniversityDatabase {

    // Database connection details
    private static final String URL =
        "jdbc:mysql://localhost:3306/university_db";

    private static final String USER = "admin";
    private static final String PASSWORD = "secure123";

    public static void main(String[] args) {

        // SQL statement for creating the courses table
        String createTableSQL = ""
        CREATE TABLE IF NOT EXISTS courses (
            course_id INT PRIMARY KEY,
            course_name VARCHAR(100),
            credits INT,
            department VARCHAR(50)
        )
        """;
```

```

        // SQL statement for inserting courses
        String insertSQL = ""
            INSERT INTO courses
            (course_id, course_name, credits, department)
            VALUES (?, ?, ?, ?)
            """;

// SQL statement for retrieving Computer Science courses
String selectSQL =
"SELECT * FROM courses WHERE department = ?";

        try (Connection connection =
            DriverManager.getConnection(URL, USER, PASSWORD)) {

            System.out.println("Database connected successfully.");

            // Create table
            try (Statement statement = connection.createStatement()) {

                statement.executeUpdate(createTableSQL);

                System.out.println("Courses table created successfully.");

            }

            // Insert sample courses
            try (PreparedStatement statement =
                connection.prepareStatement(insertSQL)) {

                insertCourse(statement, 101,
                    "Advanced Programming", 3,
                    "Computer Science");
            }
        }
    }
}

```

```

        insertCourse(statement, 102,
            "Database Systems", 3,
            "Computer Science");

        insertCourse(statement, 103,
            "Computer Networks", 3,
            "Computer Science");

        System.out.println("Sample courses inserted.");
    }

    // Retrieve Computer Science courses
    try (PreparedStatement statement =
        connection.prepareStatement(selectSQL)) {

        statement.setString(1, "Computer Science");

        try (ResultSet resultSet = statement.executeQuery()) {

            System.out.println("\nComputer Science Courses:");

            while (resultSet.next()) {

                System.out.println(
                    "ID: " + resultSet.getInt("course_id")
                    + ", Name: " + resultSet.getString("course_name")
                    + ", Credits: " + resultSet.getInt("credits")
                    + ", Department: "
                    + resultSet.getString("department")
                );
            }
        }
    }
}

```

```

        }
    }
}

    } catch (SQLException e) {
System.out.println("Database error: " + e.getMessage());
    }
}

```

// Method used to insert one course

```

private static void insertCourse(
    PreparedStatement statement,
        int id,
        String name,
        int credits,
String department) throws SQLException {

    statement.setInt(1, id);
    statement.setString(2, name);
    statement.setInt(3, credits);
    statement.setString(4, department);

    statement.executeUpdate();
    }
}
...

```

Required JDBC dependency

For the above version, the project uses ****MySQL Connector/J****.

The database should be created before running the program:

```
```sql
CREATE DATABASE university_db;
```

---
```

QUESTION THREE – 8 MARKS

(a) Servlets versus JSP

Servlet

A Servlet is a Java class that runs on a web server and handles HTTP requests and responses.

Servlets are mainly suitable for:

- * Processing requests.
- * Performing business logic.
- * Calling services and databases.
- * Controlling application flow.

JSP

JSP stands for ****JavaServer Pages****. It is mainly used to create the presentation layer of a Java web application.

JSP makes it easier to create HTML pages containing dynamic information.

Comparison

| Feature | Servlet | JSP |
|---------------|--------------------------------|----------------------------|
| ----- | ----- | ----- |
| Main purpose | Request processing and control | Presentation/UI |
| Development | More Java code | Easier for HTML/UI |
| Best MVC role | Controller | View |
| Suitable for | Business/request processing | Displaying dynamic content |
| HTML creation | More difficult | Easier |

In an MVC application, the **Servlet** normally acts as the **Controller**, while the **JSP** acts as the **View**.

When to choose a Servlet

I would choose a Servlet when the application needs to process a request, validate data, call a DAO, perform business logic, and determine which page should be displayed.

When to choose JSP

I would choose JSP when the main requirement is displaying dynamic information to the user, such as displaying a list of products or student records.

QUESTION THREE (b) – Shopping Cart Session Management

Likely cause

The most likely cause is that the shopping cart is not being stored in the user's HTTP session.

HTTP is fundamentally **stateless**, meaning the server does not automatically remember information from one request to another.

For example, if a customer adds a product during one request and the cart is stored only in a local variable, that information can be lost when the customer makes another request.

Two session-management techniques

1. HttpSession

`HttpSession` can be used to store the cart on the server for a particular user session.

Example:

```
```java
HttpSession session = request.getSession();

List<String> cart =
(List<String>) session.getAttribute("cart");

if (cart == null) {
 cart = new ArrayList<>();
}

cart.add("Laptop");

session.setAttribute("cart", cart);
...
```
```

2. Cookies

Cookies can store a small amount of information on the user's browser. A session identifier can also be stored in a cookie so that the server can associate subsequent requests with the correct session.

For a shopping cart, server-side `HttpSession` is generally preferable for storing the actual cart contents.

```
## ShoppingCartServlet.java
```

```
``java
```

```
package com.bit2203.servlet;
```

```
import jakarta.servlet.ServletException;
```

```
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;
```

```
import jakarta.servlet.http.HttpSession;
```

```
import java.io.IOException;
```

```
import java.io.PrintWriter;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
@WebServlet("/shopping-cart")
```

```
public class ShoppingCartServlet extends HttpServlet {
```

```
    private static final String CART_ATTRIBUTE = "cart";
```

```
    // Handles GET requests and displays the cart
```

```
    @Override
```

```
    protected void doGet(
```

```
        HttpServletRequest request,
```



```
        HttpServletResponse response)
        throws ServletException, IOException {

    List<String> cart = getCart(request);

    response.setContentType("text/html");

    PrintWriter out = response.getWriter();

    out.println("<html>");
    out.println("<body>");
    out.println("<h1>Shopping Cart</h1>");

    if (cart.isEmpty()) {

        out.println("<p>Your cart is empty.</p>");

    } else {

        out.println("<ul>");

        for (String item : cart) {
            out.println("<li>" + item + "</li>");
        }

        out.println("</ul>");

    }

    out.println("</body>");
    out.println("</html>");
}
```

```

// Handles POST requests for adding items

@Override
protected void doPost(
    HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {

    String item = request.getParameter("item");

    if (item != null && !item.trim().isEmpty()) {

        HttpSession session = request.getSession();

        List<String> cart = getCart(request);

        cart.add(item);

        session.setAttribute(CART_ATTRIBUTE, cart);
    }

    response.sendRedirect("shopping-cart");
}

// Retrieves the cart from the current user's session
@SuppressWarnings("unchecked")
private List<String> getCart(HttpServletRequest request) {

    HttpSession session = request.getSession();

    List<String> cart =

```

```
(List<String>) session.getAttribute(CART_ATTRIBUTE);
```

```
// Create a new cart if one does not exist
```

```
if (cart == null) {
```

```
    cart = new ArrayList<>();
```

```
    session.setAttribute(CART_ATTRIBUTE, cart);
```

```
}
```

```
    return cart;
```

```
}
```

```
}
```

```
...
```

```
---
```

QUESTION FOUR – 8 MARKS

(a) Microservices Architecture

Microservices architecture is a software architecture in which a large application is divided into several small, independent services.

Each service performs a specific business function and can communicate with other services through APIs or messaging systems.

For example, an e-commerce system could contain:

- * User Service

- * Product Service

- * Payment Service

* Order Service

* Delivery Service

Each service can be developed, deployed, and scaled independently.

Benefits

1. Independent deployment

A single service can be updated or deployed without necessarily redeploying the entire application.

2. Scalability

Individual services can be scaled according to demand.

For example, during a busy shopping period, the Order Service can be given additional resources without scaling every other service.

Challenges

1. Increased complexity

A microservices system contains many services, databases, APIs, and communication channels, making the system more difficult to manage.

2. Network and communication problems

Services communicate over networks, so problems such as latency, network failures, and service unavailability can occur.

Real-world example

****Netflix**** is a well-known example of a company that uses microservices architecture. Its large platform is divided into numerous independent services that support different functions of the streaming platform.

QUESTION FOUR (b) – Dependency Injection

Dependency Injection (DI) is a design technique where an object's dependencies are supplied to it from outside instead of the object creating those dependencies itself.

For example, instead of `UserService` creating a `UserRepository` using `new`, Spring provides the repository to the service.

Benefits of DI

- * Reduces coupling.
- * Makes applications easier to test.
- * Makes components easier to replace.
- * Improves maintainability.
- * Supports clean application architecture.

Constructor Injection

In constructor injection, dependencies are supplied through the class constructor.

Advantage

The dependency can be made `final`, ensuring that the object cannot exist without its required dependency.

Setter Injection

Setter injection supplies dependencies through setter methods.

Advantage

It is useful when a dependency is optional or needs to be changed after object creation.

Comparison

| Constructor Injection | Setter Injection |
|---|------------------------------------|
| ----- | ----- |
| Dependency supplied through constructor | Dependency supplied through setter |
| Good for required dependencies | Good for optional dependencies |
| Supports immutable dependencies | Dependency can be changed |
| Makes dependencies obvious | More flexible after construction |

UserRepository.java

```
```java
package com.bit2203.spring;

import org.springframework.stereotype.Repository;

@Repository
public class UserRepository {

 // Example repository method
 public String findUserById(int userId) {

 return "User with ID: " + userId;
 }
}
```

```
}
```

```
}
```

```
```
```

```
---
```

```
## UserService.java
```

```
```java
```

```
package com.bit2203.spring;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class UserService {
```

```
 private final UserRepository userRepository;
```

```
 // Constructor injection
```

```
 public UserService(UserRepository userRepository) {
```

```
 this.userRepository = userRepository;
```

```
 }
```

```
 // Service method
```

```
 public String getUser(int userId) {
```

```
 return userRepository.findUserById(userId);
```

```
 }
```

```
}
```

```
```
```

AppConfig.java

```java

package com.bit2203.spring;

import org.springframework.context.annotation.ComponentScan;

import org.springframework.context.annotation.Configuration;

@Configuration

@ComponentScan("com.bit2203.spring")

public class AppConfig {

// Spring automatically detects

// @Service and @Repository classes.

}

```

QUESTION FIVE – 6 MARKS

(a) Remote Method Invocation (RMI)

Java RMI, or **Remote Method Invocation**, is a Java technology that allows an object running in one Java Virtual Machine to invoke methods on an object running in another Java Virtual Machine.

RMI allows Java applications running on different computers or processes to communicate as though they were calling methods on local objects.

RMI Registry

The RMI Registry is a naming service.

It stores references to remote objects using names so that clients can locate those objects.

Server

The server:

1. Implements the remote interface.
2. Creates the remote object.
3. Exports the object.
4. Registers the object with the RMI Registry.

Client

The client:

1. Connects to the RMI Registry.
2. Looks up the remote object by name.
3. Obtains a reference to the remote service.
4. Invokes methods on the remote object.

QUESTION FIVE (b) – Java RMI Program

TemperatureConverter.java

```java

import java.rmi.Remote;

```

import java.rmi.RemoteException;

// Remote interface containing methods
// that can be called by the client.
public interface TemperatureConverter extends Remote {

 double celsiusToFahrenheit(double celsius)
 throws RemoteException;

 double fahrenheitToCelsius(double fahrenheit)
 throws RemoteException;

 }
 ...

```

## TemperatureConverterImpl.java

```

```java
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

// Implementation of the remote interface
public class TemperatureConverterImpl
    extends UnicastRemoteObject
    implements TemperatureConverter {

    // Constructor
    public TemperatureConverterImpl()
        throws RemoteException {
        super();
    }

```

```

    }

    // Converts Celsius to Fahrenheit
    @Override
    public double celsiusToFahrenheit(double celsius)
        throws RemoteException {

        return (celsius * 9 / 5) + 32;
    }

    // Converts Fahrenheit to Celsius
    @Override
    public double fahrenheitToCelsius(double fahrenheit)
        throws RemoteException {

        return (fahrenheit - 32) * 5 / 9;
    }
}

```

RMIServer.java

```

```java
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

// RMI server
public class RMIServer {

```

```
public static void main(String[] args) {

 try {

 // Create the remote service
 TemperatureConverter converter =
 new TemperatureConverterImpl();

 // Start the RMI registry on port 1099
 Registry registry =
 LocateRegistry.createRegistry(1099);

 // Bind the remote object to a name
 registry.rebind(
 "TemperatureConverter",
 converter
);

 System.out.println(
 "Temperature Converter RMI Server is running..."
);

 } catch (Exception e) {

 System.out.println(
 "Server error: " + e.getMessage()
);

 e.printStackTrace();
 }
}
```

...

---

## RMIClient.java

```java

import java.rmi.registry.LocateRegistry;

import java.rmi.registry.Registry;

// RMI client

public class RMIClient {

public static void main(String[] args) {

try {

// Locate the RMI registry

Registry registry =

LocateRegistry.getRegistry(

"localhost", 1099

);

// Look up the remote service

TemperatureConverter converter =

(TemperatureConverter)

registry.lookup("TemperatureConverter");

// Convert 25 degrees Celsius

double result =

converter.celsiusToFahrenheit(25);

```

        System.out.println(
            "25°C = " + result + "°F"
        );

    } catch (Exception e) {

        System.out.println(
            "Client error: " + e.getMessage()
        );

        e.printStackTrace();
    }
}
}
...

```

Expected output

```

```text
25°C = 77.0°F
...

```

# QUESTION SIX – 6 MARKS

## (a) Web Container

A **web container**, also called a Servlet container, is a component of a Java web server that manages the execution of Java Servlets.

Examples include Apache Tomcat, Jetty, and other Jakarta Servlet-compatible servers.

### ### Four responsibilities

#### 1. **\*\*Servlet lifecycle management\*\***

The container loads, initializes, executes, and destroys Servlets.

#### 2. **\*\*Request and response management\*\***

It receives HTTP requests and provides them to the appropriate Servlet.

#### 3. **\*\*Session management\*\***

It manages HTTP sessions and helps maintain user state.

#### 4. **\*\*Security and resource management\*\***

It can provide authentication, authorization, URL mapping, and manage Servlet resources.

---

### # QUESTION SIX (b) – Servlet Lifecycle

The main stages of a Servlet lifecycle are:

#### ### 1. Loading and Instantiation

The web container loads the Servlet class and creates an instance of it.

#### ### 2. Initialization – ``init()``

The container calls the ``init()`` method once.

This is used for initialization tasks such as loading configuration information or resources.

### ### 3. Request Processing – `service()`

For every client request, the container calls the `service()` method.

For an `HttpServlet`, the `service()` method determines whether the request is GET, POST, PUT, DELETE, etc., and calls methods such as `doGet()` or `doPost()`.

### ### 4. Destruction – `destroy()`

When the Servlet is removed from service, the container calls `destroy()` once.

This is used to release resources.

---

## LifecycleDemoServlet.java

```
```java
```

```
package com.bit2203.servlet;
```

```
import jakarta.servlet.ServletException;
```

```
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.io.IOException;
```

```
import java.io.PrintWriter;
```

```
@WebServlet("/lifecycle")
```



```

public class LifecycleDemoServlet extends HttpServlet {

    // Called once when the Servlet is initialized
    @Override
    public void init() throws ServletException {

        System.out.println(
            "LifecycleDemoServlet: init() called."
        );
    }

    // Called for each HTTP request
    @Override
    protected void service(
        HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        System.out.println(
            "LifecycleDemoServlet: service() called."
        );

        // Pass the request to the parent implementation.
        super.service(request, response);
    }

    // Handles GET requests
    @Override
    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response)

```

```

throws ServletException, IOException {

    System.out.println(
        "LifecycleDemoServlet: doGet() called."
    );

    response.setContentType("text/html");

    PrintWriter out = response.getWriter();

    out.println("<h1>Servlet Lifecycle Demo</h1>");
    out.println("<p>The Servlet is processing the request.</p>");
    }

// Called once before the Servlet is destroyed
@Override
public void destroy() {

    System.out.println(
        "LifecycleDemoServlet: destroy() called."
    );
}
}
...

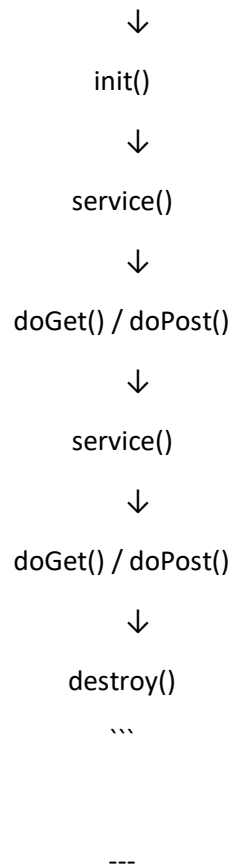
```

Lifecycle sequence

```

```text
Load Servlet
↓
Create Servlet object

```



#### # QUESTION SEVEN – 8 MARKS

##### ## (a) Model-View-Controller (MVC)

MVC stands for **\*\*Model-View-Controller\*\***.

It is a software design pattern that separates an application into three main components.

##### ### 1. Model

The Model represents the application's data and business logic.

Responsibilities include:

- \* Representing data.

- \* Applying business rules.
- \* Communicating with databases.
- \* Managing application data.

Example:

``text

Book

Student

Product

User

...

### ### 2. View

The View is the presentation layer.

It is responsible for displaying information to the user.

Examples include:

- \* JSP pages.
- \* HTML pages.
- \* User interfaces.

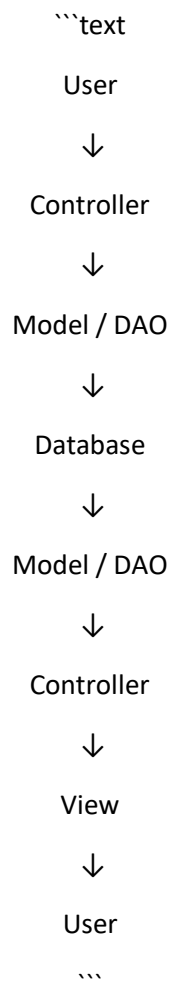
### ### 3. Controller

The Controller receives requests from the user and determines what should happen.

Responsibilities include:

- \* Receiving HTTP requests.
- \* Processing input.
- \* Calling the Model/DAO.
- \* Selecting the appropriate View.
- \* Passing data to the View.

#### ### MVC flow



#### ### Real-world example

An **online shopping system** can use MVC.

For example:

- \* \*\*Model:\*\* Product, Order and Customer classes.
- \* \*\*View:\*\* JSP pages displaying products.
- \* \*\*Controller:\*\* Servlets handling product and order requests.
- \* \*\*Database:\*\* Stores product, customer and order information.

Many enterprise Java web applications use the MVC approach because it separates presentation, processing, and data management.

---

## # QUESTION SEVEN (b) – Library Management System

### ## Model

The Model represents library data.

For this system, the `Book` class represents individual books.

It can contain:

- \* Book ID.
- \* Title.
- \* Author.
- \* ISBN.

### ## DAO

The Data Access Object, `BookDAO`, communicates with the database and retrieves books.

### ## Controller

`BookController` is a Servlet that:

1. Receives the request.
2. Calls `BookDAO`.
3. Retrieves the books.
4. Places them into the request.
5. Forwards the request to the JSP.

## View

`bookList.jsp` displays the books to the user.

---

# Book.java

```java

package com.bit2203.library.model;

// Model class representing a book

public class Book {

private int id;

private String title;

private String author;

private String isbn;

// Constructor

public Book(int id, String title, String author, String isbn) {

this.id = id;

```
        this.title = title;
    this.author = author;
    this.isbn = isbn;
    }
```

```
    // Getters
```

```
    public int getId() {
        return id;
    }
```

```
    public String getTitle() {
        return title;
    }
```

```
    public String getAuthor() {
        return author;
    }
```

```
    public String getIsbn() {
        return isbn;
    }
```

```
    // Setters
```

```
    public void setId(int id) {
        this.id = id;
    }
```

```
    public void setTitle(String title) {
        this.title = title;
    }
```



```
public void setAuthor(String author) {  
    this.author = author;  
}
```

```
public void setIsbn(String isbn) {  
    this.isbn = isbn;  
}  
}
```

```
...
```

```
---
```

```
# BookDAO.java
```

```
```java
```

```
package com.bit2203.library.dao;
```

```
import com.bit2203.library.model.Book;
```

```
import java.sql.Connection;
```

```
import java.sql.DriverManager;
```

```
import java.sql.PreparedStatement;
```

```
import java.sql.ResultSet;
```

```
import java.sql.SQLException;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
// DAO responsible for accessing book data
```

```
public class BookDAO {
```

```
private static final String URL =
 "jdbc:mysql://localhost:3306/library_db";

private static final String USER = "admin";
private static final String PASSWORD = "secure123";

// Retrieves all books from the database
public List<Book> getAllBooks() {

 List<Book> books = new ArrayList<>();

 String sql =
 "SELECT id, title, author, isbn FROM books";

 try (
 Connection connection =
 DriverManager.getConnection(
 URL, USER, PASSWORD);

 PreparedStatement statement =
 connection.prepareStatement(sql);

 ResultSet resultSet =
 statement.executeQuery()
) {

 while (resultSet.next()) {

 Book book = new Book(
 resultSet.getInt("id"),
 resultSet.getString("title"),
```

```

 resultSet.getString("author"),
 resultSet.getString("isbn")
);

 books.add(book);
}

} catch (SQLException e) {

 System.out.println(
 "Database error: " + e.getMessage()
);
}

return books;
}
}
...

```

# BookController.java

```

```java

package com.bit2203.library.controller;

import com.bit2203.library.dao.BookDAO;
import com.bit2203.library.model.Book;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.io.IOException;
import java.util.List;
```

```
// Controller Servlet for handling book requests
```

```
@WebServlet("/books")
```

```
public class BookController extends HttpServlet {
```

```
    private BookDAO bookDAO;
```

```
    // Create DAO when Servlet is initialized
```

```
    @Override
```

```
    public void init() {
```

```
        bookDAO = new BookDAO();
```

```
    }
```

```
    // Handles request for displaying all books
```

```
    @Override
```

```
    protected void doGet(
```

```
        HttpServletRequest request,
```

```
        HttpServletResponse response)
```

```
        throws ServletException, IOException {
```

```
        // Retrieve books from DAO
```

```
        List<Book> books = bookDAO.getAllBooks();
```

```
        // Place books in request scope
```

```
request.setAttribute("books", books);
```

```
// Forward request to JSP View
```

```
request.getRequestDispatcher(
```

```
    "/bookList.jsp"
```

```
).forward(request, response);
```

```
    }
```

```
    }
```

```
    ...
```

```
---
```

```
# bookList.jsp
```

```
```.jsp
```

```
<%@ page contentType="text/html; charset=UTF-8" %>
```

```
<%@ page import="java.util.List" %>
```

```
<%@ page import="com.bit2203.library.model.Book" %>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Library Books</title>
```

```
</head>
```

```
<body>
```

```
<h1>Library Books</h1>
```

```
<%
```

```
List<Book> books =
```

```
(List<Book>) request.getAttribute("books");
```

```
%>
```

```
<table border="1" cellpadding="8">
```

```
<tr>
```

```
<th>ID</th>
```

```
<th>Title</th>
```

```
<th>Author</th>
```

```
<th>ISBN</th>
```

```
</tr>
```

```
<%
```

```
if (books != null && !books.isEmpty()) {
```

```
 for (Book book : books) {
```

```
%>
```

```
<tr>
```

```
<td><%= book.getId() %></td>
```

```
<td><%= book.getTitle() %></td>
```

```
<td><%= book.getAuthor() %></td>
```

```
<td><%= book.getIsbn() %></td>
```

```
</tr>
```

```
<%
```

```
}
```

```
} else {
```

```
%>
```

```
<tr>
<td colspan="4">
 No books found.
</td>
</tr>
```

```
<%
}
%>
```

```
</table>
```

```
</body>
```

```
</html>
```

```
...
```

```

```

# BONUS / EXTRA CREDIT – 2 MARKS

## ## Inversion of Control (IoC)

Inversion of Control (IoC) is a principle in which the responsibility for creating and managing objects is transferred from the application code to a framework or container.

In traditional programming, a class may create its own dependencies:

```
```java
UserRepository repository = new UserRepository();
...
```
```

With Spring IoC, the Spring container creates and manages the object and supplies its dependencies.

### ### Relationship between IoC and DI

Dependency Injection is one of the main techniques used to implement IoC.

For example:

```
```java
@Service
public class UserService {

    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }
}
```
```

Spring creates `UserRepository` and injects it into `UserService`.

### ### Importance of IoC

IoC is important because it:

- \* Reduces coupling.
- \* Improves testability.
- \* Makes components reusable.
- \* Simplifies object management.
- \* Makes large enterprise applications easier to maintain.



Therefore, IoC is a fundamental principle in modern enterprise Java development because frameworks such as Spring can manage object creation, dependencies, configuration, and application components.

---

## # ADDITIONAL QUESTIONS FOR EXTRA PRACTICE

### ## 1. Stateful versus Stateless Session Management

#### ### Stateful session management

Stateful session management means the server maintains information about a user's previous interactions.

**Example:** An online shopping cart where the server stores the user's cart in an `HttpSession`.

#### ### Stateless session management

Stateless communication means each request contains enough information for the server to process it without depending on previously stored server-side state.

**Example:** REST APIs using tokens such as JWTs for authentication.

#### ### Comparison

| Stateful                           | Stateless                              |  |
|------------------------------------|----------------------------------------|--|
| -----                              | -----                                  |  |
| Server maintains user state        | Server does not maintain session state |  |
| Uses more server resources         | Easier to scale horizontally           |  |
| Example: HttpSession shopping cart | Example: Token-based REST API          |  |

---

## # 2. JAR FILE

JAR stands for **\*\*Java Archive\*\***.

A JAR file packages Java classes, resources, configuration files, and metadata into one compressed file.

For example:

```
```text
MyApplication.jar
```
```

A JAR can be made executable by specifying the application's main class in its manifest.

### ### Example MANIFEST.MF

```
```text
Manifest-Version: 1.0
Main-Class: com.example.Main
```
```

The JAR can then be executed using:

```
```bash
java -jar MyApplication.jar
```
```

---

### # 3. MONOLITHIC VERSUS MICROSERVICES

A monolithic application is built as one large application where the components are packaged and deployed together.

Microservices architecture divides the application into multiple smaller and independently deployable services.

| Monolithic                                | Microservices                          |  |
|-------------------------------------------|----------------------------------------|--|
| -----                                     | -----                                  |  |
| One large application                     | Multiple smaller services              |  |
| Usually deployed together                 | Services can be deployed independently |  |
| Simpler initially                         | More complex to manage                 |  |
| Scaling may involve the whole application | Individual services can be scaled      |  |
| Easier for small applications             | Suitable for large and complex systems |  |

#### ### When to choose monolithic

A monolithic architecture is suitable when:

- \* The application is small.
- \* The development team is small.
- \* Simplicity is important.
- \* The application does not require independent scaling.

#### ### When to choose microservices

Microservices are suitable when:

- \* The application is large.

- \* Different teams need to work independently.
- \* Different components need independent scaling.
- \* Independent deployment is important.

---

## # GITHUB REPOSITORY STRUCTURE

Your repository should be named exactly:

```
``text
BIT2203-AdvancedProgramming-CAT
``
```

A suitable structure is:

```
``text
BIT2203-AdvancedProgramming-CAT/
|
├─ README.md
|
├─ src/
| |
| └─ question1/
| | └─ URLParser.java
| |
| └─ question2/
| | └─ StudentLogin.java
| └─ UniversityDatabase.java
|
└─ question3/
```

```

| | └─ ShoppingCartServlet.java
| |
| | └─ question4/
| | └─ UserRepository.java
| | └─ UserService.java
| | └─ AppConfig.java
| |
| | └─ question5/
| | └─ TemperatureConverter.java
| | └─ TemperatureConverterImpl.java
| | └─ RMIServer.java
| | └─ RMIClient.java
| |
| | └─ question6/
| | └─ LifecycleDemoServlet.java
| |
| | └─ question7/
| | └─ Book.java
| | └─ BookDAO.java
| | └─ BookController.java
| |
| | └─ web/
| | └─ bookList.jsp
| |
| | ...
| |
| | ---
| |
| | # README.md

```

Your repository should contain a README explaining the programs.

```markdown

BIT2203-AdvancedProgramming-CAT

Student Information

Name: YOUR NAME

Registration Number: YOUR REGISTRATION NUMBER

Description

This repository contains Java programs developed for the BIT 2203
Advanced Programming Take-Away CAT.

Programs

Question 1 - URLParser

URLParser.java demonstrates how Java's URL class can be used to extract
URL components including the protocol, host, port, path, query parameters
and fragment.

Question 2 - JDBC

StudentLogin.java demonstrates secure student authentication using
PreparedStatement.

UniversityDatabase.java connects to the university_db database, creates
the courses table, inserts sample courses and retrieves Computer Science
courses.

Question 3 - Shopping Cart

ShoppingCartServlet.java demonstrates HTTP session management using HttpSession to maintain a customer's shopping cart.

Question 4 - Spring Dependency Injection

UserRepository.java provides the repository dependency.

UserService.java demonstrates constructor dependency injection.

AppConfig.java configures component scanning in Spring.

Question 5 - Java RMI

TemperatureConverter.java is the remote interface.

TemperatureConverterImpl.java implements the remote service.

RMIServer.java starts the RMI registry and binds the service.

RMIClient.java looks up the service and converts 25 degrees Celsius to Fahrenheit.

Question 6 - Servlet Lifecycle

LifecycleDemoServlet.java demonstrates the Servlet lifecycle methods including init(), service(), doGet() and destroy().

Question 7 - MVC Library System

Book.java represents the Model.

BookDAO.java accesses book data.

BookController.java acts as the Servlet Controller.

bookList.jsp acts as the View and displays the books.

Requirements

- Java JDK
- MySQL Server
- MySQL Connector/J
- Apache Tomcat 10 or later for Jakarta Servlet applications
- Spring Framework for the Spring example
- An IDE such as IntelliJ IDEA, Eclipse or NetBeans

Database Setup

Create the required database using:

```
CREATE DATABASE university_db;
```

For the library example:

```
CREATE DATABASE library_db;
```

Update database credentials in the Java files if necessary.

Running the RMI Program

Compile the RMI classes:

```
javac *.java
```

Start the server:

```
java RMIServer
```

In another terminal, run:

```
java RMIClient
```

Expected result:

25°C = 77.0°F

Servlet Programs

Deploy the Servlet project to a Jakarta Servlet-compatible web container
such as Apache Tomcat 10 or later.

GitHub

All Java source files and configuration files are included in this
repository.

...

FINAL SUBMISSION CHECKLIST

Before submitting, ensure you have:

- * ☐ Created a ****public**** GitHub repository.
- * ☐ Named it `BIT2203-AdvancedProgramming-CAT`.
- * ☐ Added all required `.java` files.
- * ☐ Added `README.md`.
- * ☐ Added the JSP file.
- * ☐ Added configuration files where applicable.
- * ☐ Added comments to your Java programs.
- * ☐ Tested the programs.
- * ☐ Pushed all files to GitHub.
- * ☐ Added your real name to the document.
- * ☐ Added your registration number.
- * ☐ Added your GitHub repository link.
- * ☐ Saved the typed answers as PDF or Word.
- * ☐ Checked that the GitHub repository is ****public****.
- * ☐ Verified that the GitHub link actually opens before submission.

Required Java files

```text

URLParser.java

StudentLogin.java

UniversityDatabase.java

ShoppingCartServlet.java

UserRepository.java

UserService.java

AppConfig.java

TemperatureConverter.java

TemperatureConverterImpl.java

RMIServer.java

RMIClient.java  
LifecycleDemoServlet.java  
Book.java  
BookDAO.java  
BookController.java  
bookList.jsp  
README.md  
...

**\*\*Important implementation note:\*\*** The Servlet examples above use ``jakarta.servlet.*``, which is appropriate for modern Servlet containers such as Tomcat 10+. If your lecturer/lab uses Tomcat 9 or an older Java EE setup, the imports may need to use ``javax.servlet.*`` instead. **BIT 2203 – ADVANCED PROGRAMMING**

#### TAKE-AWAY CAT – COMPLETE ANSWERS

Name: \_\_\_\_\_

Registration Number: \_\_\_\_\_

GitHub Repository: <https://github.com/YOUR-USERNAME/BIT2203-AdvancedProgramming-CAT>

---

#### QUESTION ONE – 6 MARKS

##### (a) TCP versus UDP

**TCP (Transmission Control Protocol)** and **UDP (User Datagram Protocol)** are transport-layer protocols used for communication between client and server.

##### TCP

TCP is a **connection-oriented and reliable** protocol. Before data is transmitted, a connection is established between the client and server. TCP ensures that data packets arrive, arrive in the correct order, and are retransmitted if they are lost.

##### Characteristics of TCP:

- Connection-oriented.
- Reliable data delivery.
- Packets are delivered in order.
- Error detection and retransmission are supported.
- Slower than UDP because of its reliability mechanisms.

##### UDP

UDP is a **connectionless and faster** protocol. It sends data without first establishing a connection. It does not guarantee that packets will arrive or that they will arrive in the correct order.

**Characteristics of UDP:**

- Connectionless.
- Faster than TCP.
- Lower overhead.
- Does not guarantee delivery.
- Suitable for applications where speed is more important than perfect reliability.

**Scenario where UDP is preferred**

UDP would be preferred in **live video conferencing or online gaming**.

For example, in an online game, if one packet containing a player's position is lost, it is normally better to continue receiving newer position updates rather than wait for the old packet to be retransmitted. The small amount of lost information is less important than maintaining real-time communication.

Therefore, UDP is preferred because it provides **low latency and faster communication**, which is important for real-time applications.

---

**(b) URLParser.java**

The given URL is:

<https://api.techstore.com:8443/products/category/electronics?sort=price&limit=20#reviews>

The following program uses Java's URL class to extract the required components.

**URLParser.java**

```
import java.net.URL;

import java.util.HashMap;

import java.util.Map;

public class URLParser {

 public static void main(String[] args) {

 try {

 // Create a URL object

 URL url = new URL(
```

```
"https://api.techstore.com:8443/products/category/electronics"
 + "?sort=price&limit=20#reviews"
);
```

```
 // Display URL components
 System.out.println("Protocol: " + url.getProtocol());
 System.out.println("Host: " + url.getHost());
 System.out.println("Port: " + url.getPort());
 System.out.println("Path: " + url.getPath());
```

```
 // Extract and display query parameters
 String query = url.getQuery();
```

```
 System.out.println("Query Parameters:");
```

```
 if (query != null) {
 String[] parameters = query.split("&");
```

```
 Map<String, String> queryParameters = new HashMap<>();
```

```
 for (String parameter : parameters) {
 String[] pair = parameter.split("=", 2);
```

```
 if (pair.length == 2) {
 queryParameters.put(pair[0], pair[1]);
 }
 }
 }
```

```
 for (Map.Entry<String, String> entry : queryParameters.entrySet()) {
 System.out.println(
 entry.getKey() + " = " + entry.getValue()
```

```

);
 }
}

// Display fragment
System.out.println("Fragment: " + url.getRef());

 } catch (Exception e) {
System.out.println("Error parsing URL: " + e.getMessage());
 }
}
}

```

#### **Expected output**

```

Protocol: https
Host: api.techstore.com
Port: 8443
Path: /products/category/electronics
Query Parameters:
 sort = price
 limit = 20
Fragment: reviews

```

### **QUESTION TWO – 8 MARKS**

#### **(a) PreparedStatement versus Statement**

PreparedStatement is more secure and efficient than Statement, especially when handling login credentials.

#### **Security**

A normal Statement can be vulnerable to **SQL injection** when user input is directly inserted into an SQL statement.

For example:

```

String sql = "SELECT * FROM students WHERE reg_no = '"
 + regNo + "' AND password = '" + password + "'";

```

A malicious user could enter SQL commands as part of the input.

PreparedStatement separates SQL code from user-supplied values. Values are treated as data rather than executable SQL commands.

### **Efficiency**

A prepared statement can be precompiled by the database and reused with different values. This can improve performance when the same SQL statement is executed multiple times.

### **PreparedStatement authentication example**

```
import java.sql.*;

public class StudentLogin {

 public static boolean authenticate(String regNo, String password) {

 String url = "jdbc:mysql://localhost:3306/university_db";

 String username = "admin";

 String dbPassword = "secure123";

 String sql = "SELECT * FROM students "
 + "WHERE registration_number = ? AND password = ?";

 try {

 Connection connection =
 DriverManager.getConnection(url, username, dbPassword);

 PreparedStatement statement =
 connection.prepareStatement(sql)

) {

 // Set values safely

 statement.setString(1, regNo);

 statement.setString(2, password);
```

```

 try (ResultSet resultSet = statement.executeQuery()) {

 return resultSet.next();

 }

 } catch (SQLException e) {
System.out.println("Database error: " + e.getMessage());
 return false;
 }
 }

 public static void main(String[] args) {

 String registrationNumber = "BIT/001/2026";
 String password = "mypassword";

 if (authenticate(registrationNumber, password)) {
 System.out.println("Login successful.");
 } else {
System.out.println("Invalid registration number or password.");
 }
 }
}

```

**Note:** In a real production application, passwords should not be stored as plain text. They should be securely hashed using a password-hashing algorithm such as BCrypt or Argon2.

---

### QUESTION TWO (b) – UniversityDatabase.java

The program below:

1. Connects to university\_db.
2. Creates the courses table.
3. Inserts three courses.



4. Retrieves courses belonging to the Computer Science department.

```
UniversityDatabase.java

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class UniversityDatabase {

 // Database connection details
 private static final String URL =
 "jdbc:mysql://localhost:3306/university_db";

 private static final String USER = "admin";
 private static final String PASSWORD = "secure123";

 public static void main(String[] args) {

 // SQL statement for creating the courses table
 String createTableSQL = ""
 CREATE TABLE IF NOT EXISTS courses (
 course_id INT PRIMARY KEY,
 course_name VARCHAR(100),
 credits INT,
 department VARCHAR(50)
)
 """,

 // SQL statement for inserting courses
```

```

 String insertSQL = ""
 INSERT INTO courses
 (course_id, course_name, credits, department)
 VALUES (?, ?, ?, ?)
 """,

// SQL statement for retrieving Computer Science courses
 String selectSQL =
 "SELECT * FROM courses WHERE department = ?";

 try (Connection connection =
 DriverManager.getConnection(URL, USER, PASSWORD)) {

 System.out.println("Database connected successfully.");

 // Create table
 try (Statement statement = connection.createStatement()) {

 statement.executeUpdate(createTableSQL);

 System.out.println("Courses table created successfully.");

 }

 // Insert sample courses
 try (PreparedStatement statement =
 connection.prepareStatement(insertSQL)) {

 insertCourse(statement, 101,
 "Advanced Programming", 3,
 "Computer Science");

```

```

 insertCourse(statement, 102,
 "Database Systems", 3,
 "Computer Science");

 insertCourse(statement, 103,
 "Computer Networks", 3,
 "Computer Science");

 System.out.println("Sample courses inserted.");
 }

 // Retrieve Computer Science courses
 try (PreparedStatement statement =
 connection.prepareStatement(selectSQL)) {

 statement.setString(1, "Computer Science");

 try (ResultSet resultSet = statement.executeQuery()) {

 System.out.println("\nComputer Science Courses:");

 while (resultSet.next()) {

 System.out.println(
 "ID: " + resultSet.getInt("course_id")
 + ", Name: " + resultSet.getString("course_name")
 + ", Credits: " + resultSet.getInt("credits")
 + ", Department: "
 + resultSet.getString("department")
);
 }
 }
 }

```

```

 }
 }

 } catch (SQLException e) {
 System.out.println("Database error: " + e.getMessage());
 }
}

```

// Method used to insert one course

```

private static void insertCourse(
 PreparedStatement statement,
 int id,
 String name,
 int credits,
 String department) throws SQLException {

 statement.setInt(1, id);
 statement.setString(2, name);
 statement.setInt(3, credits);
 statement.setString(4, department);

 statement.executeUpdate();
}
}

```

#### **Required JDBC dependency**

For the above version, the project uses **MySQL Connector/J**.

The database should be created before running the program:

```
CREATE DATABASE university_db;
```

---

### **QUESTION THREE – 8 MARKS**

#### **(a) Servlets versus JSP**

## Servlet

A Servlet is a Java class that runs on a web server and handles HTTP requests and responses.

Servlets are mainly suitable for:

- Processing requests.
- Performing business logic.
- Calling services and databases.
- Controlling application flow.

## JSP

JSP stands for **JavaServer Pages**. It is mainly used to create the presentation layer of a Java web application.

JSP makes it easier to create HTML pages containing dynamic information.

## Comparison

| Feature       | Servlet                        | JSP                        |
|---------------|--------------------------------|----------------------------|
| Main purpose  | Request processing and control | Presentation/UI            |
| Development   | More Java code                 | Easier for HTML/UI         |
| Best MVC role | Controller                     | View                       |
| Suitable for  | Business/request processing    | Displaying dynamic content |
| HTML creation | More difficult                 | Easier                     |

In an MVC application, the **Servlet normally acts as the Controller**, while the **JSP acts as the View**.

## When to choose a Servlet

I would choose a Servlet when the application needs to process a request, validate data, call a DAO, perform business logic, and determine which page should be displayed.

## When to choose JSP

I would choose JSP when the main requirement is displaying dynamic information to the user, such as displaying a list of products or student records.

---

## QUESTION THREE (b) – Shopping Cart Session Management

### Likely cause

The most likely cause is that the shopping cart is not being stored in the user's HTTP session.

HTTP is fundamentally **stateless**, meaning the server does not automatically remember information from one request to another.

For example, if a customer adds a product during one request and the cart is stored only in a local variable, that information can be lost when the customer makes another request.

## **Two session-management techniques**

### **1. HttpSession**

HttpSession can be used to store the cart on the server for a particular user session.

Example:

```
HttpSession session = request.getSession();
```

```
List<String> cart =
(List<String>) session.getAttribute("cart");
```

```
if (cart == null) {
 cart = new ArrayList<>();
}
```

```
cart.add("Laptop");
```

```
session.setAttribute("cart", cart);
```

### **2. Cookies**

Cookies can store a small amount of information on the user's browser. A session identifier can also be stored in a cookie so that the server can associate subsequent requests with the correct session.

For a shopping cart, server-side HttpSession is generally preferable for storing the actual cart contents.

---

### **ShoppingCartServlet.java**

```
package com.bit2203.servlet;
```

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```

import jakarta.servlet.http.HttpSession;

import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;

@WebServlet("/shopping-cart")
public class ShoppingCartServlet extends HttpServlet {

 private static final String CART_ATTRIBUTE = "cart";

 // Handles GET requests and displays the cart
 @Override
 protected void doGet(
 HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {

 List<String> cart = getCart(request);

 response.setContentType("text/html");

 PrintWriter out = response.getWriter();

 out.println("<html>");
 out.println("<body>");
 out.println("<h1>Shopping Cart</h1>");

 if (cart.isEmpty()) {

```

```

 out.println("<p>Your cart is empty.</p>");

 } else {

 out.println("");

 for (String item : cart) {
 out.println("" + item + "");
 }

 out.println("");

 }

 out.println("</body>");
 out.println("</html>");
 }

// Handles POST requests for adding items
@Override
protected void doPost(
 HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {

 String item = request.getParameter("item");

 if (item != null && !item.trim().isEmpty()) {

 HttpSession session = request.getSession();

 List<String> cart = getCart(request);

```



```

 cart.add(item);

 session.setAttribute(CART_ATTRIBUTE, cart);
 }

 response.sendRedirect("shopping-cart");
}

// Retrieves the cart from the current user's session
@SuppressWarnings("unchecked")
private List<String> getCart(HttpServletRequest request) {

 HttpSession session = request.getSession();

 List<String> cart =
 (List<String>) session.getAttribute(CART_ATTRIBUTE);

 // Create a new cart if one does not exist
 if (cart == null) {

 cart = new ArrayList<>();

 session.setAttribute(CART_ATTRIBUTE, cart);
 }

 return cart;
}
}

```

---

**QUESTION FOUR – 8 MARKS**

### **(a) Microservices Architecture**

Microservices architecture is a software architecture in which a large application is divided into several small, independent services.

Each service performs a specific business function and can communicate with other services through APIs or messaging systems.

For example, an e-commerce system could contain:

- User Service
- Product Service
- Payment Service
- Order Service
- Delivery Service

Each service can be developed, deployed, and scaled independently.

#### **Benefits**

##### **1. Independent deployment**

A single service can be updated or deployed without necessarily redeploying the entire application.

##### **2. Scalability**

Individual services can be scaled according to demand.

For example, during a busy shopping period, the Order Service can be given additional resources without scaling every other service.

#### **Challenges**

##### **1. Increased complexity**

A microservices system contains many services, databases, APIs, and communication channels, making the system more difficult to manage.

##### **2. Network and communication problems**

Services communicate over networks, so problems such as latency, network failures, and service unavailability can occur.

#### **Real-world example**

**Netflix** is a well-known example of a company that uses microservices architecture. Its large platform is divided into numerous independent services that support different functions of the streaming platform.

---

### **QUESTION FOUR (b) – Dependency Injection**

Dependency Injection (DI) is a design technique where an object's dependencies are supplied to it from outside instead of the object creating those dependencies itself.

For example, instead of UserService creating a UserRepository using new, Spring provides the repository to the service.

### Benefits of DI

- Reduces coupling.
- Makes applications easier to test.
- Makes components easier to replace.
  - Improves maintainability.
- Supports clean application architecture.

### Constructor Injection

In constructor injection, dependencies are supplied through the class constructor.

#### Advantage

The dependency can be made final, ensuring that the object cannot exist without its required dependency.

### Setter Injection

Setter injection supplies dependencies through setter methods.

#### Advantage

It is useful when a dependency is optional or needs to be changed after object creation.

### Comparison

#### Constructor Injection

#### Setter Injection

Dependency supplied through constructor      Dependency supplied through setter

Good for required dependencies

Good for optional dependencies

Supports immutable dependencies

Dependency can be changed

Makes dependencies obvious

More flexible after construction

---

### UserRepository.java

```
package com.bit2203.spring;
```

```
import org.springframework.stereotype.Repository;
```

```
@Repository
```

```
public class UserRepository {
```

```
// Example repository method
public String findUserById(int userId) {

 return "User with ID: " + userId;

}

}
```

---

### **UserService.java**

```
package com.bit2203.spring;

import org.springframework.stereotype.Service;

@Service
public class UserService {

 private final UserRepository userRepository;

 // Constructor injection
 public UserService(UserRepository userRepository) {
 this.userRepository = userRepository;
 }

 // Service method
 public String getUser(int userId) {

 return userRepository.findUserById(userId);

 }

}
```

---

### **AppConfig.java**

```
package com.bit2203.spring;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.bit2203.spring")
public class AppConfig {

 // Spring automatically detects
 // @Service and @Repository classes.

}
```

---

### QUESTION FIVE – 6 MARKS

#### (a) Remote Method Invocation (RMI)

Java RMI, or **Remote Method Invocation**, is a Java technology that allows an object running in one Java Virtual Machine to invoke methods on an object running in another Java Virtual Machine.

RMI allows Java applications running on different computers or processes to communicate as though they were calling methods on local objects.

#### RMI Registry

The RMI Registry is a naming service.

It stores references to remote objects using names so that clients can locate those objects.

#### Server

The server:

1. Implements the remote interface.
2. Creates the remote object.
3. Exports the object.
4. Registers the object with the RMI Registry.

#### Client

The client:

1. Connects to the RMI Registry.
2. Looks up the remote object by name.

3. Obtains a reference to the remote service.
4. Invokes methods on the remote object.

---

#### **QUESTION FIVE (b) – Java RMI Program**

##### **TemperatureConverter.java**

```
import java.rmi.Remote;
import java.rmi.RemoteException;

// Remote interface containing methods
// that can be called by the client.
public interface TemperatureConverter extends Remote {

 double celsiusToFahrenheit(double celsius)
 throws RemoteException;

 double fahrenheitToCelsius(double fahrenheit)
 throws RemoteException;

}
```

---

##### **TemperatureConverterImpl.java**

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

// Implementation of the remote interface
public class TemperatureConverterImpl
 extends UnicastRemoteObject
 implements TemperatureConverter {

 // Constructor
 public TemperatureConverterImpl()
 throws RemoteException {
```

```

 super();
 }

 // Converts Celsius to Fahrenheit
 @Override
 public double celsiusToFahrenheit(double celsius)
 throws RemoteException {

 return (celsius * 9 / 5) + 32;
 }

 // Converts Fahrenheit to Celsius
 @Override
 public double fahrenheitToCelsius(double fahrenheit)
 throws RemoteException {

 return (fahrenheit - 32) * 5 / 9;
 }
}

```

---

### **RMI Server.java**

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

// RMI server
public class RMIServer {

 public static void main(String[] args) {

 try {

 // Create the remote service

```

```

 TemperatureConverter converter =
 new TemperatureConverterImpl();

 // Start the RMI registry on port 1099
 Registry registry =
 LocateRegistry.createRegistry(1099);

 // Bind the remote object to a name
 registry.rebind(
 "TemperatureConverter",
 converter
);

 System.out.println(
 "Temperature Converter RMI Server is running..."
);

 } catch (Exception e) {

 System.out.println(
 "Server error: " + e.getMessage()
);

 e.printStackTrace();
 }
}
}
}

```

---

#### **RMIClient.java**

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

```



```

 // RMI client
 public class RMIClient {

 public static void main(String[] args) {

 try {

 // Locate the RMI registry
 Registry registry =
 LocateRegistry.getRegistry(
 "localhost", 1099
);

 // Look up the remote service
 TemperatureConverter converter =
 (TemperatureConverter)
 registry.lookup("TemperatureConverter");

 // Convert 25 degrees Celsius
 double result =
 converter.celsiusToFahrenheit(25);

 System.out.println(
 "25°C = " + result + "°F"
);

 } catch (Exception e) {

 System.out.println(
 "Client error: " + e.getMessage()
);
 }
 }
 }
 }
}

```

```
e.printStackTrace();
 }
}
}
```

**Expected output**

25°C = 77.0°F

---

**QUESTION SIX – 6 MARKS**

**(a) Web Container**

A **web container**, also called a Servlet container, is a component of a Java web server that manages the execution of Java Servlets.

Examples include Apache Tomcat, Jetty, and other Jakarta Servlet-compatible servers.

**Four responsibilities**

**1. Servlet lifecycle management**

The container loads, initializes, executes, and destroys Servlets.

**2. Request and response management**

It receives HTTP requests and provides them to the appropriate Servlet.

**3. Session management**

It manages HTTP sessions and helps maintain user state.

**4. Security and resource management**

It can provide authentication, authorization, URL mapping, and manage Servlet resources.

---

**QUESTION SIX (b) – Servlet Lifecycle**

The main stages of a Servlet lifecycle are:

**1. Loading and Instantiation**

The web container loads the Servlet class and creates an instance of it.

**2. Initialization – init()**

The container calls the init() method once.

This is used for initialization tasks such as loading configuration information or resources.

**3. Request Processing – service()**

For every client request, the container calls the service() method.

For an HttpServlet, the service() method determines whether the request is GET, POST, PUT, DELETE, etc., and calls methods such as doGet() or doPost().

#### 4. Destruction – destroy()

When the Servlet is removed from service, the container calls destroy() once.

This is used to release resources.

---

#### LifecycleDemoServlet.java

```
package com.bit2203.servlet;
```

```
import jakarta.servlet.ServletException;
```

```
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.io.IOException;
```

```
import java.io.PrintWriter;
```

```
@WebServlet("/lifecycle")
```

```
public class LifecycleDemoServlet extends HttpServlet {
```

```
 // Called once when the Servlet is initialized
```

```
 @Override
```

```
 public void init() throws ServletException {
```

```
 System.out.println(
```

```
 "LifecycleDemoServlet: init() called."
```

```
);
```

```
 }
```

```
 // Called for each HTTP request
```

```

 @Override

 protected void service(

 HttpServletRequest request,

 HttpServletResponse response)

 throws ServletException, IOException {

 System.out.println(

 "LifecycleDemoServlet: service() called."

);

 // Pass the request to the parent implementation.

 super.service(request, response);

 }

 // Handles GET requests

 @Override

 protected void doGet(

 HttpServletRequest request,

 HttpServletResponse response)

 throws ServletException, IOException {

 System.out.println(

 "LifecycleDemoServlet: doGet() called."

);

 response.setContentType("text/html");

 PrintWriter out = response.getWriter();

 out.println("<h1>Servlet Lifecycle Demo</h1>");

 out.println("<p>The Servlet is processing the request.</p>");

```

```

 }

 // Called once before the Servlet is destroyed

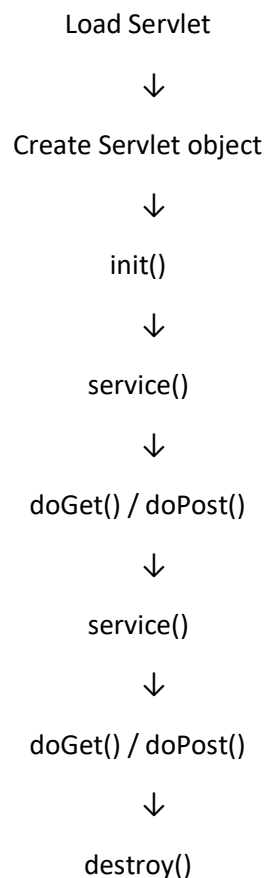
 @Override
 public void destroy() {

 System.out.println(
 "LifecycleDemoServlet: destroy() called."

);
 }
}

```

### **Lifecycle sequence**




---

### **QUESTION SEVEN – 8 MARKS**

#### **(a) Model-View-Controller (MVC)**

MVC stands for **Model-View-Controller**.

It is a software design pattern that separates an application into three main components.

### **1. Model**

The Model represents the application's data and business logic.

Responsibilities include:

- Representing data.
- Applying business rules.
- Communicating with databases.
- Managing application data.

Example:

Book

Student

Product

User

### **2. View**

The View is the presentation layer.

It is responsible for displaying information to the user.

Examples include:

- JSP pages.
- HTML pages.
- User interfaces.

### **3. Controller**

The Controller receives requests from the user and determines what should happen.

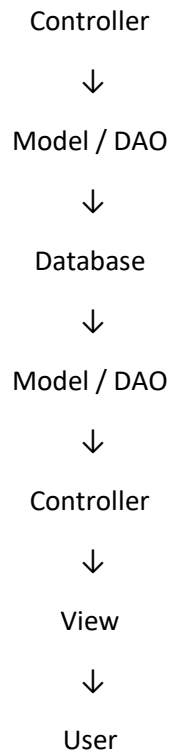
Responsibilities include:

- Receiving HTTP requests.
- Processing input.
- Calling the Model/DAO.
- Selecting the appropriate View.
- Passing data to the View.

### **MVC flow**

User





### Real-world example

An **online shopping system** can use MVC.

For example:

- **Model:** Product, Order and Customer classes.
  - **View:** JSP pages displaying products.
- **Controller:** Servlets handling product and order requests.
- **Database:** Stores product, customer and order information.

Many enterprise Java web applications use the MVC approach because it separates presentation, processing, and data management.

---

## QUESTION SEVEN (b) – Library Management System

### Model

The Model represents library data.

For this system, the Book class represents individual books.

It can contain:

- Book ID.
- Title.
- Author.

- ISBN.

### **DAO**

The Data Access Object, BookDAO, communicates with the database and retrieves books.

### **Controller**

BookController is a Servlet that:

1. Receives the request.
2. Calls BookDAO.
3. Retrieves the books.
4. Places them into the request.
5. Forwards the request to the JSP.

### **View**

bookList.jsp displays the books to the user.

---

### **Book.java**

```
package com.bit2203.library.model;
```

```
// Model class representing a book
```

```
public class Book {
```

```
 private int id;
```

```
 private String title;
```

```
 private String author;
```

```
 private String isbn;
```

```
 // Constructor
```

```
 public Book(int id, String title, String author, String isbn) {
```

```
 this.id = id;
```

```
 this.title = title;
```

```
 this.author = author;
```

```
 this.isbn = isbn;
```

```
 }
```



```
 // Getters

 public int getId() {

 return id;

 }

 public String getTitle() {

 return title;

 }

 public String getAuthor() {

 return author;

 }

 public String getIsbn() {

 return isbn;

 }

 // Setters

 public void setId(int id) {

 this.id = id;

 }

 public void setTitle(String title) {

 this.title = title;

 }

 public void setAuthor(String author) {

 this.author = author;

 }
```

```
public void setIsbn(String isbn) {
 this.isbn = isbn;
}
}
```

---

### **BookDAO.java**

```
package com.bit2203.library.dao;

import com.bit2203.library.model.Book;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import java.util.ArrayList;
import java.util.List;

// DAO responsible for accessing book data
public class BookDAO {

 private static final String URL =
 "jdbc:mysql://localhost:3306/library_db";

 private static final String USER = "admin";
 private static final String PASSWORD = "secure123";

 // Retrieves all books from the database
 public List<Book> getAllBooks() {
```

```
List<Book> books = new ArrayList<>();
```

```
String sql =
"SELECT id, title, author, isbn FROM books";
```

```
try (
 Connection connection =
 DriverManager.getConnection(
 URL, USER, PASSWORD);
```

```
 PreparedStatement statement =
 connection.prepareStatement(sql);
```

```
 ResultSet resultSet =
 statement.executeQuery()
) {
```

```
 while (resultSet.next()) {
```

```
 Book book = new Book(
 resultSet.getInt("id"),
 resultSet.getString("title"),
 resultSet.getString("author"),
 resultSet.getString("isbn")
);
```

```
 books.add(book);
 }
```

```
} catch (SQLException e) {
```

```
 System.out.println(
 "Database error: " + e.getMessage()
);
 }

 return books;
}
}
```

---

### **BookController.java**

```
package com.bit2203.library.controller;

import com.bit2203.library.dao.BookDAO;
import com.bit2203.library.model.Book;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;
import java.util.List;

// Controller Servlet for handling book requests
@WebServlet("/books")
public class BookController extends HttpServlet {

 private BookDAO bookDAO;

 // Create DAO when Servlet is initialized
```

```

 @Override
 public void init() {

 bookDAO = new BookDAO();

 }

// Handles request for displaying all books

 @Override
 protected void doGet(
 HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {

 // Retrieve books from DAO
 List<Book> books = bookDAO.getAllBooks();

 // Place books in request scope
 request.setAttribute("books", books);

 // Forward request to JSP View
 request.getRequestDispatcher(
 "/bookList.jsp"
).forward(request, response);

 }
 }

```

---

### **bookList.jsp**

```

<%@ page contentType="text/html;charset=UTF-8" %>

<%@ page import="java.util.List" %>

<%@ page import="com.bit2203.library.model.Book" %>

```

```
<!DOCTYPE html>

<html>

<head>

<title>Library Books</title>

</head>

<body>

<h1>Library Books</h1>

<%

List<Book> books =
(List<Book>) request.getAttribute("books");

%>

<table border="1" cellpadding="8">

<tr>

<th>ID</th>

<th>Title</th>

<th>Author</th>

<th>ISBN</th>

</tr>

<%

if (books != null && !books.isEmpty()) {

for (Book book : books) {

%>

<tr>
```

```

 <td><%= book.getId() %></td>
 <td><%= book.getTitle() %></td>
 <td><%= book.getAuthor() %></td>
 <td><%= book.getIsbn() %></td>
 </tr>

 <%
 }

 } else {
 %>

 <tr>
 <td colspan="4">
 No books found.
 </td>
 </tr>

 <%
 }
 %>

</table>

</body>
</html>

```

---

### **BONUS / EXTRA CREDIT – 2 MARKS**

#### **Inversion of Control (IoC)**

Inversion of Control (IoC) is a principle in which the responsibility for creating and managing objects is transferred from the application code to a framework or container.

In traditional programming, a class may create its own dependencies:

```
UserRepository repository = new UserRepository();
```

With Spring IoC, the Spring container creates and manages the object and supplies its dependencies.

### **Relationship between IoC and DI**

Dependency Injection is one of the main techniques used to implement IoC.

For example:

```
@Service

public class UserService {

 private final UserRepository repository;

 public UserService(UserRepository repository) {
 this.repository = repository;
 }
}
```

Spring creates UserRepository and injects it into UserService.

### **Importance of IoC**

IoC is important because it:

- Reduces coupling.
- Improves testability.
- Makes components reusable.
- Simplifies object management.
- Makes large enterprise applications easier to maintain.

Therefore, IoC is a fundamental principle in modern enterprise Java development because frameworks such as Spring can manage object creation, dependencies, configuration, and application components.

---

## **ADDITIONAL QUESTIONS FOR EXTRA PRACTICE**

### **1. Stateful versus Stateless Session Management**

#### **Stateful session management**

Stateful session management means the server maintains information about a user's previous interactions.



**Example:** An online shopping cart where the server stores the user's cart in an HttpSession.

### Stateless session management

Stateless communication means each request contains enough information for the server to process it without depending on previously stored server-side state.

**Example:** REST APIs using tokens such as JWTs for authentication.

### Comparison

Stateful	Stateless
Server maintains user state	Server does not maintain session state
Uses more server resources	Easier to scale horizontally
Example: HttpSession shopping cart	Example: Token-based REST API

---

## 2. JAR FILE

JAR stands for **Java Archive**.

A JAR file packages Java classes, resources, configuration files, and metadata into one compressed file.

For example:

MyApplication.jar

A JAR can be made executable by specifying the application's main class in its manifest.

### Example MANIFEST.MF

Manifest-Version: 1.0

Main-Class: com.example.Main

The JAR can then be executed using:

java -jar MyApplication.jar

---

## 3. MONOLITHIC VERSUS MICROSERVICES

A monolithic application is built as one large application where the components are packaged and deployed together.

Microservices architecture divides the application into multiple smaller and independently deployable services.

Monolithic	Microservices
One large application	Multiple smaller services

<b>Monolithic</b>	<b>Microservices</b>
Usually deployed together	Services can be deployed independently
Simpler initially	More complex to manage
Scaling may involve the whole application	Individual services can be scaled
Easier for small applications	Suitable for large and complex systems

### **When to choose monolithic**

A monolithic architecture is suitable when:

- The application is small.
- The development team is small.
- Simplicity is important.
- The application does not require independent scaling.

### **When to choose microservices**

Microservices are suitable when:

- The application is large.
- Different teams need to work independently.
- Different components need independent scaling.
- Independent deployment is important.

## **GITHUB REPOSITORY STRUCTURE**

Your repository should be named exactly:

BIT2203-AdvancedProgramming-CAT

A suitable structure is:

BIT2203-AdvancedProgramming-CAT/

```

|
├── README.md
|
├── src/
| |
| └── question1/
| └── URLParser.java

```

```

 | |
 | └─ question2/
 | | └─ StudentLogin.java
 | | └─ UniversityDatabase.java
 | |
 | |
 | | └─ question3/
 | | └─ ShoppingCartServlet.java
 | |
 | |
 | | └─ question4/
 | | └─ UserRepository.java
 | | └─ UserService.java
 | | └─ AppConfig.java
 | |
 | |
 | | └─ question5/
 | | └─ TemperatureConverter.java
 | | └─ TemperatureConverterImpl.java
 | | └─ RMIServer.java
 | | └─ RMIClient.java
 | |
 | |
 | | └─ question6/
 | | └─ LifecycleDemoServlet.java
 | |
 | |
 | | └─ question7/
 | | └─ Book.java
 | | └─ BookDAO.java
 | └─ BookController.java
 |
 └─ web/
 └─ bookList.jsp

```

Your repository should contain a README explaining the programs.

# BIT2203-AdvancedProgramming-CAT

## Student Information

Name: YOUR NAME

Registration Number: YOUR REGISTRATION NUMBER

## Description

This repository contains Java programs developed for the BIT 2203  
Advanced Programming Take-Away CAT.

## Programs

### Question 1 - URLParser

URLParser.java demonstrates how Java's URL class can be used to extract  
URL components including the protocol, host, port, path, query parameters  
and fragment.

### Question 2 - JDBC

StudentLogin.java demonstrates secure student authentication using  
PreparedStatement.

UniversityDatabase.java connects to the university\_db database, creates  
the courses table, inserts sample courses and retrieves Computer Science  
courses.

### ### Question 3 - Shopping Cart

ShoppingCartServlet.java demonstrates HTTP session management using HttpSession to maintain a customer's shopping cart.

### ### Question 4 - Spring Dependency Injection

UserRepository.java provides the repository dependency.

UserService.java demonstrates constructor dependency injection.

AppConfig.java configures component scanning in Spring.

### ### Question 5 - Java RMI

TemperatureConverter.java is the remote interface.

TemperatureConverterImpl.java implements the remote service.

RMIServer.java starts the RMI registry and binds the service.

RMIClient.java looks up the service and converts 25 degrees Celsius to Fahrenheit.

### ### Question 6 - Servlet Lifecycle

LifecycleDemoServlet.java demonstrates the Servlet lifecycle methods including init(), service(), doGet() and destroy().

### ### Question 7 - MVC Library System

Book.java represents the Model.

BookDAO.java accesses book data.

BookController.java acts as the Servlet Controller.

bookList.jsp acts as the View and displays the books.

## ## Requirements

- Java JDK
- MySQL Server
- MySQL Connector/J
- Apache Tomcat 10 or later for Jakarta Servlet applications
- Spring Framework for the Spring example
- An IDE such as IntelliJ IDEA, Eclipse or NetBeans

## ## Database Setup

Create the required database using:

```
CREATE DATABASE university_db;
```

For the library example:

```
CREATE DATABASE library_db;
```

Update database credentials in the Java files if necessary.

## ## Running the RMI Program

Compile the RMI classes:

```
javac *.java
```

Start the server:

```
java RMIServer
```

In another terminal, run:

```
java RMIClient
```

Expected result:

25°C = 77.0°F

## Servlet Programs

Deploy the Servlet project to a Jakarta Servlet-compatible web container  
such as Apache Tomcat 10 or later.

## GitHub

All Java source files and configuration files are included in this  
repository.

---

### FINAL SUBMISSION CHECKLIST

Before submitting, ensure you have:

- ☐ Created a **public** GitHub repository.
- ☐ Named it BIT2203-AdvancedProgramming-CAT.

- ☐ Added all required .java files.
  - ☐ Added README.md.
  - ☐ Added the JSP file.
- ☐ Added configuration files where applicable.
- ☐ Added comments to your Java programs.
  - ☐ Tested the programs.
  - ☐ Pushed all files to GitHub.
- ☐ Added your real name to the document.
  - ☐ Added your registration number.
  - ☐ Added your GitHub repository link.
- ☐ Saved the typed answers as PDF or Word.
- ☐ Checked that the GitHub repository is **public**.
- ☐ Verified that the GitHub link actually opens before submission.

#### **Required Java files**

URLParser.java

StudentLogin.java

UniversityDatabase.java

ShoppingCartServlet.java

UserRepository.java

UserService.java

AppConfig.java

TemperatureConverter.java

TemperatureConverterImpl.java

RMIServer.java

RMIClient.java

LifecycleDemoServlet.java

Book.java

BookDAO.java



BookController.java

bookList.jsp

README.md

**Important implementation note:** The Servlet examples above use jakarta.servlet.\*, which is appropriate for modern Servlet containers such as Tomcat 10+. If your lecturer/lab uses Tomcat 9 or an older Java EE setup, the imports may need to use javax.servlet.\* instead.

# BIT 2203 – ADVANCED PROGRAMMING

## TAKE-AWAY CAT

Name: MARYCYNTHIA GITAU

Registration Number: SCT221-0730/2023

GitHub Repository: <https://github.com/cynthiawgitau68-lab/BIT2203-AdvancedProgramming-CAT>

### QUESTION ONE – 6 MARKS

#### (a) TCP versus UDP

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are transport-layer protocols used for communication between client and server.

#### TCP

TCP is a **connection-oriented and reliable** protocol. Before data is transmitted, a connection is established between the client and server. TCP ensures that data packets arrive, arrive in the correct order, and are retransmitted if they are lost.

#### **\*\*Characteristics of TCP:\*\***

- \* Connection-oriented.
- \* Reliable data delivery.
- \* Packets are delivered in order.
- \* Error detection and retransmission are supported.
- \* Slower than UDP because of its reliability mechanisms.

#### ### UDP

# BIT 2203 – ADVANCED PROGRAMMING

## TAKE-AWAY CAT

Name: MARYCYNTHIA GITAU

Registration Number: SCT221-0730/2023

GitHub Repository: <https://github.com/cynthiawgitau68-lab/BIT2203-AdvancedProgramming-CAT>

### QUESTION ONE – 6 MARKS

#### (a) TCP versus UDP

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are transport-layer protocols used for communication between client and server.

#### TCP

TCP is a **connection-oriented and reliable** protocol. Before data is transmitted, a connection is established between the client and server. TCP ensures that data packets arrive, arrive in the correct order, and are retransmitted if they are lost.

#### **Characteristics of TCP:**

- \* Connection-oriented.
- \* Reliable data delivery.
- \* Packets are delivered in order.
- \* Error detection and retransmission are supported.
- \* Slower than UDP because of its reliability mechanisms.

#### ### UDP

UDP is a **connectionless and faster** protocol. It sends data without first establishing a connection. It does not guarantee that packets will arrive or that they will arrive in the correct order.

**Characteristics of UDP:**

- \* Connectionless.
- \* Faster than TCP.
- \* Lower overhead.
- \* Does not guarantee delivery.
- \* Suitable for applications where speed is more important than perfect reliability.

**Scenario where UDP is preferred**

UDP would be preferred in **live video conferencing or online gaming**.

For example, in an online game, if one packet containing a player's position is lost, it is normally better to continue receiving newer position updates rather than wait for the old packet to be retransmitted. The small amount of lost information is less important than maintaining real-time communication.

Therefore, UDP is preferred because it provides **low latency and faster communication**, which is important for real-time applications.

---

**(b) URLParser.java**

The given URL is:

``https://api.techstore.com:8443/products/category/electronics?sort=price&limit=20#reviews``

The following program uses Java's ``URL`` class to extract the required components.

### URLParser.java

```
```java
```

```
import java.net.URL;
```

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
public class URLParser {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            // Create a URL object
```

```
            URL url = new URL(
```

```
                "https://api.techstore.com:8443/products/category/electronics"
```

```
                + "?sort=price&limit=20#reviews"
```

```
            );
```

```
            // Display URL components
```

```
            System.out.println("Protocol: " + url.getProtocol());
```

```
            System.out.println("Host: " + url.getHost());
```

```
            System.out.println("Port: " + url.getPort());
```

```
            System.out.println("Path: " + url.getPath());
```

```
            // Extract and display query parameters
```

```
            String query = url.getQuery();
```

```
            System.out.println("Query Parameters:");
```

```
            if (query != null) {
```

```
                String[] parameters = query.split("&");
```

```

Map<String, String> queryParameters = new HashMap<>();

for (String parameter : parameters) {
    String[] pair = parameter.split("=", 2);

    if (pair.length == 2) {
        queryParameters.put(pair[0], pair[1]);
    }
}

for (Map.Entry<String, String> entry : queryParameters.entrySet()) {
    System.out.println(
        entry.getKey() + " = " + entry.getValue()
    );
}

// Display fragment
System.out.println("Fragment: " + url.getRef());

} catch (Exception e) {
    System.out.println("Error parsing URL: " + e.getMessage());
}

}

...

```

Expected output

```
```text
```

Protocol: https

Host: api.techstore.com

Port: 8443

Path: /products/category/electronics

Query Parameters:

sort = price

limit = 20

Fragment: reviews

...

---

# QUESTION TWO – 8 MARKS

## (a) PreparedStatement versus Statement

`PreparedStatement` is more secure and efficient than `Statement`, especially when handling login credentials.

### Security

A normal `Statement` can be vulnerable to **SQL injection** when user input is directly inserted into an SQL statement.

For example:

```
```java
```

```
String sql = "SELECT * FROM students WHERE reg_no = "
```

```
    + regNo + " AND password = " + password + """;
```

```
```
```

A malicious user could enter SQL commands as part of the input.

`PreparedStatement` separates SQL code from user-supplied values. Values are treated as data rather than executable SQL commands.

### ### Efficiency

A prepared statement can be precompiled by the database and reused with different values. This can improve performance when the same SQL statement is executed multiple times.

### ### PreparedStatement authentication example

```
```java
```

```
import java.sql.*;
```

```
public class StudentLogin {
```

```
    public static boolean authenticate(String regNo, String password) {
```

```
        String url = "jdbc:mysql://localhost:3306/university_db";
```

```
        String username = "admin";
```

```
        String dbPassword = "secure123";
```

```
        String sql = "SELECT * FROM students "
```

```
            + "WHERE registration_number = ? AND password = ?";
```

```
        try {
```

```
            Connection connection =
```

```
                DriverManager.getConnection(url, username, dbPassword);
```

```
            PreparedStatement statement =
```

```
                connection.prepareStatement(sql)
```

```
        } {
```



```

        // Set values safely
        statement.setString(1, regNo);
        statement.setString(2, password);

        try (ResultSet resultSet = statement.executeQuery()) {

            return resultSet.next();
        }

    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
        return false;
    }
}

public static void main(String[] args) {

    String registrationNumber = "BIT/001/2026";
    String password = "mypassword";

    if (authenticate(registrationNumber, password)) {
        System.out.println("Login successful.");
    } else {
        System.out.println("Invalid registration number or password.");
    }
}
}
'''

```

****Note:**** In a real production application, passwords should not be stored as plain text. They should be securely hashed using a password-hashing algorithm such as BCrypt or Argon2.

QUESTION TWO (b) – UniversityDatabase.java

The program below:

1. Connects to `university\_db`.
2. Creates the `courses` table.
3. Inserts three courses.
4. Retrieves courses belonging to the Computer Science department.

UniversityDatabase.java

```
```java
```

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
```

```
public class UniversityDatabase {
```

```
 // Database connection details
```

```
 private static final String URL =
 "jdbc:mysql://localhost:3306/university_db";
```

```
 private static final String USER = "admin";
```

```

private static final String PASSWORD = "secure123";

public static void main(String[] args) {

 // SQL statement for creating the courses table
 String createTableSQL = ""

 CREATE TABLE IF NOT EXISTS courses (

 course_id INT PRIMARY KEY,

 course_name VARCHAR(100),

 credits INT,

 department VARCHAR(50)

)

 """;

 // SQL statement for inserting courses
 String insertSQL = ""

 INSERT INTO courses

 (course_id, course_name, credits, department)

 VALUES (?, ?, ?, ?)

 """;

 // SQL statement for retrieving Computer Science courses
 String selectSQL =

 "SELECT * FROM courses WHERE department = ?";

 try (Connection connection =

 DriverManager.getConnection(URL, USER, PASSWORD)) {

 System.out.println("Database connected successfully.");

 // Create table

```

```
try (Statement statement = connection.createStatement()) {

 statement.executeUpdate(createTableSQL);

 System.out.println("Courses table created successfully.");
}
```

```
// Insert sample courses
```

```
try (PreparedStatement statement =
 connection.prepareStatement(insertSQL)) {
```

```
 insertCourse(statement, 101,
 "Advanced Programming", 3,
 "Computer Science");
```

```
 insertCourse(statement, 102,
 "Database Systems", 3,
 "Computer Science");
```

```
 insertCourse(statement, 103,
 "Computer Networks", 3,
 "Computer Science");
```

```
 System.out.println("Sample courses inserted.");
}
```

```
// Retrieve Computer Science courses
```

```
try (PreparedStatement statement =
 connection.prepareStatement(selectSQL)) {
```

```
 statement.setString(1, "Computer Science");
```

```

try (ResultSet resultSet = statement.executeQuery()) {

 System.out.println("\nComputer Science Courses:");

 while (resultSet.next()) {

 System.out.println(
 "ID: " + resultSet.getInt("course_id")
 + ", Name: " + resultSet.getString("course_name")
 + ", Credits: " + resultSet.getInt("credits")
 + ", Department: "
 + resultSet.getString("department")
);
 }
}

} catch (SQLException e) {
 System.out.println("Database error: " + e.getMessage());
}

}

// Method used to insert one course
private static void insertCourse(
 PreparedStatement statement,
 int id,
 String name,
 int credits,
 String department) throws SQLException {

```

```

 statement.setInt(1, id);
 statement.setString(2, name);
 statement.setInt(3, credits);
 statement.setString(4, department);

 statement.executeUpdate();
 }
}
'''

```

### Required JDBC dependency

For the above version, the project uses **\*\*MySQL Connector/J\*\***.

The database should be created before running the program:

```

```sql
CREATE DATABASE university_db;
'''

---

```

QUESTION THREE – 8 MARKS

(a) Servlets versus JSP

Servlet

A Servlet is a Java class that runs on a web server and handles HTTP requests and responses.

Servlets are mainly suitable for:

- * Processing requests.
- * Performing business logic.
- * Calling services and databases.
- * Controlling application flow.

JSP

JSP stands for **JavaServer Pages**. It is mainly used to create the presentation layer of a Java web application.

JSP makes it easier to create HTML pages containing dynamic information.

Comparison

Feature	Servlet	JSP
-----	-----	-----
Main purpose	Request processing and control	Presentation/UI
Development	More Java code	Easier for HTML/UI
Best MVC role	Controller	View
Suitable for	Business/request processing	Displaying dynamic content
HTML creation	More difficult	Easier

In an MVC application, the **Servlet** normally acts as the **Controller**, while the **JSP** acts as the **View**.

When to choose a Servlet

I would choose a Servlet when the application needs to process a request, validate data, call a DAO, perform business logic, and determine which page should be displayed.

When to choose JSP

I would choose JSP when the main requirement is displaying dynamic information to the user, such as displaying a list of products or student records.

QUESTION THREE (b) – Shopping Cart Session Management

Likely cause

The most likely cause is that the shopping cart is not being stored in the user's HTTP session.

HTTP is fundamentally **stateless**, meaning the server does not automatically remember information from one request to another.

For example, if a customer adds a product during one request and the cart is stored only in a local variable, that information can be lost when the customer makes another request.

Two session-management techniques

1. HttpSession

`HttpSession` can be used to store the cart on the server for a particular user session.

Example:

```
```java
```

```
HttpSession session = request.getSession();
```

```
List<String> cart =
```

```
 (List<String>) session.getAttribute("cart");
```



```

if (cart == null) {
 cart = new ArrayList<>();
}

cart.add("Laptop");

session.setAttribute("cart", cart);
...

```

## #### 2. Cookies

Cookies can store a small amount of information on the user's browser. A session identifier can also be stored in a cookie so that the server can associate subsequent requests with the correct session.

For a shopping cart, server-side `HttpSession` is generally preferable for storing the actual cart contents.

---

```
ShoppingCartServlet.java
```

```

``java
package com.bit2203.servlet;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;

```

```
import java.io.PrintWriter;

import java.util.ArrayList;

import java.util.List;

@WebServlet("/shopping-cart")

public class ShoppingCartServlet extends HttpServlet {

 private static final String CART_ATTRIBUTE = "cart";

 // Handles GET requests and displays the cart

 @Override

 protected void doGet(

 HttpServletRequest request,

 HttpServletResponse response)

 throws ServletException, IOException {

 List<String> cart = getCart(request);

 response.setContentType("text/html");

 PrintWriter out = response.getWriter();

 out.println("<html>");
 out.println("<body>");
 out.println("<h1>Shopping Cart</h1>");

 if (cart.isEmpty()) {

 out.println("<p>Your cart is empty.</p>");

 } else {
```

```

 out.println("");

 for (String item : cart) {
 out.println("" + item + "");
 }

 out.println("");
 }

 out.println("</body>");
 out.println("</html>");
}

// Handles POST requests for adding items
@Override
protected void doPost(
 HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {

 String item = request.getParameter("item");

 if (item != null && !item.trim().isEmpty()) {

 HttpSession session = request.getSession();

 List<String> cart = getCart(request);

 cart.add(item);
 }
}

```

```

 session.setAttribute(CART_ATTRIBUTE, cart);
 }

 response.sendRedirect("shopping-cart");
}

// Retrieves the cart from the current user's session
@SuppressWarnings("unchecked")
private List<String> getCart(HttpServletRequest request) {

 HttpSession session = request.getSession();

 List<String> cart =
 (List<String>) session.getAttribute(CART_ATTRIBUTE);

 // Create a new cart if one does not exist
 if (cart == null) {

 cart = new ArrayList<>();

 session.setAttribute(CART_ATTRIBUTE, cart);
 }

 return cart;
}
}
...

```

# QUESTION FOUR – 8 MARKS

## ## (a) Microservices Architecture

Microservices architecture is a software architecture in which a large application is divided into several small, independent services.

Each service performs a specific business function and can communicate with other services through APIs or messaging systems.

For example, an e-commerce system could contain:

- \* User Service
- \* Product Service
- \* Payment Service
- \* Order Service
- \* Delivery Service

Each service can be developed, deployed, and scaled independently.

### ### Benefits

#### #### 1. Independent deployment

A single service can be updated or deployed without necessarily redeploying the entire application.

#### #### 2. Scalability

Individual services can be scaled according to demand.

For example, during a busy shopping period, the Order Service can be given additional resources without scaling every other service.

### ### Challenges

#### #### 1. Increased complexity

A microservices system contains many services, databases, APIs, and communication channels, making the system more difficult to manage.

#### #### 2. Network and communication problems

Services communicate over networks, so problems such as latency, network failures, and service unavailability can occur.

### ### Real-world example

**\*\*Netflix\*\*** is a well-known example of a company that uses microservices architecture. Its large platform is divided into numerous independent services that support different functions of the streaming platform.

---

### # QUESTION FOUR (b) – Dependency Injection

Dependency Injection (DI) is a design technique where an object's dependencies are supplied to it from outside instead of the object creating those dependencies itself.

For example, instead of ``UserService`` creating a ``UserRepository`` using ``new``, Spring provides the repository to the service.

### ### Benefits of DI

- \* Reduces coupling.
- \* Makes applications easier to test.
- \* Makes components easier to replace.

- \* Improves maintainability.
- \* Supports clean application architecture.

## ## Constructor Injection

In constructor injection, dependencies are supplied through the class constructor.

### ### Advantage

The dependency can be made `final`, ensuring that the object cannot exist without its required dependency.

## ## Setter Injection

Setter injection supplies dependencies through setter methods.

### ### Advantage

It is useful when a dependency is optional or needs to be changed after object creation.

### ### Comparison

Constructor Injection	Setter Injection	
-----	-----	
Dependency supplied through constructor	Dependency supplied through setter	
Good for required dependencies	Good for optional dependencies	
Supports immutable dependencies	Dependency can be changed	
Makes dependencies obvious	More flexible after construction	

---

```
UserRepository.java
```

```
```java
```

```
package com.bit2203.spring;
```

```
import org.springframework.stereotype.Repository;
```

```
@Repository
```

```
public class UserRepository {
```

```
    // Example repository method
```

```
    public String findUserById(int userId) {
```

```
        return "User with ID: " + userId;
```

```
    }
```

```
}
```

```
```
```

```

```

```
UserService.java
```

```
```java
```

```
package com.bit2203.spring;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class UserService {
```

```
    private final UserRepository userRepository;
```



```

// Constructor injection
public UserService(UserRepository userRepository) {
    this.userRepository = userRepository;
}

// Service method
public String getUser(int userId) {

    return userRepository.findUserById(userId);
}
}
...

---

## AppConfig.java

```java
package com.bit2203.spring;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.bit2203.spring")
public class AppConfig {

 // Spring automatically detects
 // @Service and @Repository classes.
}

```

...

---

## # QUESTION FIVE – 6 MARKS

### ## (a) Remote Method Invocation (RMI)

Java RMI, or **Remote Method Invocation**, is a Java technology that allows an object running in one Java Virtual Machine to invoke methods on an object running in another Java Virtual Machine.

RMI allows Java applications running on different computers or processes to communicate as though they were calling methods on local objects.

#### ### RMI Registry

The RMI Registry is a naming service.

It stores references to remote objects using names so that clients can locate those objects.

#### ### Server

The server:

1. Implements the remote interface.
2. Creates the remote object.
3. Exports the object.
4. Registers the object with the RMI Registry.

#### ### Client

The client:

1. Connects to the RMI Registry.
2. Looks up the remote object by name.
3. Obtains a reference to the remote service.
4. Invokes methods on the remote object.

---

#### # QUESTION FIVE (b) – Java RMI Program

## TemperatureConverter.java

```java

import java.rmi.Remote;

import java.rmi.RemoteException;

// Remote interface containing methods

// that can be called by the client.

public interface TemperatureConverter extends Remote {

double celsiusToFahrenheit(double celsius)

throws RemoteException;

double fahrenheitToCelsius(double fahrenheit)

throws RemoteException;

}

```

---

## TemperatureConverterImpl.java

```

```java
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

// Implementation of the remote interface
public class TemperatureConverterImpl
    extends UnicastRemoteObject
    implements TemperatureConverter {

    // Constructor
    public TemperatureConverterImpl()
        throws RemoteException {
        super();
    }

    // Converts Celsius to Fahrenheit
    @Override
    public double celsiusToFahrenheit(double celsius)
        throws RemoteException {

        return (celsius * 9 / 5) + 32;
    }

    // Converts Fahrenheit to Celsius
    @Override
    public double fahrenheitToCelsius(double fahrenheit)
        throws RemoteException {

        return (fahrenheit - 32) * 5 / 9;
    }
}

```

```
}  
'''
```

```
---
```

```
## RMIServer.java
```

```
```java
```

```
import java.rmi.registry.LocateRegistry;
```

```
import java.rmi.registry.Registry;
```

```
// RMI server
```

```
public class RMIServer {
```

```
 public static void main(String[] args) {
```

```
 try {
```

```
 // Create the remote service
```

```
 TemperatureConverter converter =
```

```
 new TemperatureConverterImpl();
```

```
 // Start the RMI registry on port 1099
```

```
 Registry registry =
```

```
 LocateRegistry.createRegistry(1099);
```

```
 // Bind the remote object to a name
```

```
 registry.rebind(
```

```
 "TemperatureConverter",
```

```
 converter
```

```
);
```

```

 System.out.println(
 "Temperature Converter RMI Server is running..."
);

 } catch (Exception e) {

 System.out.println(
 "Server error: " + e.getMessage()
);

 e.printStackTrace();
 }
}
...

```

---

## RMIClient.java

```

``java
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

// RMI client
public class RMIClient {

 public static void main(String[] args) {

 try {
 // Locate the RMI registry

```

```

Registry registry =
 LocateRegistry.getRegistry(
 "localhost", 1099
);

// Look up the remote service
TemperatureConverter converter =
 (TemperatureConverter)
 registry.lookup("TemperatureConverter");

// Convert 25 degrees Celsius
double result =
 converter.celsiusToFahrenheit(25);

System.out.println(
 "25°C = " + result + "°F"
);

} catch (Exception e) {

 System.out.println(
 "Client error: " + e.getMessage()
);

 e.printStackTrace();
}
}
}
'''

```

### Expected output

```text

25°C = 77.0°F

```

---

## # QUESTION SIX – 6 MARKS

### ## (a) Web Container

A **web container**, also called a Servlet container, is a component of a Java web server that manages the execution of Java Servlets.

Examples include Apache Tomcat, Jetty, and other Jakarta Servlet-compatible servers.

### ### Four responsibilities

#### 1. **Servlet lifecycle management**

The container loads, initializes, executes, and destroys Servlets.

#### 2. **Request and response management**

It receives HTTP requests and provides them to the appropriate Servlet.

#### 3. **Session management**

It manages HTTP sessions and helps maintain user state.

#### 4. **Security and resource management**

It can provide authentication, authorization, URL mapping, and manage Servlet resources.

---



## # QUESTION SIX (b) – Servlet Lifecycle

The main stages of a Servlet lifecycle are:

### ### 1. Loading and Instantiation

The web container loads the Servlet class and creates an instance of it.

### ### 2. Initialization – `init()`

The container calls the `init()` method once.

This is used for initialization tasks such as loading configuration information or resources.

### ### 3. Request Processing – `service()`

For every client request, the container calls the `service()` method.

For an `HttpServlet`, the `service()` method determines whether the request is GET, POST, PUT, DELETE, etc., and calls methods such as `doGet()` or `doPost()`.

### ### 4. Destruction – `destroy()`

When the Servlet is removed from service, the container calls `destroy()` once.

This is used to release resources.

---

## LifecycleDemoServlet.java

```

```java
package com.bit2203.servlet;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;
import java.io.PrintWriter;

@WebServlet("/lifecycle")
public class LifecycleDemoServlet extends HttpServlet {

    // Called once when the Servlet is initialized
    @Override
    public void init() throws ServletException {

        System.out.println(
            "LifecycleDemoServlet: init() called."
        );
    }

    // Called for each HTTP request
    @Override
    protected void service(
        HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

```

```

        System.out.println(
            "LifecycleDemoServlet: service() called."
        );

        // Pass the request to the parent implementation.
        super.service(request, response);
    }

    // Handles GET requests
    @Override
    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        System.out.println(
            "LifecycleDemoServlet: doGet() called."
        );

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<h1>Servlet Lifecycle Demo</h1>");
        out.println("<p>The Servlet is processing the request.</p>");
    }

    // Called once before the Servlet is destroyed
    @Override
    public void destroy() {

```

```
        System.out.println(
            "LifecycleDemoServlet: destroy() called."
        );
    }
}
...

```

Lifecycle sequence

``text

Load Servlet



Create Servlet object



init()



service()



doGet() / doPost()



service()



doGet() / doPost()



destroy()

...

QUESTION SEVEN – 8 MARKS

(a) Model-View-Controller (MVC)

MVC stands for ****Model-View-Controller****.

It is a software design pattern that separates an application into three main components.

1. Model

The Model represents the application's data and business logic.

Responsibilities include:

- * Representing data.
- * Applying business rules.
- * Communicating with databases.
- * Managing application data.

Example:

```
```text
```

Book

Student

Product

User

```
```
```

2. View

The View is the presentation layer.

It is responsible for displaying information to the user.

Examples include:

- * JSP pages.
- * HTML pages.
- * User interfaces.

3. Controller

The Controller receives requests from the user and determines what should happen.

Responsibilities include:

- * Receiving HTTP requests.
- * Processing input.
- * Calling the Model/DAO.
- * Selecting the appropriate View.
- * Passing data to the View.

MVC flow

```text

User



Controller



Model / DAO



Database



Model / DAO



Controller



View



User

...

### Real-world example

An **online shopping system** can use MVC.

For example:

**Model:** Product, Order and Customer classes.

**View:** JSP pages displaying products.

**Controller:** Servlets handling product and order requests.

**Database:** Stores product, customer and order information.

Many enterprise Java web applications use the MVC approach because it separates presentation, processing, and data management.

---

# QUESTION SEVEN (b) – Library Management System

## Model

The Model represents library data.

For this system, the `Book` class represents individual books.

It can contain:

- \* Book ID.
- \* Title.
- \* Author.
- \* ISBN.

## DAO

The Data Access Object, `BookDAO`, communicates with the database and retrieves books.

## Controller

`BookController` is a Servlet that:

1. Receives the request.
2. Calls `BookDAO`.
3. Retrieves the books.
4. Places them into the request.
5. Forwards the request to the JSP.

## View

`bookList.jsp` displays the books to the user.

---

# Book.java



```
``java

package com.bit2203.library.model;

// Model class representing a book
public class Book {

 private int id;
 private String title;
 private String author;
 private String isbn;

 // Constructor
 public Book(int id, String title, String author, String isbn) {
 this.id = id;
 this.title = title;
 this.author = author;
 this.isbn = isbn;
 }

 // Getters
 public int getId() {
 return id;
 }

 public String getTitle() {
 return title;
 }

 public String getAuthor() {
 return author;
 }
}
```

```

 public String getIsbn() {
 return isbn;
 }

 // Setters
 public void setId(int id) {
 this.id = id;
 }

 public void setTitle(String title) {
 this.title = title;
 }

 public void setAuthor(String author) {
 this.author = author;
 }

 public void setIsbn(String isbn) {
 this.isbn = isbn;
 }
}
...

BookDAO.java

```java
package com.bit2203.library.dao;

```

```
import com.bit2203.library.model.Book;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import java.util.ArrayList;
import java.util.List;

// DAO responsible for accessing book data
public class BookDAO {

    private static final String URL =
        "jdbc:mysql://localhost:3306/library_db";

    private static final String USER = "admin";
    private static final String PASSWORD = "secure123";

    // Retrieves all books from the database
    public List<Book> getAllBooks() {

        List<Book> books = new ArrayList<>();

        String sql =
            "SELECT id, title, author, isbn FROM books";

        try (
            Connection connection =
                DriverManager.getConnection(
```

```
URL, USER, PASSWORD);
```

```
PreparedStatement statement =
```

```
    connection.prepareStatement(sql);
```

```
ResultSet resultSet =
```

```
    statement.executeQuery()
```

```
) {
```

```
    while (resultSet.next()) {
```

```
        Book book = new Book(
```

```
            resultSet.getInt("id"),
```

```
            resultSet.getString("title"),
```

```
            resultSet.getString("author"),
```

```
            resultSet.getString("isbn")
```

```
        );
```

```
        books.add(book);
```

```
    }
```

```
} catch (SQLException e) {
```

```
    System.out.println(
```

```
        "Database error: " + e.getMessage()
```

```
    );
```

```
}
```

```
return books;
```

```
}
```

```
}
```

...

BookController.java

```java

package com.bit2203.library.controller;

import com.bit2203.library.dao.BookDAO;

import com.bit2203.library.model.Book;

import jakarta.servlet.ServletException;

import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.HttpServlet;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

import java.util.List;

// Controller Servlet for handling book requests

@WebServlet("/books")

public class BookController extends HttpServlet {

private BookDAO bookDAO;

// Create DAO when Servlet is initialized

@Override

public void init() {

```

 bookDAO = new BookDAO();
 }

 // Handles request for displaying all books
 @Override
 protected void doGet(
 HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {

 // Retrieve books from DAO
 List<Book> books = bookDAO.getAllBooks();

 // Place books in request scope
 request.setAttribute("books", books);

 // Forward request to JSP View
 request.getRequestDispatcher(
 "/bookList.jsp"
).forward(request, response);
 }
}
...

bookList.jsp

```.jsp
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ page import="java.util.List" %>

```

```
<%@ page import="com.bit2203.library.model.Book" %>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Library Books</title>
```

```
</head>
```

```
<body>
```

```
<h1>Library Books</h1>
```

```
<%
```

```
    List<Book> books =
```

```
        (List<Book>) request.getAttribute("books");
```

```
%>
```

```
<table border="1" cellpadding="8">
```

```
    <tr>
```

```
        <th>ID</th>
```

```
        <th>Title</th>
```

```
        <th>Author</th>
```

```
        <th>ISBN</th>
```

```
    </tr>
```

```
<%
```

```
    if (books != null && !books.isEmpty()) {
```

```
        for (Book book : books) {
```

```
%>
```

```
<tr>

    <td><%= book.getId() %></td>

    <td><%= book.getTitle() %></td>

    <td><%= book.getAuthor() %></td>

    <td><%= book.getIsbn() %></td>

</tr>
```

```
<%
    }
```

```
    } else {
%>
```

```
<tr>

    <td colspan="4">

        No books found.

    </td>

</tr>
```

```
<%
    }
%>
```

```
</table>
```

```
</body>
```

```
</html>
```

```
'''
```

```
---
```


BONUS / EXTRA CREDIT – 2 MARKS

Inversion of Control (IoC)

Inversion of Control (IoC) is a principle in which the responsibility for creating and managing objects is transferred from the application code to a framework or container.

In traditional programming, a class may create its own dependencies:

```
```java
UserRepository repository = new UserRepository();
```
```

With Spring IoC, the Spring container creates and manages the object and supplies its dependencies.

Relationship between IoC and DI

Dependency Injection is one of the main techniques used to implement IoC.

For example:

```
```java
@Service
public class UserService {

 private final UserRepository repository;

 public UserService(UserRepository repository) {
 this.repository = repository;
 }
```
```

```
}  
...  

```

Spring creates `UserRepository` and injects it into `UserService`.

Importance of IoC

IoC is important because it:

- * Reduces coupling.
- * Improves testability.
- * Makes components reusable.
- * Simplifies object management.
- * Makes large enterprise applications easier to maintain.

Therefore, IoC is a fundamental principle in modern enterprise Java development because frameworks such as Spring can manage object creation, dependencies, configuration, and application components.

ADDITIONAL QUESTIONS FOR EXTRA PRACTICE

1. Stateful versus Stateless Session Management

Stateful session management

Stateful session management means the server maintains information about a user's previous interactions.

****Example:**** An online shopping cart where the server stores the user's cart in an `HttpSession`.

Stateless session management

Stateless communication means each request contains enough information for the server to process it without depending on previously stored server-side state.

Example: REST APIs using tokens such as JWTs for authentication.

Comparison

| Stateful | Stateless |
|------------------------------------|--|
| ----- | ----- |
| Server maintains user state | Server does not maintain session state |
| Uses more server resources | Easier to scale horizontally |
| Example: HttpSession shopping cart | Example: Token-based REST API |

2. JAR FILE

JAR stands for **Java Archive**.

A JAR file packages Java classes, resources, configuration files, and metadata into one compressed file.

For example:

```
```text
MyApplication.jar
```
```

A JAR can be made executable by specifying the application's main class in its manifest.

Example MANIFEST.MF

```
```text
```

```
Manifest-Version: 1.0
```

```
Main-Class: com.example.Main
```

```
```
```

The JAR can then be executed using:

```
```bash
```

```
java -jar MyApplication.jar
```

```
```
```

3. MONOLITHIC VERSUS MICROSERVICES

A monolithic application is built as one large application where the components are packaged and deployed together.

Microservices architecture divides the application into multiple smaller and independently deployable services.

| Monolithic | Microservices | |
|---|--|--|
| ----- | ----- | |
| One large application | Multiple smaller services | |
| Usually deployed together | Services can be deployed independently | |
| Simpler initially | More complex to manage | |
| Scaling may involve the whole application | Individual services can be scaled | |
| Easier for small applications | Suitable for large and complex systems | |

When to choose monolithic

A monolithic architecture is suitable when:

- * The application is small.
- * The development team is small.
- * Simplicity is important.
- * The application does not require independent scaling.

When to choose microservices

Microservices are suitable when:

- * The application is large.
- * Different teams need to work independently.
- * Different components need independent scaling.
- * Independent deployment is important.

GITHUB REPOSITORY STRUCTURE

Your repository should be named exactly:

```
```text
```

```
BIT2203-AdvancedProgramming-CAT
```

```
```
```

A suitable structure is:

```
```text
```

```
BIT2203-AdvancedProgramming-CAT/
```

```
|
|
| └─ README.md
|
|
| └─ src/
| |
| | └─ question1/
| | └─ URLParser.java
| |
| |
| | └─ question2/
| | └─ StudentLogin.java
| | └─ UniversityDatabase.java
| |
| |
| | └─ question3/
| | └─ ShoppingCartServlet.java
| |
| |
| | └─ question4/
| | └─ UserRepository.java
| | └─ UserService.java
| | └─ AppConfig.java
| |
| |
| | └─ question5/
| | └─ TemperatureConverter.java
| | └─ TemperatureConverterImpl.java
| | └─ RMIServer.java
| | └─ RMIClient.java
| |
| |
| | └─ question6/
| | └─ LifecycleDemoServlet.java
| |
| |
| └─ question7/
| └─ Book.java
```

```
| └─ BookDAO.java
| └─ BookController.java
|
└─ web/
 └─ bookList.jsp
...

```

# README.md

Your repository should contain a README explaining the programs.

```markdown

BIT2203-AdvancedProgramming-CAT

Student Information

Name: YOUR NAME

Registration Number: YOUR REGISTRATION NUMBER

Description

This repository contains Java programs developed for the BIT 2203
Advanced Programming Take-Away CAT.

Programs

Question 1 - URLParser

URLParser.java demonstrates how Java's URL class can be used to extract URL components including the protocol, host, port, path, query parameters and fragment.

Question 2 - JDBC

StudentLogin.java demonstrates secure student authentication using PreparedStatement.

UniversityDatabase.java connects to the university_db database, creates the courses table, inserts sample courses and retrieves Computer Science courses.

Question 3 - Shopping Cart

ShoppingCartServlet.java demonstrates HTTP session management using HttpSession to maintain a customer's shopping cart.

Question 4 - Spring Dependency Injection

UserRepository.java provides the repository dependency.

UserService.java demonstrates constructor dependency injection.

AppConfig.java configures component scanning in Spring.

Question 5 - Java RMI

TemperatureConverter.java is the remote interface.

TemperatureConverterImpl.java implements the remote service.

RMIServer.java starts the RMI registry and binds the service.

RMIClient.java looks up the service and converts 25 degrees Celsius to Fahrenheit.

Question 6 - Servlet Lifecycle

LifecycleDemoServlet.java demonstrates the Servlet lifecycle methods including init(), service(), doGet() and destroy().

Question 7 - MVC Library System

Book.java represents the Model.

BookDAO.java accesses book data.

BookController.java acts as the Servlet Controller.

bookList.jsp acts as the View and displays the books.

Requirements

- Java JDK
- MySQL Server
- MySQL Connector/J
- Apache Tomcat 10 or later for Jakarta Servlet applications
- Spring Framework for the Spring example
- An IDE such as IntelliJ IDEA, Eclipse or NetBeans

Database Setup

Create the required database using:

```
CREATE DATABASE university_db;
```

For the library example:

```
CREATE DATABASE library_db;
```

Update database credentials in the Java files if necessary.

Running the RMI Program

Compile the RMI classes:

```
javac *.java
```

Start the server:

```
java RMIServer
```

In another terminal, run:

```
java RMIClient
```

Expected result:

25°C = 77.0°F

Servlet Programs

Deploy the Servlet project to a Jakarta Servlet-compatible web container such as Apache Tomcat 10 or later.

GitHub

All Java source files and configuration files are included in this repository.

...

FINAL SUBMISSION CHECKLIST

Before submitting, ensure you have:

- * ☐ Created a **public** GitHub repository.
- * ☐ Named it `BIT2203-AdvancedProgramming-CAT`.
- * ☐ Added all required `.java` files.
- * ☐ Added `README.md`.
- * ☐ Added the JSP file.
- * ☐ Added configuration files where applicable.
- * ☐ Added comments to your Java programs.
- * ☐ Tested the programs.
- * ☐ Pushed all files to GitHub.
- * ☐ Added your real name to the document.
- * ☐ Added your registration number.
- * ☐ Added your GitHub repository link.
- * ☐ Saved the typed answers as PDF or Word.
- * ☐ Checked that the GitHub repository is **public**.
- * ☐ Verified that the GitHub link actually opens before submission.

Required Java files

```text

URLParser.java

StudentLogin.java

UniversityDatabase.java

ShoppingCartServlet.java

UserRepository.java

UserService.java

AppConfig.java

TemperatureConverter.java

TemperatureConverterImpl.java

RMIserver.java

RMIclient.java

LifecycleDemoServlet.java

Book.java

BookDAO.java

BookController.java

bookList.jsp

README.md

```

****Important implementation note:**** The Servlet examples above use `jakarta.servlet.\*`, which is appropriate for modern Servlet containers such as Tomcat 10+. If your lecturer/lab uses Tomcat 9 or an older Java EE setup, the imports may need to use `javax.servlet.\*` instead. **BIT 2203 –**

ADVANCED PROGRAMMING

TAKE-AWAY CAT – COMPLETE ANSWERS

Name: _____

Registration Number: _____

GitHub Repository: <https://github.com/YOUR-USERNAME/BIT2203-AdvancedProgramming-CAT>

QUESTION ONE – 6 MARKS

(a) TCP versus UDP

TCP (Transmission Control Protocol) and **UDP (User Datagram Protocol)** are transport-layer protocols used for communication between client and server.

TCP

TCP is a **connection-oriented and reliable** protocol. Before data is transmitted, a connection is established between the client and server. TCP ensures that data packets arrive, arrive in the correct order, and are retransmitted if they are lost.

Characteristics of TCP:

- Connection-oriented.
- Reliable data delivery.
- Packets are delivered in order.
- Error detection and retransmission are supported.
- Slower than UDP because of its reliability mechanisms.

UDP

UDP is a **connectionless and faster** protocol. It sends data without first establishing a connection. It does not guarantee that packets will arrive or that they will arrive in the correct order.

Characteristics of UDP:

- Connectionless.
- Faster than TCP.
- Lower overhead.
- Does not guarantee delivery.
- Suitable for applications where speed is more important than perfect reliability.

Scenario where UDP is preferred

UDP would be preferred in **live video conferencing or online gaming**.

For example, in an online game, if one packet containing a player's position is lost, it is normally better to continue receiving newer position updates rather than wait for the old packet to be retransmitted. The small amount of lost information is less important than maintaining real-time communication.

Therefore, UDP is preferred because it provides **low latency and faster communication**, which is important for real-time applications.

(b) URLParser.java

The given URL is:

<https://api.techstore.com:8443/products/category/electronics?sort=price&limit=20#reviews>

The following program uses Java's URL class to extract the required components.

URLParser.java

```
import java.net.URL;

import java.util.HashMap;

import java.util.Map;

public class URLParser {

    public static void main(String[] args) {

        try {

            // Create a URL object

            URL url = new URL(

                "https://api.techstore.com:8443/products/category/electronics"

                + "?sort=price&limit=20#reviews"

            );

            // Display URL components

            System.out.println("Protocol: " + url.getProtocol());

            System.out.println("Host: " + url.getHost());

            System.out.println("Port: " + url.getPort());

            System.out.println("Path: " + url.getPath());

            // Extract and display query parameters

            String query = url.getQuery();

            System.out.println("Query Parameters:");

            if (query != null) {

                String[] parameters = query.split("&");

                Map<String, String> queryParameters = new HashMap<>();
```

```

        for (String parameter : parameters) {
            String[] pair = parameter.split("=", 2);

            if (pair.length == 2) {
                queryParameters.put(pair[0], pair[1]);
            }
        }

        for (Map.Entry<String, String> entry : queryParameters.entrySet()) {
            System.out.println(
                entry.getKey() + " = " + entry.getValue()
            );
        }
    }

    // Display fragment
    System.out.println("Fragment: " + url.getRef());

} catch (Exception e) {
    System.out.println("Error parsing URL: " + e.getMessage());
}
}
}

```

Expected output

Protocol: https

Host: api.techstore.com

Port: 8443

Path: /products/category/electronics

Query Parameters:

sort = price

limit = 20

Fragment: reviews

QUESTION TWO – 8 MARKS

(a) PreparedStatement versus Statement

PreparedStatement is more secure and efficient than Statement, especially when handling login credentials.

Security

A normal Statement can be vulnerable to **SQL injection** when user input is directly inserted into an SQL statement.

For example:

```
String sql = "SELECT * FROM students WHERE reg_no = " +  
    regNo + " AND password = " + password + "";
```

A malicious user could enter SQL commands as part of the input.

PreparedStatement separates SQL code from user-supplied values. Values are treated as data rather than executable SQL commands.

Efficiency

A prepared statement can be precompiled by the database and reused with different values. This can improve performance when the same SQL statement is executed multiple times.

PreparedStatement authentication example

```
import java.sql.*;
```

```
public class StudentLogin {
```

```
    public static boolean authenticate(String regNo, String password) {
```

```
        String url = "jdbc:mysql://localhost:3306/university_db";
```

```
        String username = "admin";
```

```
        String dbPassword = "secure123";
```

```
        String sql = "SELECT * FROM students "
```

```
            + "WHERE registration_number = ? AND password = ?";
```



```

try (
    Connection connection =
        DriverManager.getConnection(url, username, dbPassword);

    PreparedStatement statement =
        connection.prepareStatement(sql)
) {

    // Set values safely
    statement.setString(1, regNo);
    statement.setString(2, password);

    try (ResultSet resultSet = statement.executeQuery()) {

        return resultSet.next();
    }

} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
    return false;
}
}

public static void main(String[] args) {

    String registrationNumber = "BIT/001/2026";
    String password = "mypassword";

    if (authenticate(registrationNumber, password)) {
        System.out.println("Login successful.");
    }
}

```

```

    } else {
        System.out.println("Invalid registration number or password.");
    }
}
}

```

Note: In a real production application, passwords should not be stored as plain text. They should be securely hashed using a password-hashing algorithm such as BCrypt or Argon2.

QUESTION TWO (b) – UniversityDatabase.java

The program below:

5. Connects to university_db.
6. Creates the courses table.
7. Inserts three courses.
8. Retrieves courses belonging to the Computer Science department.

UniversityDatabase.java

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class UniversityDatabase {

    // Database connection details
    private static final String URL =
        "jdbc:mysql://localhost:3306/university_db";

    private static final String USER = "admin";
    private static final String PASSWORD = "secure123";

```

```

public static void main(String[] args) {

    // SQL statement for creating the courses table
    String createTableSQL = ""

        CREATE TABLE IF NOT EXISTS courses (

            course_id INT PRIMARY KEY,

            course_name VARCHAR(100),

            credits INT,

            department VARCHAR(50)

        )

        """;

    // SQL statement for inserting courses
    String insertSQL = ""

        INSERT INTO courses

        (course_id, course_name, credits, department)

        VALUES (?, ?, ?, ?)

        """;

    // SQL statement for retrieving Computer Science courses
    String selectSQL =

        "SELECT * FROM courses WHERE department = ?";

    try (Connection connection =

        DriverManager.getConnection(URL, USER, PASSWORD)) {

        System.out.println("Database connected successfully.");

        // Create table
        try (Statement statement = connection.createStatement()) {

```

```
statement.executeUpdate(createTableSQL);

System.out.println("Courses table created successfully.");
}

// Insert sample courses
try (PreparedStatement statement =
    connection.prepareStatement(insertSQL)) {

    insertCourse(statement, 101,
        "Advanced Programming", 3,
        "Computer Science");

    insertCourse(statement, 102,
        "Database Systems", 3,
        "Computer Science");

    insertCourse(statement, 103,
        "Computer Networks", 3,
        "Computer Science");

    System.out.println("Sample courses inserted.");
}

// Retrieve Computer Science courses
try (PreparedStatement statement =
    connection.prepareStatement(selectSQL)) {

    statement.setString(1, "Computer Science");

    try (ResultSet resultSet = statement.executeQuery()) {
```

```

        System.out.println("\nComputer Science Courses:");

        while (resultSet.next()) {

            System.out.println(
                "ID: " + resultSet.getInt("course_id")
                + ", Name: " + resultSet.getString("course_name")
                + ", Credits: " + resultSet.getInt("credits")
                + ", Department: "
                + resultSet.getString("department")
            );
        }
    }

} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}

}

// Method used to insert one course
private static void insertCourse(
    PreparedStatement statement,
    int id,
    String name,
    int credits,
    String department) throws SQLException {

    statement.setInt(1, id);
    statement.setString(2, name);

```

```

        statement.setInt(3, credits);

        statement.setString(4, department);


        statement.executeUpdate();
    }
}

```

Required JDBC dependency

For the above version, the project uses **MySQL Connector/J**.

The database should be created before running the program:

```
CREATE DATABASE university_db;
```

QUESTION THREE – 8 MARKS

(a) Servlets versus JSP

Servlet

A Servlet is a Java class that runs on a web server and handles HTTP requests and responses.

Servlets are mainly suitable for:

- Processing requests.
- Performing business logic.
- Calling services and databases.
- Controlling application flow.

JSP

JSP stands for **JavaServer Pages**. It is mainly used to create the presentation layer of a Java web application.

JSP makes it easier to create HTML pages containing dynamic information.

Comparison

Feature	Servlet	JSP
Main purpose	Request processing and control	Presentation/UI
Development	More Java code	Easier for HTML/UI
Best MVC role	Controller	View
Suitable for	Business/request processing	Displaying dynamic content

Feature	Servlet	JSP
HTML creation	More difficult	Easier

In an MVC application, the **Servlet normally acts as the Controller**, while the **JSP acts as the View**.

When to choose a Servlet

I would choose a Servlet when the application needs to process a request, validate data, call a DAO, perform business logic, and determine which page should be displayed.

When to choose JSP

I would choose JSP when the main requirement is displaying dynamic information to the user, such as displaying a list of products or student records.

QUESTION THREE (b) – Shopping Cart Session Management

Likely cause

The most likely cause is that the shopping cart is not being stored in the user's HTTP session.

HTTP is fundamentally **stateless**, meaning the server does not automatically remember information from one request to another.

For example, if a customer adds a product during one request and the cart is stored only in a local variable, that information can be lost when the customer makes another request.

Two session-management techniques

1. HttpSession

HttpSession can be used to store the cart on the server for a particular user session.

Example:

```
HttpSession session = request.getSession();
```

```
List<String> cart =
```

```
    (List<String>) session.getAttribute("cart");
```

```
if (cart == null) {
```

```
    cart = new ArrayList<>();
```

```
}
```

```
cart.add("Laptop");
```

```
session.setAttribute("cart", cart);
```

2. Cookies

Cookies can store a small amount of information on the user's browser. A session identifier can also be stored in a cookie so that the server can associate subsequent requests with the correct session.

For a shopping cart, server-side HttpSession is generally preferable for storing the actual cart contents.

ShoppingCartServlet.java

```
package com.bit2203.servlet;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;

@WebServlet("/shopping-cart")
public class ShoppingCartServlet extends HttpServlet {

    private static final String CART_ATTRIBUTE = "cart";

    // Handles GET requests and displays the cart
    @Override
    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response)
```



```
throws ServletException, IOException {

    List<String> cart = getCart(request);

    response.setContentType("text/html");

    PrintWriter out = response.getWriter();

    out.println("<html>");
    out.println("<body>");
    out.println("<h1>Shopping Cart</h1>");

    if (cart.isEmpty()) {

        out.println("<p>Your cart is empty.</p>");

    } else {

        out.println("<ul>");

        for (String item : cart) {
            out.println("<li>" + item + "</li>");
        }

        out.println("</ul>");
    }

    out.println("</body>");
    out.println("</html>");
}
```

```

// Handles POST requests for adding items
@Override
protected void doPost(
    HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {

    String item = request.getParameter("item");

    if (item != null && !item.trim().isEmpty()) {

        HttpSession session = request.getSession();

        List<String> cart = getCart(request);

        cart.add(item);

        session.setAttribute(CART_ATTRIBUTE, cart);
    }

    response.sendRedirect("shopping-cart");
}

// Retrieves the cart from the current user's session
@SuppressWarnings("unchecked")
private List<String> getCart(HttpServletRequest request) {

    HttpSession session = request.getSession();

    List<String> cart =
        (List<String>) session.getAttribute(CART_ATTRIBUTE);

```

```
// Create a new cart if one does not exist
if (cart == null) {

    cart = new ArrayList<>();

    session.setAttribute(CART_ATTRIBUTE, cart);
}

return cart;
}
}
```

QUESTION FOUR – 8 MARKS

(a) Microservices Architecture

Microservices architecture is a software architecture in which a large application is divided into several small, independent services.

Each service performs a specific business function and can communicate with other services through APIs or messaging systems.

For example, an e-commerce system could contain:

- User Service
- Product Service
- Payment Service
- Order Service
- Delivery Service

Each service can be developed, deployed, and scaled independently.

Benefits

1. Independent deployment

A single service can be updated or deployed without necessarily redeploying the entire application.

2. Scalability

Individual services can be scaled according to demand.

For example, during a busy shopping period, the Order Service can be given additional resources without scaling every other service.

Challenges

1. Increased complexity

A microservices system contains many services, databases, APIs, and communication channels, making the system more difficult to manage.

2. Network and communication problems

Services communicate over networks, so problems such as latency, network failures, and service unavailability can occur.

Real-world example

Netflix is a well-known example of a company that uses microservices architecture. Its large platform is divided into numerous independent services that support different functions of the streaming platform.

QUESTION FOUR (b) – Dependency Injection

Dependency Injection (DI) is a design technique where an object's dependencies are supplied to it from outside instead of the object creating those dependencies itself.

For example, instead of UserService creating a UserRepository using new, Spring provides the repository to the service.

Benefits of DI

- Reduces coupling.
- Makes applications easier to test.
- Makes components easier to replace.
- Improves maintainability.
- Supports clean application architecture.

Constructor Injection

In constructor injection, dependencies are supplied through the class constructor.

Advantage

The dependency can be made final, ensuring that the object cannot exist without its required dependency.

Setter Injection

Setter injection supplies dependencies through setter methods.

Advantage

It is useful when a dependency is optional or needs to be changed after object creation.

Comparison

Constructor Injection

Dependency supplied through constructor

Good for required dependencies

Supports immutable dependencies

Makes dependencies obvious

Setter Injection

Dependency supplied through setter

Good for optional dependencies

Dependency can be changed

More flexible after construction

UserRepository.java

```
package com.bit2203.spring;
```

```
import org.springframework.stereotype.Repository;
```

```
@Repository
```

```
public class UserRepository {
```

```
    // Example repository method
```

```
    public String findUserById(int userId) {
```

```
        return "User with ID: " + userId;
```

```
    }
```

```
}
```

UserService.java

```
package com.bit2203.spring;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class UserService {
```

```
private final UserRepository userRepository;

// Constructor injection
public UserService(UserRepository userRepository) {
    this.userRepository = userRepository;
}

// Service method
public String getUser(int userId) {

    return userRepository.findUserById(userId);
}
}
```

AppConfig.java

```
package com.bit2203.spring;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.bit2203.spring")
public class AppConfig {

    // Spring automatically detects
    // @Service and @Repository classes.
}
```

QUESTION FIVE – 6 MARKS

(a) Remote Method Invocation (RMI)

Java RMI, or **Remote Method Invocation**, is a Java technology that allows an object running in one Java Virtual Machine to invoke methods on an object running in another Java Virtual Machine.

RMI allows Java applications running on different computers or processes to communicate as though they were calling methods on local objects.

RMI Registry

The RMI Registry is a naming service.

It stores references to remote objects using names so that clients can locate those objects.

Server

The server:

5. Implements the remote interface.
6. Creates the remote object.
7. Exports the object.
8. Registers the object with the RMI Registry.

Client

The client:

5. Connects to the RMI Registry.
6. Looks up the remote object by name.
7. Obtains a reference to the remote service.
8. Invokes methods on the remote object.

QUESTION FIVE (b) – Java RMI Program

TemperatureConverter.java

```
import java.rmi.Remote;
```

```
import java.rmi.RemoteException;
```

```
// Remote interface containing methods
```

```
// that can be called by the client.
```

```
public interface TemperatureConverter extends Remote {
```

```
    double celsiusToFahrenheit(double celsius)
```

```
        throws RemoteException;
```

```
        double fahrenheitToCelsius(double fahrenheit)
            throws RemoteException;
    }
```

TemperatureConverterImpl.java

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

// Implementation of the remote interface
public class TemperatureConverterImpl
    extends UnicastRemoteObject
    implements TemperatureConverter {

    // Constructor
    public TemperatureConverterImpl()
        throws RemoteException {
        super();
    }

    // Converts Celsius to Fahrenheit
    @Override
    public double celsiusToFahrenheit(double celsius)
        throws RemoteException {

        return (celsius * 9 / 5) + 32;
    }

    // Converts Fahrenheit to Celsius
    @Override
    public double fahrenheitToCelsius(double fahrenheit)
        throws RemoteException {
```



```
        return (fahrenheit - 32) * 5 / 9;
    }
}
```

RMI Server.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

// RMI server
public class RMIServer {

    public static void main(String[] args) {

        try {
            // Create the remote service
            TemperatureConverter converter =
                new TemperatureConverterImpl();

            // Start the RMI registry on port 1099
            Registry registry =
                LocateRegistry.createRegistry(1099);

            // Bind the remote object to a name
            registry.rebind(
                "TemperatureConverter",
                converter
            );

            System.out.println(
                "Temperature Converter RMI Server is running..."
            );
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

    );

    } catch (Exception e) {

        System.out.println(
            "Server error: " + e.getMessage()
        );

        e.printStackTrace();
    }
}
}

```

RMIClient.java

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

// RMI client
public class RMIClient {

    public static void main(String[] args) {

        try {
            // Locate the RMI registry
            Registry registry =
                LocateRegistry.getRegistry(
                    "localhost", 1099
                );

            // Look up the remote service
            TemperatureConverter converter =

```

```

        (TemperatureConverter)
        registry.lookup("TemperatureConverter");

    // Convert 25 degrees Celsius
    double result =
        converter.celsiusToFahrenheit(25);

    System.out.println(
        "25°C = " + result + "°F"
    );

} catch (Exception e) {

    System.out.println(
        "Client error: " + e.getMessage()
    );

    e.printStackTrace();
}
}
}

```

Expected output

25°C = 77.0°F

QUESTION SIX – 6 MARKS

(a) Web Container

A **web container**, also called a Servlet container, is a component of a Java web server that manages the execution of Java Servlets.

Examples include Apache Tomcat, Jetty, and other Jakarta Servlet-compatible servers.

Four responsibilities

5. Servlet lifecycle management

The container loads, initializes, executes, and destroys Servlets.

6. **Request and response management**

It receives HTTP requests and provides them to the appropriate Servlet.

7. **Session management**

It manages HTTP sessions and helps maintain user state.

8. **Security and resource management**

It can provide authentication, authorization, URL mapping, and manage Servlet resources.

QUESTION SIX (b) – Servlet Lifecycle

The main stages of a Servlet lifecycle are:

1. Loading and Instantiation

The web container loads the Servlet class and creates an instance of it.

2. Initialization – init()

The container calls the init() method once.

This is used for initialization tasks such as loading configuration information or resources.

3. Request Processing – service()

For every client request, the container calls the service() method.

For an HttpServlet, the service() method determines whether the request is GET, POST, PUT, DELETE, etc., and calls methods such as doGet() or doPost().

4. Destruction – destroy()

When the Servlet is removed from service, the container calls destroy() once.

This is used to release resources.

LifecycleDemoServlet.java

```
package com.bit2203.servlet;
```

```
import jakarta.servlet.ServletException;
```

```
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.io.IOException;
```

```
import java.io.PrintWriter;

@WebServlet("/lifecycle")
public class LifecycleDemoServlet extends HttpServlet {

    // Called once when the Servlet is initialized
    @Override
    public void init() throws ServletException {

        System.out.println(
            "LifecycleDemoServlet: init() called."
        );
    }

    // Called for each HTTP request
    @Override
    protected void service(
        HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        System.out.println(
            "LifecycleDemoServlet: service() called."
        );

        // Pass the request to the parent implementation.
        super.service(request, response);
    }

    // Handles GET requests
    @Override
```

```

protected void doGet(
    HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {

    System.out.println(
        "LifecycleDemoServlet: doGet() called."
    );

    response.setContentType("text/html");

    PrintWriter out = response.getWriter();

    out.println("<h1>Servlet Lifecycle Demo</h1>");
    out.println("<p>The Servlet is processing the request.</p>");
}

// Called once before the Servlet is destroyed
@Override
public void destroy() {

    System.out.println(
        "LifecycleDemoServlet: destroy() called."
    );
}
}

```

Lifecycle sequence

Load Servlet



Create Servlet object



init()



service()



doGet() / doPost()



service()



doGet() / doPost()



destroy()

QUESTION SEVEN – 8 MARKS

(a) Model-View-Controller (MVC)

MVC stands for **Model-View-Controller**.

It is a software design pattern that separates an application into three main components.

1. Model

The Model represents the application's data and business logic.

Responsibilities include:

- Representing data.
- Applying business rules.
- Communicating with databases.
- Managing application data.

Example:

Book

Student

Product

User

2. View

The View is the presentation layer.

It is responsible for displaying information to the user.

Examples include:

- JSP pages.
- HTML pages.
- User interfaces.

3. Controller

The Controller receives requests from the user and determines what should happen.

Responsibilities include:

- Receiving HTTP requests.
- Processing input.
- Calling the Model/DAO.
- Selecting the appropriate View.
- Passing data to the View.

MVC flow

User



Controller



Model / DAO



Database



Model / DAO



Controller



View



User

Real-world example

An **online shopping system** can use MVC.

For example:

- **Model:** Product, Order and Customer classes.
- **View:** JSP pages displaying products.
- **Controller:** Servlets handling product and order requests.
- **Database:** Stores product, customer and order information.

Many enterprise Java web applications use the MVC approach because it separates presentation, processing, and data management.

QUESTION SEVEN (b) – Library Management System

Model

The Model represents library data.

For this system, the Book class represents individual books.

It can contain:

- Book ID.
- Title.
- Author.
- ISBN.

DAO

The Data Access Object, BookDAO, communicates with the database and retrieves books.

Controller

BookController is a Servlet that:

6. Receives the request.
7. Calls BookDAO.
8. Retrieves the books.
9. Places them into the request.
10. Forwards the request to the JSP.

View

bookList.jsp displays the books to the user.

Book.java

```
package com.bit2203.library.model;
```

```
// Model class representing a book

public class Book {

    private int id;

    private String title;

    private String author;

    private String isbn;

    // Constructor

    public Book(int id, String title, String author, String isbn) {

        this.id = id;

        this.title = title;

        this.author = author;

        this.isbn = isbn;

    }

    // Getters

    public int getId() {

        return id;

    }

    public String getTitle() {

        return title;

    }

    public String getAuthor() {

        return author;

    }

    public String getIsbn() {

        return isbn;

    }

}
```

```
}

// Setters
public void setId(int id) {
    this.id = id;
}

public void setTitle(String title) {
    this.title = title;
}

public void setAuthor(String author) {
    this.author = author;
}

public void setIsbn(String isbn) {
    this.isbn = isbn;
}
}
```

BookDAO.java

```
package com.bit2203.library.dao;

import com.bit2203.library.model.Book;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

```
import java.util.ArrayList;

import java.util.List;


// DAO responsible for accessing book data
public class BookDAO {


    private static final String URL =

        "jdbc:mysql://localhost:3306/library_db";


    private static final String USER = "admin";
    private static final String PASSWORD = "secure123";


    // Retrieves all books from the database
    public List<Book> getAllBooks() {


        List<Book> books = new ArrayList<>();


        String sql =

            "SELECT id, title, author, isbn FROM books";


        try (

            Connection connection =

                DriverManager.getConnection(

                    URL, USER, PASSWORD);


            PreparedStatement statement =

                connection.prepareStatement(sql);


            ResultSet resultSet =

                statement.executeQuery()

        ) {
```

```

        while (resultSet.next()) {

            Book book = new Book(
                resultSet.getInt("id"),
                resultSet.getString("title"),
                resultSet.getString("author"),
                resultSet.getString("isbn")
            );

            books.add(book);
        }

    } catch (SQLException e) {

        System.out.println(
            "Database error: " + e.getMessage()
        );
    }

    return books;
}
}

```

BookController.java

```

package com.bit2203.library.controller;

import com.bit2203.library.dao.BookDAO;
import com.bit2203.library.model.Book;

import jakarta.servlet.ServletException;

```

```
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.io.IOException;
import java.util.List;
```

```
// Controller Servlet for handling book requests
@WebServlet("/books")
public class BookController extends HttpServlet {
```

```
    private BookDAO bookDAO;
```

```
    // Create DAO when Servlet is initialized
```

```
    @Override
```

```
    public void init() {
```

```
        bookDAO = new BookDAO();
```

```
    }
```

```
    // Handles request for displaying all books
```

```
    @Override
```

```
    protected void doGet(
```

```
        HttpServletRequest request,
```

```
        HttpServletResponse response)
```

```
        throws ServletException, IOException {
```

```
        // Retrieve books from DAO
```

```
        List<Book> books = bookDAO.getAllBooks();
```

```
// Place books in request scope
request.setAttribute("books", books);

// Forward request to JSP View
request.getRequestDispatcher(
    "/bookList.jsp"
).forward(request, response);
}
}
```

bookList.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ page import="java.util.List" %>
<%@ page import="com.bit2203.library.model.Book" %>

<!DOCTYPE html>

<html>
<head>
    <title>Library Books</title>
</head>

<body>

<h1>Library Books</h1>

<%
    List<Book> books =
        (List<Book>) request.getAttribute("books");
%>

<table border="1" cellpadding="8">
```

```
<tr>
    <th>ID</th>
    <th>Title</th>
    <th>Author</th>
    <th>ISBN</th>
</tr>
```

```
<%
    if (books != null && !books.isEmpty()) {

        for (Book book : books) {
%>
```

```
<tr>
    <td><%= book.getId() %></td>
    <td><%= book.getTitle() %></td>
    <td><%= book.getAuthor() %></td>
    <td><%= book.getIsbn() %></td>
</tr>
```

```
<%
    }
```

```
    } else {
%>
```

```
<tr>
    <td colspan="4">
        No books found.
    </td>
```


</tr>

<%

}

%>

</table>

</body>

</html>

BONUS / EXTRA CREDIT – 2 MARKS

Inversion of Control (IoC)

Inversion of Control (IoC) is a principle in which the responsibility for creating and managing objects is transferred from the application code to a framework or container.

In traditional programming, a class may create its own dependencies:

```
UserRepository repository = new UserRepository();
```

With Spring IoC, the Spring container creates and manages the object and supplies its dependencies.

Relationship between IoC and DI

Dependency Injection is one of the main techniques used to implement IoC.

For example:

```
@Service
```

```
public class UserService {
```

```
    private final UserRepository repository;
```

```
    public UserService(UserRepository repository) {
```

```
        this.repository = repository;
```

```
    }
```

```
}
```

Spring creates UserRepository and injects it into UserService.

Importance of IoC

IoC is important because it:

- Reduces coupling.
- Improves testability.
- Makes components reusable.
- Simplifies object management.
- Makes large enterprise applications easier to maintain.

Therefore, IoC is a fundamental principle in modern enterprise Java development because frameworks such as Spring can manage object creation, dependencies, configuration, and application components.

ADDITIONAL QUESTIONS FOR EXTRA PRACTICE

1. Stateful versus Stateless Session Management

Stateful session management

Stateful session management means the server maintains information about a user's previous interactions.

Example: An online shopping cart where the server stores the user's cart in an HttpSession.

Stateless session management

Stateless communication means each request contains enough information for the server to process it without depending on previously stored server-side state.

Example: REST APIs using tokens such as JWTs for authentication.

Comparison

Stateful	Stateless
Server maintains user state	Server does not maintain session state
Uses more server resources	Easier to scale horizontally
Example: HttpSession shopping cart	Example: Token-based REST API

2. JAR FILE

JAR stands for **Java Archive**.

A JAR file packages Java classes, resources, configuration files, and metadata into one compressed file.

For example:

MyApplication.jar

A JAR can be made executable by specifying the application's main class in its manifest.

Example MANIFEST.MF

Manifest-Version: 1.0

Main-Class: com.example.Main

The JAR can then be executed using:

```
java -jar MyApplication.jar
```

3. MONOLITHIC VERSUS MICROSERVICES

A monolithic application is built as one large application where the components are packaged and deployed together.

Microservices architecture divides the application into multiple smaller and independently deployable services.

Monolithic

One large application

Usually deployed together

Simpler initially

Scaling may involve the whole application

Easier for small applications

Microservices

Multiple smaller services

Services can be deployed independently

More complex to manage

Individual services can be scaled

Suitable for large and complex systems

When to choose monolithic

A monolithic architecture is suitable when:

- The application is small.
- The development team is small.
- Simplicity is important.
- The application does not require independent scaling.

When to choose microservices

Microservices are suitable when:

- The application is large.
- Different teams need to work independently.
- Different components need independent scaling.
- Independent deployment is important.

GITHUB REPOSITORY STRUCTURE

Your repository should be named exactly:

BIT2203-AdvancedProgramming-CAT

A suitable structure is:

BIT2203-AdvancedProgramming-CAT/

```
|
|
| └─ README.md
|
|
| └─ src/
|   |
|   |
|   | └─ question1/
|   |   └─ URLParser.java
|   |
|   |
|   | └─ question2/
|   |   └─ StudentLogin.java
|   |   └─ UniversityDatabase.java
|   |
|   |
|   | └─ question3/
|   |   └─ ShoppingCartServlet.java
|   |
|   |
|   | └─ question4/
|   |   └─ UserRepository.java
|   |   └─ UserService.java
|   |   └─ AppConfig.java
|   |
|   |
|   | └─ question5/
|   |   └─ TemperatureConverter.java
|   |   └─ TemperatureConverterImpl.java
|   |   └─ RMIServer.java
|   |   └─ RMIClient.java
```

```
| |
| └─ question6/
| |   └─ LifecycleDemoServlet.java
| |
| |
| └─ question7/
|   └─ Book.java
|   └─ BookDAO.java
|   └─ BookController.java
|
└─ web/
    └─ bookList.jsp
```

README.md

Your repository should contain a README explaining the programs.

BIT2203-AdvancedProgramming-CAT

Student Information

Name: YOUR NAME

Registration Number: YOUR REGISTRATION NUMBER

Description

This repository contains Java programs developed for the BIT 2203

Advanced Programming Take-Away CAT.

Programs

Question 1 - URLParser

URLParser.java demonstrates how Java's URL class can be used to extract URL components including the protocol, host, port, path, query parameters and fragment.

Question 2 - JDBC

StudentLogin.java demonstrates secure student authentication using PreparedStatement.

UniversityDatabase.java connects to the university_db database, creates the courses table, inserts sample courses and retrieves Computer Science courses.

Question 3 - Shopping Cart

ShoppingCartServlet.java demonstrates HTTP session management using HttpSession to maintain a customer's shopping cart.

Question 4 - Spring Dependency Injection

UserRepository.java provides the repository dependency.

UserService.java demonstrates constructor dependency injection.

AppConfig.java configures component scanning in Spring.

Question 5 - Java RMI

TemperatureConverter.java is the remote interface.

TemperatureConverterImpl.java implements the remote service.

RMIServer.java starts the RMI registry and binds the service.

RMIClient.java looks up the service and converts 25 degrees Celsius to Fahrenheit.

Question 6 - Servlet Lifecycle

LifecycleDemoServlet.java demonstrates the Servlet lifecycle methods including init(), service(), doGet() and destroy().

Question 7 - MVC Library System

Book.java represents the Model.

BookDAO.java accesses book data.

BookController.java acts as the Servlet Controller.

bookList.jsp acts as the View and displays the books.

Requirements

- Java JDK
- MySQL Server
- MySQL Connector/J
- Apache Tomcat 10 or later for Jakarta Servlet applications
- Spring Framework for the Spring example
- An IDE such as IntelliJ IDEA, Eclipse or NetBeans

Database Setup

Create the required database using:

```
CREATE DATABASE university_db;
```

For the library example:

```
CREATE DATABASE library_db;
```

Update database credentials in the Java files if necessary.

Running the RMI Program

Compile the RMI classes:

```
javac *.java
```

Start the server:

```
java RMIServer
```

In another terminal, run:

```
java RMIClient
```

Expected result:

25°C = 77.0°F

Servlet Programs

Deploy the Servlet project to a Jakarta Servlet-compatible web container such as Apache Tomcat 10 or later.

GitHub

All Java source files and configuration files are included in this repository.

FINAL SUBMISSION CHECKLIST

Before submitting, ensure you have:

- ☐ Created a **public** GitHub repository.
- ☐ Named it BIT2203-AdvancedProgramming-CAT.
- ☐ Added all required .java files.
- ☐ Added README.md.
- ☐ Added the JSP file.
- ☐ Added configuration files where applicable.
- ☐ Added comments to your Java programs.
- ☐ Tested the programs.
- ☐ Pushed all files to GitHub.
- ☐ Added your real name to the document.
- ☐ Added your registration number.
- ☐ Added your GitHub repository link.
- ☐ Saved the typed answers as PDF or Word.
- ☐ Checked that the GitHub repository is **public**.
- ☐ Verified that the GitHub link actually opens before submission.

Required Java files

URLParser.java

StudentLogin.java

UniversityDatabase.java
ShoppingCartServlet.java
UserRepository.java
UserService.java
AppConfig.java
TemperatureConverter.java
TemperatureConverterImpl.java
RMIServer.java
RMIClient.java
LifecycleDemoServlet.java
Book.java
BookDAO.java
BookController.java
bookList.jsp
README.md

Important implementation note: The Servlet examples above use `jakarta.servlet.*`, which is appropriate for modern Servlet containers such as Tomcat 10+. If your lecturer/lab uses Tomcat 9 or an older Java EE setup, the imports may need to use `javax.servlet.*` instead.

* protocol. Before data is transmitted, a connection is established between the client and server. TCP ensures that data packets arrive, arrive in the correct order, and are retransmitted if they are lost.

****Characteristics of TCP:****

- * Connection-oriented.
- * Reliable data delivery.
- * Packets are delivered in order.
- * Error detection and retransmission are supported.
- * Slower than UDP because of its reliability mechanisms.

UDP

UDP is a **connectionless and faster** protocol. It sends data without first establishing a connection. It does not guarantee that packets will arrive or that they will arrive in the correct order.

Characteristics of UDP:

- * Connectionless.
- * Faster than TCP.
- * Lower overhead.
- * Does not guarantee delivery.
- * Suitable for applications where speed is more important than perfect reliability.

Scenario where UDP is preferred

UDP would be preferred in **live video conferencing or online gaming**.

For example, in an online game, if one packet containing a player's position is lost, it is normally better to continue receiving newer position updates rather than wait for the old packet to be retransmitted. The small amount of lost information is less important than maintaining real-time communication.

Therefore, UDP is preferred because it provides **low latency and faster communication**, which is important for real-time applications.

(b) URLParser.java

The given URL is:

``https://api.techstore.com:8443/products/category/electronics?sort=price&limit=20#reviews``

The following program uses Java's ``URL`` class to extract the required components.

URLParser.java

```
```java
```

```
import java.net.URL;
```

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
public class URLParser {
```

```
 public static void main(String[] args) {
```

```
 try {
```

```
 // Create a URL object
```

```
 URL url = new URL(
```

```
 "https://api.techstore.com:8443/products/category/electronics"
```

```
 + "?sort=price&limit=20#reviews"
```

```
);
```

```
 // Display URL components
```

```
 System.out.println("Protocol: " + url.getProtocol());
```

```
 System.out.println("Host: " + url.getHost());
```

```
 System.out.println("Port: " + url.getPort());
```

```
 System.out.println("Path: " + url.getPath());
```

```
 // Extract and display query parameters
```

```
 String query = url.getQuery();
```

```
 System.out.println("Query Parameters:");
```

```
 if (query != null) {
```

```

String[] parameters = query.split("&");

Map<String, String> queryParameters = new HashMap<>();

for (String parameter : parameters) {
 String[] pair = parameter.split("=", 2);

 if (pair.length == 2) {
 queryParameters.put(pair[0], pair[1]);
 }
}

for (Map.Entry<String, String> entry : queryParameters.entrySet()) {
 System.out.println(
 entry.getKey() + " = " + entry.getValue()
);
}

// Display fragment
System.out.println("Fragment: " + url.getRef());

} catch (Exception e) {
 System.out.println("Error parsing URL: " + e.getMessage());
}

}

'''

```

### Expected output

```
```text
```

Protocol: https

Host: api.techstore.com

Port: 8443

Path: /products/category/electronics

Query Parameters:

sort = price

limit = 20

Fragment: reviews

```
```
```

```

```

# QUESTION TWO – 8 MARKS

## (a) PreparedStatement versus Statement

`PreparedStatement` is more secure and efficient than `Statement`, especially when handling login credentials.

### Security

A normal `Statement` can be vulnerable to **SQL injection** when user input is directly inserted into an SQL statement.

For example:

```
```java
```

```
String sql = "SELECT * FROM students WHERE reg_no = "  
    + regNo + " AND password = " + password + """;
```

```
```
```

A malicious user could enter SQL commands as part of the input.

`PreparedStatement` separates SQL code from user-supplied values. Values are treated as data rather than executable SQL commands.

### ### Efficiency

A prepared statement can be precompiled by the database and reused with different values. This can improve performance when the same SQL statement is executed multiple times.

### ### PreparedStatement authentication example

```
```java
import java.sql.*;

public class StudentLogin {

    public static boolean authenticate(String regNo, String password) {

        String url = "jdbc:mysql://localhost:3306/university_db";
        String username = "admin";
        String dbPassword = "secure123";

        String sql = "SELECT * FROM students "
            + "WHERE registration_number = ? AND password = ?";

        try (
            Connection connection =
                DriverManager.getConnection(url, username, dbPassword);

            PreparedStatement statement =
                connection.prepareStatement(sql)
```

```

    ){

        // Set values safely
        statement.setString(1, regNo);
        statement.setString(2, password);

        try (ResultSet resultSet = statement.executeQuery()) {

            return resultSet.next();
        }

    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
        return false;
    }
}

public static void main(String[] args) {

    String registrationNumber = "BIT/001/2026";
    String password = "mypassword";

    if (authenticate(registrationNumber, password)) {
        System.out.println("Login successful.");
    } else {
        System.out.println("Invalid registration number or password.");
    }
}
}
'''

```


****Note:**** In a real production application, passwords should not be stored as plain text. They should be securely hashed using a password-hashing algorithm such as BCrypt or Argon2.

QUESTION TWO (b) – UniversityDatabase.java

The program below:

1. Connects to `university\_db`.
2. Creates the `courses` table.
3. Inserts three courses.
4. Retrieves courses belonging to the Computer Science department.

UniversityDatabase.java

```
```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class UniversityDatabase {

 // Database connection details
 private static final String URL =
 "jdbc:mysql://localhost:3306/university_db";

 private static final String USER = "admin";
```

```

private static final String PASSWORD = "secure123";

public static void main(String[] args) {

 // SQL statement for creating the courses table
 String createTableSQL = ""

 CREATE TABLE IF NOT EXISTS courses (

 course_id INT PRIMARY KEY,

 course_name VARCHAR(100),

 credits INT,

 department VARCHAR(50)

)

 """;

 // SQL statement for inserting courses
 String insertSQL = ""

 INSERT INTO courses

 (course_id, course_name, credits, department)

 VALUES (?, ?, ?, ?)

 """;

 // SQL statement for retrieving Computer Science courses
 String selectSQL =

 "SELECT * FROM courses WHERE department = ?";

 try (Connection connection =

 DriverManager.getConnection(URL, USER, PASSWORD)) {

 System.out.println("Database connected successfully.");

 // Create table

```

```
try (Statement statement = connection.createStatement()) {

 statement.executeUpdate(createTableSQL);

 System.out.println("Courses table created successfully.");
}
```

```
// Insert sample courses
```

```
try (PreparedStatement statement =
 connection.prepareStatement(insertSQL)) {
```

```
 insertCourse(statement, 101,
 "Advanced Programming", 3,
 "Computer Science");
```

```
 insertCourse(statement, 102,
 "Database Systems", 3,
 "Computer Science");
```

```
 insertCourse(statement, 103,
 "Computer Networks", 3,
 "Computer Science");
```

```
 System.out.println("Sample courses inserted.");
}
```

```
// Retrieve Computer Science courses
```

```
try (PreparedStatement statement =
 connection.prepareStatement(selectSQL)) {
```

```
 statement.setString(1, "Computer Science");
```

```

try (ResultSet resultSet = statement.executeQuery()) {

 System.out.println("\nComputer Science Courses:");

 while (resultSet.next()) {

 System.out.println(
 "ID: " + resultSet.getInt("course_id")
 + ", Name: " + resultSet.getString("course_name")
 + ", Credits: " + resultSet.getInt("credits")
 + ", Department: "
 + resultSet.getString("department")
);
 }
}

} catch (SQLException e) {
 System.out.println("Database error: " + e.getMessage());
}

}

// Method used to insert one course
private static void insertCourse(
 PreparedStatement statement,
 int id,
 String name,
 int credits,
 String department) throws SQLException {

```

```

 statement.setInt(1, id);
 statement.setString(2, name);
 statement.setInt(3, credits);
 statement.setString(4, department);

 statement.executeUpdate();
 }
}
'''

```

### Required JDBC dependency

For the above version, the project uses **\*\*MySQL Connector/J\*\***.

The database should be created before running the program:

```

```sql
CREATE DATABASE university_db;
'''

---

```

QUESTION THREE – 8 MARKS

(a) Servlets versus JSP

Servlet

A Servlet is a Java class that runs on a web server and handles HTTP requests and responses.

Servlets are mainly suitable for:

- * Processing requests.
- * Performing business logic.
- * Calling services and databases.
- * Controlling application flow.

JSP

JSP stands for **JavaServer Pages**. It is mainly used to create the presentation layer of a Java web application.

JSP makes it easier to create HTML pages containing dynamic information.

Comparison

Feature	Servlet	JSP
-----	-----	-----
Main purpose	Request processing and control	Presentation/UI
Development	More Java code	Easier for HTML/UI
Best MVC role	Controller	View
Suitable for	Business/request processing	Displaying dynamic content
HTML creation	More difficult	Easier

In an MVC application, the **Servlet** normally acts as the **Controller**, while the **JSP** acts as the **View**.

When to choose a Servlet

I would choose a Servlet when the application needs to process a request, validate data, call a DAO, perform business logic, and determine which page should be displayed.

When to choose JSP

I would choose JSP when the main requirement is displaying dynamic information to the user, such as displaying a list of products or student records.

QUESTION THREE (b) – Shopping Cart Session Management

Likely cause

The most likely cause is that the shopping cart is not being stored in the user's HTTP session.

HTTP is fundamentally **stateless**, meaning the server does not automatically remember information from one request to another.

For example, if a customer adds a product during one request and the cart is stored only in a local variable, that information can be lost when the customer makes another request.

Two session-management techniques

1. HttpSession

`HttpSession` can be used to store the cart on the server for a particular user session.

Example:

```
```java
```

```
HttpSession session = request.getSession();
```

```
List<String> cart =
```

```
 (List<String>) session.getAttribute("cart");
```

```

if (cart == null) {
 cart = new ArrayList<>();
}

cart.add("Laptop");

session.setAttribute("cart", cart);
...

```

## #### 2. Cookies

Cookies can store a small amount of information on the user's browser. A session identifier can also be stored in a cookie so that the server can associate subsequent requests with the correct session.

For a shopping cart, server-side `HttpSession` is generally preferable for storing the actual cart contents.

---

```
ShoppingCartServlet.java
```

```

``java
package com.bit2203.servlet;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;

```



```
import java.io.PrintWriter;

import java.util.ArrayList;

import java.util.List;

@WebServlet("/shopping-cart")

public class ShoppingCartServlet extends HttpServlet {

 private static final String CART_ATTRIBUTE = "cart";

 // Handles GET requests and displays the cart

 @Override

 protected void doGet(

 HttpServletRequest request,

 HttpServletResponse response)

 throws ServletException, IOException {

 List<String> cart = getCart(request);

 response.setContentType("text/html");

 PrintWriter out = response.getWriter();

 out.println("<html>");
 out.println("<body>");
 out.println("<h1>Shopping Cart</h1>");

 if (cart.isEmpty()) {

 out.println("<p>Your cart is empty.</p>");

 } else {
```

```

 out.println("");

 for (String item : cart) {
 out.println("" + item + "");
 }

 out.println("");
 }

 out.println("</body>");
 out.println("</html>");
}

// Handles POST requests for adding items
@Override
protected void doPost(
 HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {

 String item = request.getParameter("item");

 if (item != null && !item.trim().isEmpty()) {

 HttpSession session = request.getSession();

 List<String> cart = getCart(request);

 cart.add(item);
 }
}

```

```

 session.setAttribute(CART_ATTRIBUTE, cart);
 }

 response.sendRedirect("shopping-cart");
}

// Retrieves the cart from the current user's session
@SuppressWarnings("unchecked")
private List<String> getCart(HttpServletRequest request) {

 HttpSession session = request.getSession();

 List<String> cart =
 (List<String>) session.getAttribute(CART_ATTRIBUTE);

 // Create a new cart if one does not exist
 if (cart == null) {

 cart = new ArrayList<>();

 session.setAttribute(CART_ATTRIBUTE, cart);
 }

 return cart;
}
}
...

```

# QUESTION FOUR – 8 MARKS

## ## (a) Microservices Architecture

Microservices architecture is a software architecture in which a large application is divided into several small, independent services.

Each service performs a specific business function and can communicate with other services through APIs or messaging systems.

For example, an e-commerce system could contain:

- \* User Service
- \* Product Service
- \* Payment Service
- \* Order Service
- \* Delivery Service

Each service can be developed, deployed, and scaled independently.

### ### Benefits

#### #### 1. Independent deployment

A single service can be updated or deployed without necessarily redeploying the entire application.

#### #### 2. Scalability

Individual services can be scaled according to demand.

For example, during a busy shopping period, the Order Service can be given additional resources without scaling every other service.

### ### Challenges

#### #### 1. Increased complexity

A microservices system contains many services, databases, APIs, and communication channels, making the system more difficult to manage.

#### #### 2. Network and communication problems

Services communicate over networks, so problems such as latency, network failures, and service unavailability can occur.

### ### Real-world example

**\*\*Netflix\*\*** is a well-known example of a company that uses microservices architecture. Its large platform is divided into numerous independent services that support different functions of the streaming platform.

---

### # QUESTION FOUR (b) – Dependency Injection

Dependency Injection (DI) is a design technique where an object's dependencies are supplied to it from outside instead of the object creating those dependencies itself.

For example, instead of ``UserService`` creating a ``UserRepository`` using ``new``, Spring provides the repository to the service.

### ### Benefits of DI

- \* Reduces coupling.
- \* Makes applications easier to test.
- \* Makes components easier to replace.

- \* Improves maintainability.
- \* Supports clean application architecture.

## ## Constructor Injection

In constructor injection, dependencies are supplied through the class constructor.

### ### Advantage

The dependency can be made `final`, ensuring that the object cannot exist without its required dependency.

## ## Setter Injection

Setter injection supplies dependencies through setter methods.

### ### Advantage

It is useful when a dependency is optional or needs to be changed after object creation.

### ### Comparison

Constructor Injection	Setter Injection	
-----	-----	
Dependency supplied through constructor	Dependency supplied through setter	
Good for required dependencies	Good for optional dependencies	
Supports immutable dependencies	Dependency can be changed	
Makes dependencies obvious	More flexible after construction	

---

```
UserRepository.java
```

```
```java
```

```
package com.bit2203.spring;
```

```
import org.springframework.stereotype.Repository;
```

```
@Repository
```

```
public class UserRepository {
```

```
    // Example repository method
```

```
    public String findUserById(int userId) {
```

```
        return "User with ID: " + userId;
```

```
    }
```

```
}
```

```
```
```

```

```

```
UserService.java
```

```
```java
```

```
package com.bit2203.spring;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class UserService {
```

```
    private final UserRepository userRepository;
```

```

// Constructor injection
public UserService(UserRepository userRepository) {
    this.userRepository = userRepository;
}

// Service method
public String getUser(int userId) {

    return userRepository.findUserById(userId);
}
}
...

---

## AppConfig.java

```java
package com.bit2203.spring;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.bit2203.spring")
public class AppConfig {

 // Spring automatically detects
 // @Service and @Repository classes.
}

```



...

---

## # QUESTION FIVE – 6 MARKS

### ## (a) Remote Method Invocation (RMI)

Java RMI, or **Remote Method Invocation**, is a Java technology that allows an object running in one Java Virtual Machine to invoke methods on an object running in another Java Virtual Machine.

RMI allows Java applications running on different computers or processes to communicate as though they were calling methods on local objects.

#### ### RMI Registry

The RMI Registry is a naming service.

It stores references to remote objects using names so that clients can locate those objects.

#### ### Server

The server:

1. Implements the remote interface.
2. Creates the remote object.
3. Exports the object.
4. Registers the object with the RMI Registry.

#### ### Client

The client:

1. Connects to the RMI Registry.
2. Looks up the remote object by name.
3. Obtains a reference to the remote service.
4. Invokes methods on the remote object.

---

#### # QUESTION FIVE (b) – Java RMI Program

## TemperatureConverter.java

```
``java
import java.rmi.Remote;
import java.rmi.RemoteException;

// Remote interface containing methods
// that can be called by the client.
public interface TemperatureConverter extends Remote {

 double celsiusToFahrenheit(double celsius)
 throws RemoteException;

 double fahrenheitToCelsius(double fahrenheit)
 throws RemoteException;
}
...

```

## TemperatureConverterImpl.java

```

```java
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

// Implementation of the remote interface
public class TemperatureConverterImpl
    extends UnicastRemoteObject
    implements TemperatureConverter {

    // Constructor
    public TemperatureConverterImpl()
        throws RemoteException {
        super();
    }

    // Converts Celsius to Fahrenheit
    @Override
    public double celsiusToFahrenheit(double celsius)
        throws RemoteException {

        return (celsius * 9 / 5) + 32;
    }

    // Converts Fahrenheit to Celsius
    @Override
    public double fahrenheitToCelsius(double fahrenheit)
        throws RemoteException {

        return (fahrenheit - 32) * 5 / 9;
    }
}

```

```
}  
'''
```

```
---
```

```
## RMIServer.java
```

```
```java
```

```
import java.rmi.registry.LocateRegistry;
```

```
import java.rmi.registry.Registry;
```

```
// RMI server
```

```
public class RMIServer {
```

```
 public static void main(String[] args) {
```

```
 try {
```

```
 // Create the remote service
```

```
 TemperatureConverter converter =
```

```
 new TemperatureConverterImpl();
```

```
 // Start the RMI registry on port 1099
```

```
 Registry registry =
```

```
 LocateRegistry.createRegistry(1099);
```

```
 // Bind the remote object to a name
```

```
 registry.rebind(
```

```
 "TemperatureConverter",
```

```
 converter
```

```
);
```

```

 System.out.println(
 "Temperature Converter RMI Server is running..."
);

 } catch (Exception e) {

 System.out.println(
 "Server error: " + e.getMessage()
);

 e.printStackTrace();
 }
}
...

```

## RMIClient.java

```

```java
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

// RMI client
public class RMIClient {

    public static void main(String[] args) {

        try {
            // Locate the RMI registry

```

```

Registry registry =
    LocateRegistry.getRegistry(
        "localhost", 1099
    );

// Look up the remote service
TemperatureConverter converter =
    (TemperatureConverter)
        registry.lookup("TemperatureConverter");

// Convert 25 degrees Celsius
double result =
    converter.celsiusToFahrenheit(25);

System.out.println(
    "25°C = " + result + "°F"
);

} catch (Exception e) {

    System.out.println(
        "Client error: " + e.getMessage()
    );

    e.printStackTrace();
}
}
}
'''

```

Expected output

```text

25°C = 77.0°F

```

QUESTION SIX – 6 MARKS

(a) Web Container

A **web container**, also called a Servlet container, is a component of a Java web server that manages the execution of Java Servlets.

Examples include Apache Tomcat, Jetty, and other Jakarta Servlet-compatible servers.

Four responsibilities

1. **Servlet lifecycle management**

The container loads, initializes, executes, and destroys Servlets.

2. **Request and response management**

It receives HTTP requests and provides them to the appropriate Servlet.

3. **Session management**

It manages HTTP sessions and helps maintain user state.

4. **Security and resource management**

It can provide authentication, authorization, URL mapping, and manage Servlet resources.

QUESTION SIX (b) – Servlet Lifecycle

The main stages of a Servlet lifecycle are:

1. Loading and Instantiation

The web container loads the Servlet class and creates an instance of it.

2. Initialization – `init()`

The container calls the `init()` method once.

This is used for initialization tasks such as loading configuration information or resources.

3. Request Processing – `service()`

For every client request, the container calls the `service()` method.

For an `HttpServlet`, the `service()` method determines whether the request is GET, POST, PUT, DELETE, etc., and calls methods such as `doGet()` or `doPost()`.

4. Destruction – `destroy()`

When the Servlet is removed from service, the container calls `destroy()` once.

This is used to release resources.

LifecycleDemoServlet.java


```
```java
package com.bit2203.servlet;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;
import java.io.PrintWriter;

@WebServlet("/lifecycle")
public class LifecycleDemoServlet extends HttpServlet {

 // Called once when the Servlet is initialized
 @Override
 public void init() throws ServletException {

 System.out.println(
 "LifecycleDemoServlet: init() called."
);
 }

 // Called for each HTTP request
 @Override
 protected void service(
 HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {
```

```

 System.out.println(
 "LifecycleDemoServlet: service() called."
);

 // Pass the request to the parent implementation.
 super.service(request, response);
 }

 // Handles GET requests
 @Override
 protected void doGet(
 HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {

 System.out.println(
 "LifecycleDemoServlet: doGet() called."
);

 response.setContentType("text/html");

 PrintWriter out = response.getWriter();

 out.println("<h1>Servlet Lifecycle Demo</h1>");
 out.println("<p>The Servlet is processing the request.</p>");
 }

 // Called once before the Servlet is destroyed
 @Override
 public void destroy() {

```

```
 System.out.println(
 "LifecycleDemoServlet: destroy() called."
);
 }
}
...

```

### ### Lifecycle sequence

```text

Load Servlet



Create Servlet object



init()



service()



doGet() / doPost()



service()



doGet() / doPost()



destroy()

...

QUESTION SEVEN – 8 MARKS

(a) Model-View-Controller (MVC)

MVC stands for ****Model-View-Controller****.

It is a software design pattern that separates an application into three main components.

1. Model

The Model represents the application's data and business logic.

Responsibilities include:

- * Representing data.
- * Applying business rules.
- * Communicating with databases.
- * Managing application data.

Example:

```
```text
```

Book

Student

Product

User

```
```
```

2. View

The View is the presentation layer.

It is responsible for displaying information to the user.

Examples include:

- * JSP pages.
- * HTML pages.
- * User interfaces.

3. Controller

The Controller receives requests from the user and determines what should happen.

Responsibilities include:

- * Receiving HTTP requests.
- * Processing input.
- * Calling the Model/DAO.
- * Selecting the appropriate View.
- * Passing data to the View.

MVC flow

```text

User



Controller



Model / DAO



Database



Model / DAO



Controller



View



User

...

### ### Real-world example

An **online shopping system** can use MVC.

For example:

**Model:** Product, Order and Customer classes.

**View:** JSP pages displaying products.

**Controller:** Servlets handling product and order requests.

**Database:** Stores product, customer and order information.

Many enterprise Java web applications use the MVC approach because it separates presentation, processing, and data management.

---

### # QUESTION SEVEN (b) – Library Management System

#### ## Model

The Model represents library data.

For this system, the `Book` class represents individual books.

It can contain:

- \* Book ID.
- \* Title.
- \* Author.
- \* ISBN.

## DAO

The Data Access Object, `BookDAO`, communicates with the database and retrieves books.

## Controller

`BookController` is a Servlet that:

1. Receives the request.
2. Calls `BookDAO`.
3. Retrieves the books.
4. Places them into the request.
5. Forwards the request to the JSP.

## View

`bookList.jsp` displays the books to the user.

---

# Book.java

```
``java

package com.bit2203.library.model;

// Model class representing a book
public class Book {

 private int id;
 private String title;
 private String author;
 private String isbn;

 // Constructor
 public Book(int id, String title, String author, String isbn) {
 this.id = id;
 this.title = title;
 this.author = author;
 this.isbn = isbn;
 }

 // Getters
 public int getId() {
 return id;
 }

 public String getTitle() {
 return title;
 }

 public String getAuthor() {
 return author;
 }
}
```



```

 public String getIsbn() {
 return isbn;
 }

 // Setters
 public void setId(int id) {
 this.id = id;
 }

 public void setTitle(String title) {
 this.title = title;
 }

 public void setAuthor(String author) {
 this.author = author;
 }

 public void setIsbn(String isbn) {
 this.isbn = isbn;
 }
}

...

BookDAO.java

```java
package com.bit2203.library.dao;

```

```
import com.bit2203.library.model.Book;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import java.util.ArrayList;
import java.util.List;

// DAO responsible for accessing book data
public class BookDAO {

    private static final String URL =
        "jdbc:mysql://localhost:3306/library_db";

    private static final String USER = "admin";
    private static final String PASSWORD = "secure123";

    // Retrieves all books from the database
    public List<Book> getAllBooks() {

        List<Book> books = new ArrayList<>();

        String sql =
            "SELECT id, title, author, isbn FROM books";

        try (
            Connection connection =
                DriverManager.getConnection(
```

```
URL, USER, PASSWORD);
```

```
PreparedStatement statement =
```

```
    connection.prepareStatement(sql);
```

```
ResultSet resultSet =
```

```
    statement.executeQuery()
```

```
) {
```

```
    while (resultSet.next()) {
```

```
        Book book = new Book(
```

```
            resultSet.getInt("id"),
```

```
            resultSet.getString("title"),
```

```
            resultSet.getString("author"),
```

```
            resultSet.getString("isbn")
```

```
        );
```

```
        books.add(book);
```

```
    }
```

```
} catch (SQLException e) {
```

```
    System.out.println(
```

```
        "Database error: " + e.getMessage()
```

```
    );
```

```
}
```

```
return books;
```

```
}
```

```
}
```

...

BookController.java

```java

package com.bit2203.library.controller;

import com.bit2203.library.dao.BookDAO;

import com.bit2203.library.model.Book;

import jakarta.servlet.ServletException;

import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.HttpServlet;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

import java.util.List;

// Controller Servlet for handling book requests

@WebServlet("/books")

public class BookController extends HttpServlet {

private BookDAO bookDAO;

// Create DAO when Servlet is initialized

@Override

public void init() {

```

 bookDAO = new BookDAO();
 }

 // Handles request for displaying all books
 @Override
 protected void doGet(
 HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {

 // Retrieve books from DAO
 List<Book> books = bookDAO.getAllBooks();

 // Place books in request scope
 request.setAttribute("books", books);

 // Forward request to JSP View
 request.getRequestDispatcher(
 "/bookList.jsp"
).forward(request, response);
 }
}
...

bookList.jsp

```.jsp
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ page import="java.util.List" %>

```

```
<%@ page import="com.bit2203.library.model.Book" %>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Library Books</title>
```

```
</head>
```

```
<body>
```

```
<h1>Library Books</h1>
```

```
<%
```

```
    List<Book> books =
```

```
        (List<Book>) request.getAttribute("books");
```

```
%>
```

```
<table border="1" cellpadding="8">
```

```
    <tr>
```

```
        <th>ID</th>
```

```
        <th>Title</th>
```

```
        <th>Author</th>
```

```
        <th>ISBN</th>
```

```
    </tr>
```

```
<%
```

```
    if (books != null && !books.isEmpty()) {
```

```
        for (Book book : books) {
```

```
%>
```

```
<tr>

    <td><%= book.getId() %></td>

    <td><%= book.getTitle() %></td>

    <td><%= book.getAuthor() %></td>

    <td><%= book.getIsbn() %></td>

</tr>
```

```
<%
    }
```

```
    } else {
%>
```

```
<tr>

    <td colspan="4">

        No books found.

    </td>

</tr>
```

```
<%
    }
%>
```

```
</table>
```

```
</body>
</html>
'''
```

```
---
```

BONUS / EXTRA CREDIT – 2 MARKS

Inversion of Control (IoC)

Inversion of Control (IoC) is a principle in which the responsibility for creating and managing objects is transferred from the application code to a framework or container.

In traditional programming, a class may create its own dependencies:

```
```java
UserRepository repository = new UserRepository();
```
```

With Spring IoC, the Spring container creates and manages the object and supplies its dependencies.

Relationship between IoC and DI

Dependency Injection is one of the main techniques used to implement IoC.

For example:

```
```java
@Service
public class UserService {

 private final UserRepository repository;

 public UserService(UserRepository repository) {
 this.repository = repository;
 }
```
```



```
}  
...  

```

Spring creates `UserRepository` and injects it into `UserService`.

Importance of IoC

IoC is important because it:

- * Reduces coupling.
- * Improves testability.
- * Makes components reusable.
- * Simplifies object management.
- * Makes large enterprise applications easier to maintain.

Therefore, IoC is a fundamental principle in modern enterprise Java development because frameworks such as Spring can manage object creation, dependencies, configuration, and application components.

ADDITIONAL QUESTIONS FOR EXTRA PRACTICE

1. Stateful versus Stateless Session Management

Stateful session management

Stateful session management means the server maintains information about a user's previous interactions.

****Example:**** An online shopping cart where the server stores the user's cart in an `HttpSession`.

Stateless session management

Stateless communication means each request contains enough information for the server to process it without depending on previously stored server-side state.

Example: REST APIs using tokens such as JWTs for authentication.

Comparison

| Stateful | Stateless |
|------------------------------------|--|
| ----- | ----- |
| Server maintains user state | Server does not maintain session state |
| Uses more server resources | Easier to scale horizontally |
| Example: HttpSession shopping cart | Example: Token-based REST API |

2. JAR FILE

JAR stands for **Java Archive**.

A JAR file packages Java classes, resources, configuration files, and metadata into one compressed file.

For example:

```
``text
MyApplication.jar
``
```

A JAR can be made executable by specifying the application's main class in its manifest.

Example MANIFEST.MF

```
```text
```

```
Manifest-Version: 1.0
```

```
Main-Class: com.example.Main
```

```
```
```

The JAR can then be executed using:

```
```bash
```

```
java -jar MyApplication.jar
```

```
```
```

3. MONOLITHIC VERSUS MICROSERVICES

A monolithic application is built as one large application where the components are packaged and deployed together.

Microservices architecture divides the application into multiple smaller and independently deployable services.

| Monolithic | Microservices | |
|---|--|--|
| ----- | ----- | |
| One large application | Multiple smaller services | |
| Usually deployed together | Services can be deployed independently | |
| Simpler initially | More complex to manage | |
| Scaling may involve the whole application | Individual services can be scaled | |
| Easier for small applications | Suitable for large and complex systems | |

When to choose monolithic

A monolithic architecture is suitable when:

- * The application is small.
- * The development team is small.
- * Simplicity is important.
- * The application does not require independent scaling.

When to choose microservices

Microservices are suitable when:

- * The application is large.
- * Different teams need to work independently.
- * Different components need independent scaling.
- * Independent deployment is important.

GITHUB REPOSITORY STRUCTURE

Your repository should be named exactly:

```
```text
```

```
BIT2203-AdvancedProgramming-CAT
```

```
```
```

A suitable structure is:

```
```text
```

```
BIT2203-AdvancedProgramming-CAT/
```

```
|
|
| └─ README.md
|
|
| └─ src/
| |
| | └─ question1/
| | └─ URLParser.java
| |
| |
| | └─ question2/
| | └─ StudentLogin.java
| | └─ UniversityDatabase.java
| |
| |
| | └─ question3/
| | └─ ShoppingCartServlet.java
| |
| |
| | └─ question4/
| | └─ UserRepository.java
| | └─ UserService.java
| | └─ AppConfig.java
| |
| |
| | └─ question5/
| | └─ TemperatureConverter.java
| | └─ TemperatureConverterImpl.java
| | └─ RMIServer.java
| | └─ RMIClient.java
| |
| |
| | └─ question6/
| | └─ LifecycleDemoServlet.java
| |
| |
| └─ question7/
| └─ Book.java
```

```
| ├── BookDAO.java
| └── BookController.java
|
└── web/
 └── bookList.jsp
...

```

# README.md

Your repository should contain a README explaining the programs.

```markdown

BIT2203-AdvancedProgramming-CAT

Student Information

Name: YOUR NAME

Registration Number: YOUR REGISTRATION NUMBER

Description

This repository contains Java programs developed for the BIT 2203
Advanced Programming Take-Away CAT.

Programs

Question 1 - URLParser

URLParser.java demonstrates how Java's URL class can be used to extract URL components including the protocol, host, port, path, query parameters and fragment.

Question 2 - JDBC

StudentLogin.java demonstrates secure student authentication using PreparedStatement.

UniversityDatabase.java connects to the university_db database, creates the courses table, inserts sample courses and retrieves Computer Science courses.

Question 3 - Shopping Cart

ShoppingCartServlet.java demonstrates HTTP session management using HttpSession to maintain a customer's shopping cart.

Question 4 - Spring Dependency Injection

UserRepository.java provides the repository dependency.

UserService.java demonstrates constructor dependency injection.

AppConfig.java configures component scanning in Spring.

Question 5 - Java RMI

TemperatureConverter.java is the remote interface.

TemperatureConverterImpl.java implements the remote service.

RMIServer.java starts the RMI registry and binds the service.

RMIClient.java looks up the service and converts 25 degrees Celsius to Fahrenheit.

Question 6 - Servlet Lifecycle

LifecycleDemoServlet.java demonstrates the Servlet lifecycle methods including init(), service(), doGet() and destroy().

Question 7 - MVC Library System

Book.java represents the Model.

BookDAO.java accesses book data.

BookController.java acts as the Servlet Controller.

bookList.jsp acts as the View and displays the books.

Requirements

- Java JDK
- MySQL Server
- MySQL Connector/J
- Apache Tomcat 10 or later for Jakarta Servlet applications
- Spring Framework for the Spring example
- An IDE such as IntelliJ IDEA, Eclipse or NetBeans

Database Setup

Create the required database using:

```
CREATE DATABASE university_db;
```

For the library example:

```
CREATE DATABASE library_db;
```

Update database credentials in the Java files if necessary.

Running the RMI Program

Compile the RMI classes:

```
javac *.java
```

Start the server:

```
java RMIServer
```

In another terminal, run:

```
java RMIClient
```

Expected result:

25°C = 77.0°F

Servlet Programs

Deploy the Servlet project to a Jakarta Servlet-compatible web container such as Apache Tomcat 10 or later.

GitHub

All Java source files and configuration files are included in this repository.

...

FINAL SUBMISSION CHECKLIST

Before submitting, ensure you have:

- * ☐ Created a **public** GitHub repository.
- * ☐ Named it `BIT2203-AdvancedProgramming-CAT`.
- * ☐ Added all required `.java` files.
- * ☐ Added `README.md`.
- * ☐ Added the JSP file.
- * ☐ Added configuration files where applicable.
- * ☐ Added comments to your Java programs.
- * ☐ Tested the programs.
- * ☐ Pushed all files to GitHub.
- * ☐ Added your real name to the document.
- * ☐ Added your registration number.
- * ☐ Added your GitHub repository link.
- * ☐ Saved the typed answers as PDF or Word.
- * ☐ Checked that the GitHub repository is **public**.
- * ☐ Verified that the GitHub link actually opens before submission.

Required Java files

```text

URLParser.java

StudentLogin.java

UniversityDatabase.java

ShoppingCartServlet.java

UserRepository.java

UserService.java

AppConfig.java

TemperatureConverter.java

TemperatureConverterImpl.java

RMIserver.java

RMIclient.java

LifecycleDemoServlet.java

Book.java

BookDAO.java

BookController.java

bookList.jsp

README.md

```

****Important implementation note:**** The Servlet examples above use `jakarta.servlet.\*`, which is appropriate for modern Servlet containers such as Tomcat 10+. If your lecturer/lab uses Tomcat 9 or an older Java EE setup, the imports may need to use `javax.servlet.\*` instead. **BIT 2203 –**

ADVANCED PROGRAMMING

TAKE-AWAY CAT – COMPLETE ANSWERS

Name: _____

Registration Number: _____

GitHub Repository: <https://github.com/YOUR-USERNAME/BIT2203-AdvancedProgramming-CAT>

QUESTION ONE – 6 MARKS

(a) TCP versus UDP

TCP (Transmission Control Protocol) and **UDP (User Datagram Protocol)** are transport-layer protocols used for communication between client and server.

TCP

TCP is a **connection-oriented and reliable** protocol. Before data is transmitted, a connection is established between the client and server. TCP ensures that data packets arrive, arrive in the correct order, and are retransmitted if they are lost.

Characteristics of TCP:

- Connection-oriented.
- Reliable data delivery.
- Packets are delivered in order.
- Error detection and retransmission are supported.
- Slower than UDP because of its reliability mechanisms.

UDP

UDP is a **connectionless and faster** protocol. It sends data without first establishing a connection. It does not guarantee that packets will arrive or that they will arrive in the correct order.

Characteristics of UDP:

- Connectionless.
- Faster than TCP.
- Lower overhead.
- Does not guarantee delivery.
- Suitable for applications where speed is more important than perfect reliability.

Scenario where UDP is preferred

UDP would be preferred in **live video conferencing or online gaming**.

For example, in an online game, if one packet containing a player's position is lost, it is normally better to continue receiving newer position updates rather than wait for the old packet to be retransmitted. The small amount of lost information is less important than maintaining real-time communication.

Therefore, UDP is preferred because it provides **low latency and faster communication**, which is important for real-time applications.

(b) URLParser.java

The given URL is:

<https://api.techstore.com:8443/products/category/electronics?sort=price&limit=20#reviews>

The following program uses Java's URL class to extract the required components.

URLParser.java

```
import java.net.URL;

import java.util.HashMap;

import java.util.Map;

public class URLParser {

    public static void main(String[] args) {

        try {

            // Create a URL object

            URL url = new URL(

                "https://api.techstore.com:8443/products/category/electronics"

                + "?sort=price&limit=20#reviews"

            );

            // Display URL components

            System.out.println("Protocol: " + url.getProtocol());

            System.out.println("Host: " + url.getHost());

            System.out.println("Port: " + url.getPort());

            System.out.println("Path: " + url.getPath());

            // Extract and display query parameters

            String query = url.getQuery();

            System.out.println("Query Parameters:");

            if (query != null) {

                String[] parameters = query.split("&");

                Map<String, String> queryParameters = new HashMap<>();
```

```

        for (String parameter : parameters) {
            String[] pair = parameter.split("=", 2);

            if (pair.length == 2) {
                queryParameters.put(pair[0], pair[1]);
            }
        }

        for (Map.Entry<String, String> entry : queryParameters.entrySet()) {
            System.out.println(
                entry.getKey() + " = " + entry.getValue()
            );
        }
    }

    // Display fragment
    System.out.println("Fragment: " + url.getRef());

} catch (Exception e) {
    System.out.println("Error parsing URL: " + e.getMessage());
}
}

```

Expected output

Protocol: https

Host: api.techstore.com

Port: 8443

Path: /products/category/electronics

Query Parameters:

sort = price

limit = 20

Fragment: reviews

QUESTION TWO – 8 MARKS

(a) PreparedStatement versus Statement

PreparedStatement is more secure and efficient than Statement, especially when handling login credentials.

Security

A normal Statement can be vulnerable to **SQL injection** when user input is directly inserted into an SQL statement.

For example:

```
String sql = "SELECT * FROM students WHERE reg_no = '"  
    + regNo + "' AND password = '" + password + "'";
```

A malicious user could enter SQL commands as part of the input.

PreparedStatement separates SQL code from user-supplied values. Values are treated as data rather than executable SQL commands.

Efficiency

A prepared statement can be precompiled by the database and reused with different values. This can improve performance when the same SQL statement is executed multiple times.

PreparedStatement authentication example

```
import java.sql.*;
```

```
public class StudentLogin {
```

```
    public static boolean authenticate(String regNo, String password) {
```

```
        String url = "jdbc:mysql://localhost:3306/university_db";
```

```
        String username = "admin";
```

```
        String dbPassword = "secure123";
```

```
        String sql = "SELECT * FROM students "
```

```
            + "WHERE registration_number = ? AND password = ?";
```

```

try (
    Connection connection =
        DriverManager.getConnection(url, username, dbPassword);

    PreparedStatement statement =
        connection.prepareStatement(sql)
) {

    // Set values safely
    statement.setString(1, regNo);
    statement.setString(2, password);

    try (ResultSet resultSet = statement.executeQuery()) {

        return resultSet.next();
    }

} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
    return false;
}
}

public static void main(String[] args) {

    String registrationNumber = "BIT/001/2026";
    String password = "mypassword";

    if (authenticate(registrationNumber, password)) {
        System.out.println("Login successful.");
    }
}

```



```

    } else {
        System.out.println("Invalid registration number or password.");
    }
}
}

```

Note: In a real production application, passwords should not be stored as plain text. They should be securely hashed using a password-hashing algorithm such as BCrypt or Argon2.

QUESTION TWO (b) – UniversityDatabase.java

The program below:

9. Connects to university_db.
10. Creates the courses table.
11. Inserts three courses.
12. Retrieves courses belonging to the Computer Science department.

UniversityDatabase.java

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class UniversityDatabase {

    // Database connection details
    private static final String URL =
        "jdbc:mysql://localhost:3306/university_db";

    private static final String USER = "admin";
    private static final String PASSWORD = "secure123";

```

```
public static void main(String[] args) {

    // SQL statement for creating the courses table
    String createTableSQL = ""

        CREATE TABLE IF NOT EXISTS courses (

            course_id INT PRIMARY KEY,

            course_name VARCHAR(100),

            credits INT,

            department VARCHAR(50)

        )

        """;

    // SQL statement for inserting courses
    String insertSQL = ""

        INSERT INTO courses

        (course_id, course_name, credits, department)

        VALUES (?, ?, ?, ?)

        """;

    // SQL statement for retrieving Computer Science courses
    String selectSQL =

        "SELECT * FROM courses WHERE department = ?";

    try (Connection connection =

        DriverManager.getConnection(URL, USER, PASSWORD)) {

        System.out.println("Database connected successfully.");

        // Create table
        try (Statement statement = connection.createStatement()) {
```

```
statement.executeUpdate(createTableSQL);

System.out.println("Courses table created successfully.");
}

// Insert sample courses
try (PreparedStatement statement =
    connection.prepareStatement(insertSQL)) {

    insertCourse(statement, 101,
        "Advanced Programming", 3,
        "Computer Science");

    insertCourse(statement, 102,
        "Database Systems", 3,
        "Computer Science");

    insertCourse(statement, 103,
        "Computer Networks", 3,
        "Computer Science");

    System.out.println("Sample courses inserted.");
}

// Retrieve Computer Science courses
try (PreparedStatement statement =
    connection.prepareStatement(selectSQL)) {

    statement.setString(1, "Computer Science");

    try (ResultSet resultSet = statement.executeQuery()) {
```

```

        System.out.println("\nComputer Science Courses:");

        while (resultSet.next()) {

            System.out.println(
                "ID: " + resultSet.getInt("course_id")
                + ", Name: " + resultSet.getString("course_name")
                + ", Credits: " + resultSet.getInt("credits")
                + ", Department: "
                + resultSet.getString("department")
            );
        }
    }

} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}

}

// Method used to insert one course
private static void insertCourse(
    PreparedStatement statement,
    int id,
    String name,
    int credits,
    String department) throws SQLException {

    statement.setInt(1, id);
    statement.setString(2, name);

```

```

        statement.setInt(3, credits);

        statement.setString(4, department);

        statement.executeUpdate();
    }
}

```

Required JDBC dependency

For the above version, the project uses **MySQL Connector/J**.

The database should be created before running the program:

```
CREATE DATABASE university_db;
```

QUESTION THREE – 8 MARKS

(a) Servlets versus JSP

Servlet

A Servlet is a Java class that runs on a web server and handles HTTP requests and responses.

Servlets are mainly suitable for:

- Processing requests.
- Performing business logic.
- Calling services and databases.
- Controlling application flow.

JSP

JSP stands for **JavaServer Pages**. It is mainly used to create the presentation layer of a Java web application.

JSP makes it easier to create HTML pages containing dynamic information.

Comparison

Feature	Servlet	JSP
Main purpose	Request processing and control	Presentation/UI
Development	More Java code	Easier for HTML/UI
Best MVC role	Controller	View
Suitable for	Business/request processing	Displaying dynamic content

Feature	Servlet	JSP
HTML creation	More difficult	Easier

In an MVC application, the **Servlet normally acts as the Controller**, while the **JSP acts as the View**.

When to choose a Servlet

I would choose a Servlet when the application needs to process a request, validate data, call a DAO, perform business logic, and determine which page should be displayed.

When to choose JSP

I would choose JSP when the main requirement is displaying dynamic information to the user, such as displaying a list of products or student records.

QUESTION THREE (b) – Shopping Cart Session Management

Likely cause

The most likely cause is that the shopping cart is not being stored in the user's HTTP session.

HTTP is fundamentally **stateless**, meaning the server does not automatically remember information from one request to another.

For example, if a customer adds a product during one request and the cart is stored only in a local variable, that information can be lost when the customer makes another request.

Two session-management techniques

1. HttpSession

HttpSession can be used to store the cart on the server for a particular user session.

Example:

```
HttpSession session = request.getSession();
```

```
List<String> cart =
```

```
    (List<String>) session.getAttribute("cart");
```

```
if (cart == null) {
```

```
    cart = new ArrayList<>();
```

```
}
```

```
cart.add("Laptop");
```

```
session.setAttribute("cart", cart);
```

2. Cookies

Cookies can store a small amount of information on the user's browser. A session identifier can also be stored in a cookie so that the server can associate subsequent requests with the correct session.

For a shopping cart, server-side HttpSession is generally preferable for storing the actual cart contents.

ShoppingCartServlet.java

```
package com.bit2203.servlet;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;

@WebServlet("/shopping-cart")
public class ShoppingCartServlet extends HttpServlet {

    private static final String CART_ATTRIBUTE = "cart";

    // Handles GET requests and displays the cart
    @Override
    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response)
```

```
throws ServletException, IOException {

    List<String> cart = getCart(request);

    response.setContentType("text/html");

    PrintWriter out = response.getWriter();

    out.println("<html>");
    out.println("<body>");
    out.println("<h1>Shopping Cart</h1>");

    if (cart.isEmpty()) {

        out.println("<p>Your cart is empty.</p>");

    } else {

        out.println("<ul>");

        for (String item : cart) {
            out.println("<li>" + item + "</li>");
        }

        out.println("</ul>");
    }

    out.println("</body>");
    out.println("</html>");
}
```



```

// Handles POST requests for adding items
@Override
protected void doPost(
    HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {

    String item = request.getParameter("item");

    if (item != null && !item.trim().isEmpty()) {

        HttpSession session = request.getSession();

        List<String> cart = getCart(request);

        cart.add(item);

        session.setAttribute(CART_ATTRIBUTE, cart);
    }

    response.sendRedirect("shopping-cart");
}

// Retrieves the cart from the current user's session
@SuppressWarnings("unchecked")
private List<String> getCart(HttpServletRequest request) {

    HttpSession session = request.getSession();

    List<String> cart =
        (List<String>) session.getAttribute(CART_ATTRIBUTE);

```

```
// Create a new cart if one does not exist
if (cart == null) {

    cart = new ArrayList<>();

    session.setAttribute(CART_ATTRIBUTE, cart);
}

return cart;
}
}
```

QUESTION FOUR – 8 MARKS

(a) Microservices Architecture

Microservices architecture is a software architecture in which a large application is divided into several small, independent services.

Each service performs a specific business function and can communicate with other services through APIs or messaging systems.

For example, an e-commerce system could contain:

- User Service
- Product Service
- Payment Service
- Order Service
- Delivery Service

Each service can be developed, deployed, and scaled independently.

Benefits

1. Independent deployment

A single service can be updated or deployed without necessarily redeploying the entire application.

2. Scalability

Individual services can be scaled according to demand.

For example, during a busy shopping period, the Order Service can be given additional resources without scaling every other service.

Challenges

1. Increased complexity

A microservices system contains many services, databases, APIs, and communication channels, making the system more difficult to manage.

2. Network and communication problems

Services communicate over networks, so problems such as latency, network failures, and service unavailability can occur.

Real-world example

Netflix is a well-known example of a company that uses microservices architecture. Its large platform is divided into numerous independent services that support different functions of the streaming platform.

QUESTION FOUR (b) – Dependency Injection

Dependency Injection (DI) is a design technique where an object's dependencies are supplied to it from outside instead of the object creating those dependencies itself.

For example, instead of UserService creating a UserRepository using new, Spring provides the repository to the service.

Benefits of DI

- Reduces coupling.
- Makes applications easier to test.
- Makes components easier to replace.
- Improves maintainability.
- Supports clean application architecture.

Constructor Injection

In constructor injection, dependencies are supplied through the class constructor.

Advantage

The dependency can be made final, ensuring that the object cannot exist without its required dependency.

Setter Injection

Setter injection supplies dependencies through setter methods.

Advantage

It is useful when a dependency is optional or needs to be changed after object creation.

Comparison

Constructor Injection

Dependency supplied through constructor

Good for required dependencies

Supports immutable dependencies

Makes dependencies obvious

Setter Injection

Dependency supplied through setter

Good for optional dependencies

Dependency can be changed

More flexible after construction

UserRepository.java

```
package com.bit2203.spring;
```

```
import org.springframework.stereotype.Repository;
```

```
@Repository
```

```
public class UserRepository {
```

```
    // Example repository method
```

```
    public String findUserById(int userId) {
```

```
        return "User with ID: " + userId;
```

```
    }
```

```
}
```

UserService.java

```
package com.bit2203.spring;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class UserService {
```

```
private final UserRepository userRepository;

// Constructor injection
public UserService(UserRepository userRepository) {
    this.userRepository = userRepository;
}

// Service method
public String getUser(int userId) {

    return userRepository.findUserById(userId);
}
}
```

AppConfig.java

```
package com.bit2203.spring;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.bit2203.spring")
public class AppConfig {

    // Spring automatically detects
    // @Service and @Repository classes.
}
```

QUESTION FIVE – 6 MARKS

(a) Remote Method Invocation (RMI)

Java RMI, or **Remote Method Invocation**, is a Java technology that allows an object running in one Java Virtual Machine to invoke methods on an object running in another Java Virtual Machine.

RMI allows Java applications running on different computers or processes to communicate as though they were calling methods on local objects.

RMI Registry

The RMI Registry is a naming service.

It stores references to remote objects using names so that clients can locate those objects.

Server

The server:

9. Implements the remote interface.
10. Creates the remote object.
11. Exports the object.
12. Registers the object with the RMI Registry.

Client

The client:

9. Connects to the RMI Registry.
10. Looks up the remote object by name.
11. Obtains a reference to the remote service.
12. Invokes methods on the remote object.

QUESTION FIVE (b) – Java RMI Program

TemperatureConverter.java

```
import java.rmi.Remote;
```

```
import java.rmi.RemoteException;
```

```
// Remote interface containing methods
```

```
// that can be called by the client.
```

```
public interface TemperatureConverter extends Remote {
```

```
    double celsiusToFahrenheit(double celsius)
```

```
        throws RemoteException;
```

```
        double fahrenheitToCelsius(double fahrenheit)
            throws RemoteException;
    }
```

TemperatureConverterImpl.java

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

// Implementation of the remote interface
public class TemperatureConverterImpl
    extends UnicastRemoteObject
    implements TemperatureConverter {

    // Constructor
    public TemperatureConverterImpl()
        throws RemoteException {
        super();
    }

    // Converts Celsius to Fahrenheit
    @Override
    public double celsiusToFahrenheit(double celsius)
        throws RemoteException {

        return (celsius * 9 / 5) + 32;
    }

    // Converts Fahrenheit to Celsius
    @Override
    public double fahrenheitToCelsius(double fahrenheit)
        throws RemoteException {
```

```
        return (fahrenheit - 32) * 5 / 9;
    }
}
```

RMI Server.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

// RMI server
public class RMIServer {

    public static void main(String[] args) {

        try {
            // Create the remote service
            TemperatureConverter converter =
                new TemperatureConverterImpl();

            // Start the RMI registry on port 1099
            Registry registry =
                LocateRegistry.createRegistry(1099);

            // Bind the remote object to a name
            registry.rebind(
                "TemperatureConverter",
                converter
            );

            System.out.println(
                "Temperature Converter RMI Server is running..."
            );
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



```

    );

    } catch (Exception e) {

        System.out.println(
            "Server error: " + e.getMessage()
        );

        e.printStackTrace();
    }
}
}

```

RMIClient.java

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

// RMI client
public class RMIClient {

    public static void main(String[] args) {

        try {
            // Locate the RMI registry
            Registry registry =
                LocateRegistry.getRegistry(
                    "localhost", 1099
                );

            // Look up the remote service
            TemperatureConverter converter =

```

```

        (TemperatureConverter)
        registry.lookup("TemperatureConverter");

    // Convert 25 degrees Celsius
    double result =
        converter.celsiusToFahrenheit(25);

    System.out.println(
        "25°C = " + result + "°F"
    );

} catch (Exception e) {

    System.out.println(
        "Client error: " + e.getMessage()
    );

    e.printStackTrace();
}
}
}

```

Expected output

25°C = 77.0°F

QUESTION SIX – 6 MARKS

(a) Web Container

A **web container**, also called a Servlet container, is a component of a Java web server that manages the execution of Java Servlets.

Examples include Apache Tomcat, Jetty, and other Jakarta Servlet-compatible servers.

Four responsibilities

9. Servlet lifecycle management

The container loads, initializes, executes, and destroys Servlets.

10. Request and response management

It receives HTTP requests and provides them to the appropriate Servlet.

11. Session management

It manages HTTP sessions and helps maintain user state.

12. Security and resource management

It can provide authentication, authorization, URL mapping, and manage Servlet resources.

QUESTION SIX (b) – Servlet Lifecycle

The main stages of a Servlet lifecycle are:

1. Loading and Instantiation

The web container loads the Servlet class and creates an instance of it.

2. Initialization – init()

The container calls the init() method once.

This is used for initialization tasks such as loading configuration information or resources.

3. Request Processing – service()

For every client request, the container calls the service() method.

For an HttpServlet, the service() method determines whether the request is GET, POST, PUT, DELETE, etc., and calls methods such as doGet() or doPost().

4. Destruction – destroy()

When the Servlet is removed from service, the container calls destroy() once.

This is used to release resources.

LifecycleDemoServlet.java

```
package com.bit2203.servlet;
```

```
import jakarta.servlet.ServletException;
```

```
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.io.IOException;
```

```
import java.io.PrintWriter;

@WebServlet("/lifecycle")
public class LifecycleDemoServlet extends HttpServlet {

    // Called once when the Servlet is initialized
    @Override
    public void init() throws ServletException {

        System.out.println(
            "LifecycleDemoServlet: init() called."
        );
    }

    // Called for each HTTP request
    @Override
    protected void service(
        HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        System.out.println(
            "LifecycleDemoServlet: service() called."
        );

        // Pass the request to the parent implementation.
        super.service(request, response);
    }

    // Handles GET requests
    @Override
```

```

protected void doGet(
    HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {

    System.out.println(
        "LifecycleDemoServlet: doGet() called."
    );

    response.setContentType("text/html");

    PrintWriter out = response.getWriter();

    out.println("<h1>Servlet Lifecycle Demo</h1>");
    out.println("<p>The Servlet is processing the request.</p>");
}

// Called once before the Servlet is destroyed
@Override
public void destroy() {

    System.out.println(
        "LifecycleDemoServlet: destroy() called."
    );
}
}

```

Lifecycle sequence

Load Servlet



Create Servlet object



init()



service()



doGet() / doPost()



service()



doGet() / doPost()



destroy()

QUESTION SEVEN – 8 MARKS

(a) Model-View-Controller (MVC)

MVC stands for **Model-View-Controller**.

It is a software design pattern that separates an application into three main components.

1. Model

The Model represents the application's data and business logic.

Responsibilities include:

- Representing data.
- Applying business rules.
- Communicating with databases.
- Managing application data.

Example:

Book

Student

Product

User

2. View

The View is the presentation layer.

It is responsible for displaying information to the user.

Examples include:

- JSP pages.
- HTML pages.
- User interfaces.

3. Controller

The Controller receives requests from the user and determines what should happen.

Responsibilities include:

- Receiving HTTP requests.
- Processing input.
- Calling the Model/DAO.
- Selecting the appropriate View.
- Passing data to the View.

MVC flow

User



Controller



Model / DAO



Database



Model / DAO



Controller



View



User

Real-world example

An **online shopping system** can use MVC.

For example:

- **Model:** Product, Order and Customer classes.
- **View:** JSP pages displaying products.
- **Controller:** Servlets handling product and order requests.
- **Database:** Stores product, customer and order information.

Many enterprise Java web applications use the MVC approach because it separates presentation, processing, and data management.

QUESTION SEVEN (b) – Library Management System

Model

The Model represents library data.

For this system, the Book class represents individual books.

It can contain:

- Book ID.
- Title.
- Author.
- ISBN.

DAO

The Data Access Object, BookDAO, communicates with the database and retrieves books.

Controller

BookController is a Servlet that:

11. Receives the request.
12. Calls BookDAO.
13. Retrieves the books.
14. Places them into the request.
15. Forwards the request to the JSP.

View

bookList.jsp displays the books to the user.

Book.java

```
package com.bit2203.library.model;
```



```
// Model class representing a book

public class Book {

    private int id;

    private String title;

    private String author;

    private String isbn;

    // Constructor

    public Book(int id, String title, String author, String isbn) {

        this.id = id;

        this.title = title;

        this.author = author;

        this.isbn = isbn;

    }

    // Getters

    public int getId() {

        return id;

    }

    public String getTitle() {

        return title;

    }

    public String getAuthor() {

        return author;

    }

    public String getIsbn() {

        return isbn;

    }

}
```

```
}

// Setters
public void setId(int id) {
    this.id = id;
}

public void setTitle(String title) {
    this.title = title;
}

public void setAuthor(String author) {
    this.author = author;
}

public void setIsbn(String isbn) {
    this.isbn = isbn;
}
}
```

BookDAO.java

```
package com.bit2203.library.dao;

import com.bit2203.library.model.Book;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

```
import java.util.ArrayList;

import java.util.List;


// DAO responsible for accessing book data
public class BookDAO {


    private static final String URL =

        "jdbc:mysql://localhost:3306/library_db";


    private static final String USER = "admin";
    private static final String PASSWORD = "secure123";


    // Retrieves all books from the database
    public List<Book> getAllBooks() {


        List<Book> books = new ArrayList<>();


        String sql =

            "SELECT id, title, author, isbn FROM books";


        try (

            Connection connection =

                DriverManager.getConnection(

                    URL, USER, PASSWORD);


            PreparedStatement statement =

                connection.prepareStatement(sql);


            ResultSet resultSet =

                statement.executeQuery()

        ) {
```

```

        while (resultSet.next()) {

            Book book = new Book(
                resultSet.getInt("id"),
                resultSet.getString("title"),
                resultSet.getString("author"),
                resultSet.getString("isbn")
            );

            books.add(book);
        }

    } catch (SQLException e) {

        System.out.println(
            "Database error: " + e.getMessage()
        );
    }

    return books;
}
}

```

BookController.java

```

package com.bit2203.library.controller;

import com.bit2203.library.dao.BookDAO;
import com.bit2203.library.model.Book;

import jakarta.servlet.ServletException;

```

```
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;
import java.util.List;

// Controller Servlet for handling book requests
@WebServlet("/books")
public class BookController extends HttpServlet {

    private BookDAO bookDAO;

    // Create DAO when Servlet is initialized
    @Override
    public void init() {

        bookDAO = new BookDAO();
    }

    // Handles request for displaying all books
    @Override
    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        // Retrieve books from DAO
        List<Book> books = bookDAO.getAllBooks();
```

```
// Place books in request scope
request.setAttribute("books", books);

// Forward request to JSP View
request.getRequestDispatcher(
    "/bookList.jsp"
).forward(request, response);
}
}
```

bookList.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ page import="java.util.List" %>
<%@ page import="com.bit2203.library.model.Book" %>

<!DOCTYPE html>

<html>
<head>
    <title>Library Books</title>
</head>

<body>

<h1>Library Books</h1>

<%
    List<Book> books =
        (List<Book>) request.getAttribute("books");
%>

<table border="1" cellpadding="8">
```

```
<tr>
    <th>ID</th>
    <th>Title</th>
    <th>Author</th>
    <th>ISBN</th>
</tr>
```

```
<%
    if (books != null && !books.isEmpty()) {

        for (Book book : books) {
%>
```

```
<tr>
    <td><%= book.getId() %></td>
    <td><%= book.getTitle() %></td>
    <td><%= book.getAuthor() %></td>
    <td><%= book.getIsbn() %></td>
</tr>
```

```
<%
    }
```

```
    } else {
%>
```

```
<tr>
    <td colspan="4">
        No books found.
    </td>
```

</tr>

<%

}

%>

</table>

</body>

</html>

BONUS / EXTRA CREDIT – 2 MARKS

Inversion of Control (IoC)

Inversion of Control (IoC) is a principle in which the responsibility for creating and managing objects is transferred from the application code to a framework or container.

In traditional programming, a class may create its own dependencies:

```
UserRepository repository = new UserRepository();
```

With Spring IoC, the Spring container creates and manages the object and supplies its dependencies.

Relationship between IoC and DI

Dependency Injection is one of the main techniques used to implement IoC.

For example:

```
@Service
```

```
public class UserService {
```

```
    private final UserRepository repository;
```

```
    public UserService(UserRepository repository) {
```

```
        this.repository = repository;
```

```
    }
```

```
}
```

Spring creates UserRepository and injects it into UserService.

Importance of IoC

IoC is important because it:

- Reduces coupling.
- Improves testability.
- Makes components reusable.
- Simplifies object management.
- Makes large enterprise applications easier to maintain.

Therefore, IoC is a fundamental principle in modern enterprise Java development because frameworks such as Spring can manage object creation, dependencies, configuration, and application components.

ADDITIONAL QUESTIONS FOR EXTRA PRACTICE

1. Stateful versus Stateless Session Management

Stateful session management

Stateful session management means the server maintains information about a user's previous interactions.

Example: An online shopping cart where the server stores the user's cart in an HttpSession.

Stateless session management

Stateless communication means each request contains enough information for the server to process it without depending on previously stored server-side state.

Example: REST APIs using tokens such as JWTs for authentication.

Comparison

Stateful	Stateless
Server maintains user state	Server does not maintain session state
Uses more server resources	Easier to scale horizontally
Example: HttpSession shopping cart	Example: Token-based REST API

2. JAR FILE

JAR stands for **Java Archive**.

A JAR file packages Java classes, resources, configuration files, and metadata into one compressed file.

For example:

MyApplication.jar

A JAR can be made executable by specifying the application's main class in its manifest.

Example MANIFEST.MF

Manifest-Version: 1.0

Main-Class: com.example.Main

The JAR can then be executed using:

```
java -jar MyApplication.jar
```

3. MONOLITHIC VERSUS MICROSERVICES

A monolithic application is built as one large application where the components are packaged and deployed together.

Microservices architecture divides the application into multiple smaller and independently deployable services.

Monolithic

One large application

Usually deployed together

Simpler initially

Scaling may involve the whole application

Easier for small applications

Microservices

Multiple smaller services

Services can be deployed independently

More complex to manage

Individual services can be scaled

Suitable for large and complex systems

When to choose monolithic

A monolithic architecture is suitable when:

- The application is small.
- The development team is small.
- Simplicity is important.
- The application does not require independent scaling.

When to choose microservices

Microservices are suitable when:

- The application is large.
- Different teams need to work independently.
- Different components need independent scaling.
- Independent deployment is important.

GITHUB REPOSITORY STRUCTURE

Your repository should be named exactly:

BIT2203-AdvancedProgramming-CAT

A suitable structure is:

BIT2203-AdvancedProgramming-CAT/

```
|
|
| └─ README.md
|
|
| └─ src/
|   |
|   |
|   | └─ question1/
|   |   └─ URLParser.java
|   |
|   |
|   | └─ question2/
|   |   └─ StudentLogin.java
|   |   └─ UniversityDatabase.java
|   |
|   |
|   | └─ question3/
|   |   └─ ShoppingCartServlet.java
|   |
|   |
|   | └─ question4/
|   |   └─ UserRepository.java
|   |   └─ UserService.java
|   |   └─ AppConfig.java
|   |
|   |
|   | └─ question5/
|   |   └─ TemperatureConverter.java
|   |   └─ TemperatureConverterImpl.java
|   |   └─ RMIServer.java
|   |   └─ RMIClient.java
```

```
| |
| └─ question6/
| |   └─ LifecycleDemoServlet.java
| |
| |
| └─ question7/
|   └─ Book.java
|   └─ BookDAO.java
|   └─ BookController.java
|
└─ web/
    └─ bookList.jsp
```

README.md

Your repository should contain a README explaining the programs.

BIT2203-AdvancedProgramming-CAT

Student Information

Name: YOUR NAME

Registration Number: YOUR REGISTRATION NUMBER

Description

This repository contains Java programs developed for the BIT 2203

Advanced Programming Take-Away CAT.

Programs

Question 1 - URLParser

URLParser.java demonstrates how Java's URL class can be used to extract URL components including the protocol, host, port, path, query parameters and fragment.

Question 2 - JDBC

StudentLogin.java demonstrates secure student authentication using PreparedStatement.

UniversityDatabase.java connects to the university_db database, creates the courses table, inserts sample courses and retrieves Computer Science courses.

Question 3 - Shopping Cart

ShoppingCartServlet.java demonstrates HTTP session management using HttpSession to maintain a customer's shopping cart.

Question 4 - Spring Dependency Injection

UserRepository.java provides the repository dependency.

UserService.java demonstrates constructor dependency injection.

AppConfig.java configures component scanning in Spring.

Question 5 - Java RMI

TemperatureConverter.java is the remote interface.

TemperatureConverterImpl.java implements the remote service.

RMIServer.java starts the RMI registry and binds the service.

RMIClient.java looks up the service and converts 25 degrees Celsius to Fahrenheit.

Question 6 - Servlet Lifecycle

LifecycleDemoServlet.java demonstrates the Servlet lifecycle methods including init(), service(), doGet() and destroy().

Question 7 - MVC Library System

Book.java represents the Model.

BookDAO.java accesses book data.

BookController.java acts as the Servlet Controller.

bookList.jsp acts as the View and displays the books.

Requirements

- Java JDK
- MySQL Server
- MySQL Connector/J
- Apache Tomcat 10 or later for Jakarta Servlet applications
- Spring Framework for the Spring example
- An IDE such as IntelliJ IDEA, Eclipse or NetBeans

Database Setup

Create the required database using:

```
CREATE DATABASE university_db;
```

For the library example:

```
CREATE DATABASE library_db;
```

Update database credentials in the Java files if necessary.

Running the RMI Program

Compile the RMI classes:

```
javac *.java
```

Start the server:

```
java RMIServer
```

In another terminal, run:

```
java RMIClient
```

Expected result:

25°C = 77.0°F

Servlet Programs

Deploy the Servlet project to a Jakarta Servlet-compatible web container such as Apache Tomcat 10 or later.

GitHub

All Java source files and configuration files are included in this repository.

FINAL SUBMISSION CHECKLIST

Before submitting, ensure you have:

- ☐ Created a **public** GitHub repository.
- ☐ Named it BIT2203-AdvancedProgramming-CAT.
- ☐ Added all required .java files.
- ☐ Added README.md.
- ☐ Added the JSP file.
- ☐ Added configuration files where applicable.
- ☐ Added comments to your Java programs.
- ☐ Tested the programs.
- ☐ Pushed all files to GitHub.
- ☐ Added your real name to the document.
- ☐ Added your registration number.
- ☐ Added your GitHub repository link.
- ☐ Saved the typed answers as PDF or Word.
- ☐ Checked that the GitHub repository is **public**.
- ☐ Verified that the GitHub link actually opens before submission.

Required Java files

URLParser.java

StudentLogin.java

UniversityDatabase.java

ShoppingCartServlet.java

UserRepository.java

UserService.java

AppConfig.java

TemperatureConverter.java

TemperatureConverterImpl.java

RMI Server.java

RMI Client.java

LifecycleDemoServlet.java

Book.java

BookDAO.java

BookController.java

bookList.jsp

README.md

Important implementation note: The Servlet examples above use jakarta.servlet.*, which is appropriate for modern Servlet containers such as Tomcat 10+. If your lecturer/lab uses Tomcat 9 or an older Java EE setup, the imports may need to use javax.servlet.* instead.